Deep Learning for Biological Discovery Across Omics Layers: From Phenomics to Agentic Reasoning

Dissertation Proposal

The University of Tennessee, Knoxville

Peter Kruse

June 2026

Contents

List of Tables

List of Figures

Executive Summary

0.1 Motivation

Modern biology has become increasingly data-rich but interpretation-limited. Data collection is now cost-effective in many realms, enabling the construction of novel datasets. High-throughput imaging technology has led to population-scale phenotyping datasets, while advanced sequencing methods allow researchers to capture genome-scale variation across populations. Global consortia efforts have standardized environmental metadata across climates and sites, while novel experimental data and computational frameworks have prompted the curation and creation of biological networks and large literature corpora. These data span fundamentally different structures: images contain spatial, pixel-level information; genotypes comprise high-dimensional, sparse signals; environments exist in a continuous and contextual space; and networks represent relational graph structures. Handling these different structures requires systems with different inductive biases; no single model class can handle them all.

0.2 Thesis Statement

While progress has been made applying deep learning to localized areas, biological data are heterogeneous and structurally varied, making the gains from current deep learning methods unequally distributed. Because of the nature of biological data,

bridging the gap between inputs and interpretation requires problem-specific systems. Architectures, objectives, and benchmarks must be designed with appropriate inductive biases. This proposal demonstrates the utility of this framework across three linked stages of biological discovery: *measurement*, *prediction*, and *interpretation.*

0.3 Chapter 2 Summary

Chapter 2 addresses the *measurement* stage by extracting quantitative phenotypes from raw imagery. No deep learning framework currently exists for phenotyping pennycress (*Thlaspi arvense L.*), an emerging oilseed cover crop. In this work, we present an automated deep learning pipeline for intensive pennycress seed pod phenotyping, extracting over 50 phenotypes per image. We implemented U-Net, a state-of-the-art deep learning model, to semantically segment wing, envelope, and seed pixels in seed pod images, and we created an open-source ground truth dataset of 365 hand-annotated label images for training and validation. Our model achieved over 96% accuracy for wing segmentation, 95% accuracy for envelope segmentation, and almost 80% accuracy for seed segmentation. We further validated the model through a population-scale study on over 11,000 seed pods from 771 unique genotypes.

0.4 Chapter 3 Summary

Chapter 3 addresses the *prediction* stage by mapping genotype and environment to phenotype. Accurate maize yield prediction is crucial for global food security, but remains challenging due to complex genotype-by-environment (G×E) interactions. In this work, we present a unified transformer architecture for maize yield prediction that tokenizes single-nucleotide polymorphism (SNP) markers and environmental covariates into a single input sequence, learning G×E interactions through self-attention. Mixture-of-experts (MoE) layers with Top-K routing replace the dense feed-forward blocks, allowing the model to specialize across genetic and environmental

contexts. We benchmark this architecture against a suite of variants, including a three-prong encoder-fusion design with dedicated genotype, LD, and environment branches. Training on the publicly available Genomes2Fields (G2F) prediction competition dataset comprising 21 environments over 11 years, we achieve a within-environment Pearson correlation coefficient (PCC) of 0.423 on the 2024 test set, approaching the competition-winning score of 0.45 and exceeding the strongest previously published deep learning baselines we evaluated.

0.5 Chapter 4 Summary

Chapter 4 addresses the *interpretation* stage by reasoning over biological networks to produce evidence-grounded mechanistic hypotheses. While expert-curated corpora and AI-based methods have produced rich relational structures at scale, downstream synthesis remains constrained by human bandwidth and model cost. We propose MENTOR-RL, an open-source, tool-using reasoning agent trained with reinforcement learning over verifiable biological-network tasks. MENTOR-RL consists of a single policy operating in an act-verify agent loop and is trained with deterministic, schema-grounded rewards that require no proprietary judge models. We expect MENTOR-RL to achieve a composite F_1 score competitive with frontier models at a substantially lower per-query inference cost, while producing human-readable interpretations with preserved provenance.

0.6 Completion Plan and Expected Outputs

Chapters 2 and 3 are in preparation for submission to [venue] and [venue], respectively; Chapter 4 is proposed work targeting [venue/timeline]. Models in Chapters 3 and 4 are trained at scale using distributed training on the OLCF Frontier supercomputer [3]. Upon publication, we will release the open-source ground truth dataset of hand-annotated seed pod images and segmentation pipeline (Chapter 2); the training and

evaluation code (Chapter 3); and the training environment, evaluation harness, and trained checkpoints across pipeline stages (Chapter 4).

Chapter 1

Literature Review

1.1 Deep Learning History

According to Goodfellow et al., the history of neural network research can be divided into three eras: cybernetics, connectionism, and deep learning [51]. In the following sections, we adopt this organizational structure to explore the development of deep learning.

1.1.1 Cybernetics

The cybernetics era, spanning the 1940s and 1960s, was characterized by early efforts to formalize neural computation through the mathematical modeling of biological nervous systems. The first foundational contribution from this period was presented by McCulloch and Pitts in *A Logical Calculus of Ideas Immanent in Nervous Activity* [106]. The authors provided a modeling framework for neuronal activity through the lens of formal logic; each neuron was framed as a binary computational unit within a larger symbolic logic network. Neuronal activation was determined by inputs corresponding to excitation and inhibition; an output was produced when the aggregated inputs exceeded a fixed threshold. Neurons emitted discrete logical outputs $True$ or $False$, enabling the construction of networks that implemented

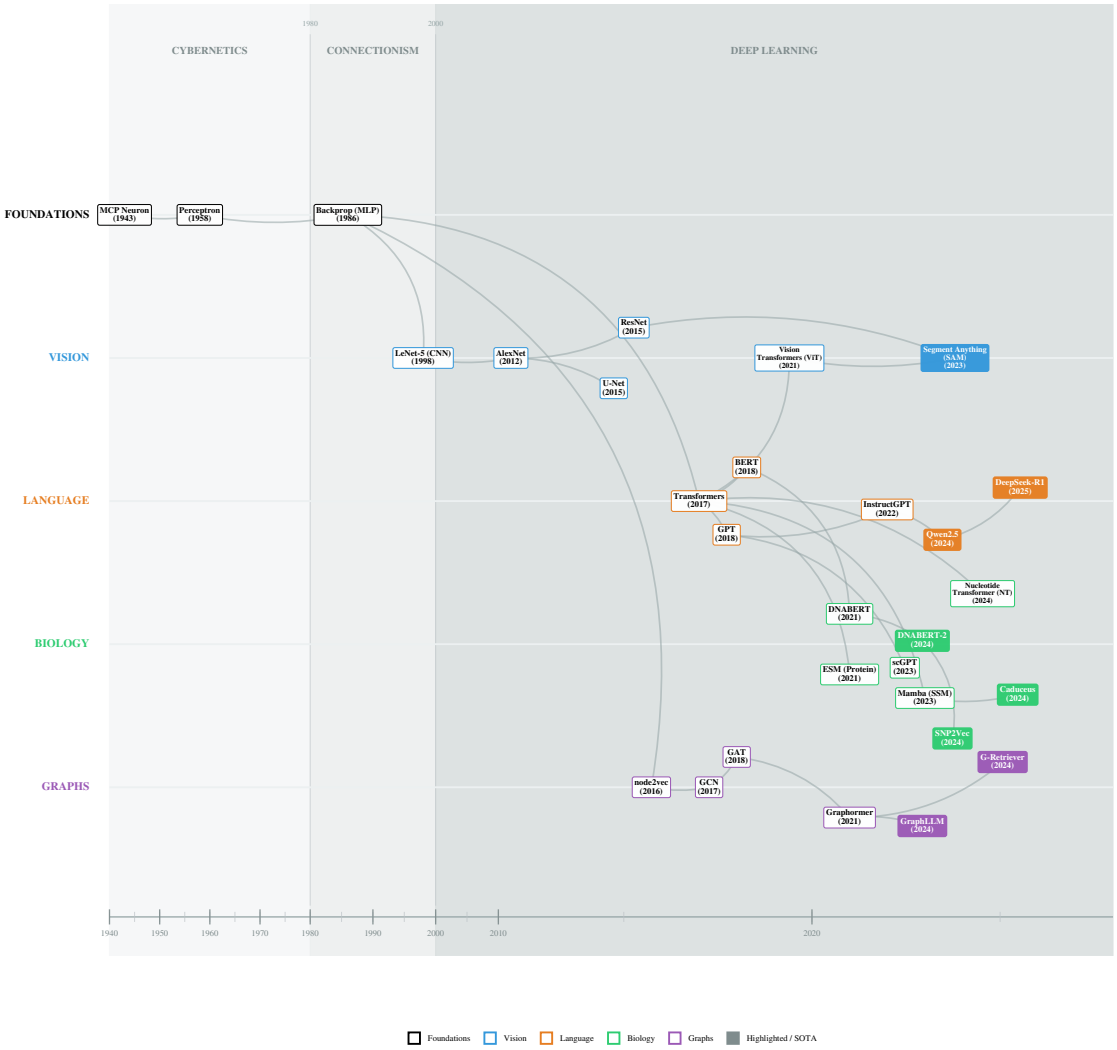

Figure 1.1: History of Deep Learning (1940-2026)

logical functions. Through this connected framework, McCulloch and Pitts aimed to model neural circuitry in the brain. Importantly, they emphasized that these networks worked over discrete time steps t, allowing them to represent sequential dynamics and foreshadowing later concepts like recurrence and memory in neural systems.

Despite establishing a foundational framework for neural computation, the McCulloch-Pitts model exhibited several fundamental limitations. Most notably, the formulation lacked a learning mechanism: network structure and parameters were fixed, requiring manual design rather than learning from data. This static framework stood in contrast to Donald Hebb's work on synaptic plasticity, which postulated that neural pathways are strengthened by repeated co-activation [59]. Without such a mechanism, the behavior of the McCulloch-Pitts networks could not be modified in response to novel stimuli, limiting this framework's utility for modeling adaptive biological or cognitive processes.

Furthermore, relying on a purely logical formulation imposed strict representational constraints. Neurons were restricted to binary activation states, resulting in rigid decision boundaries and precluding the learning of continuous representations. Together, these limitations prevented the McCulloch-Pitts networks from scaling to tasks requiring robust function approximation and learning from noisy, high-dimensional data.

The limitations of the McCulloch-Pitts neuron motivated subsequent efforts to develop neural models capable of adaptation, most notably Frank Rosenblatt's 1958 work *The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain* [142]. Rosenblatt described the perceptron as "a hypothetical nervous system," reflecting an effort to maintain biological inspiration while introducing mechanisms for experience-driven learning. In contrast to the symbolic logic-based formulation of McCulloch and Pitts, Rosenblatt adopted a probabilistic perspective. He argued that the extensive assumptions and structure imposed by their model limited its ability to yield meaningful biological insight.

Instead, the perceptron implemented a paradigm that treated data as statistically separable, with a learning mechanism that allowed connections to be reinforced or discouraged via positive or negative stimuli from the data.

While innovations introduced by the perceptron, such as trainable neurons and learning from data, are central to modern deep learning models, the original model had significant shortcomings. Most notably, Rosenblatt's formulation was incapable of representing non-linearly separable functions, with the XOR problem serving as a canonical example. This limitation was a consequence of the perceptron's reliance on linear decision boundaries and its restriction to single-layer architectures, and it severely limited its expressivity. Additionally, Rosenblatt's training rules could not be extended to multi-layer networks due to the lack of a credit assignment mechanism across multiple layers of computation. Combined with the use of binary outputs, these constraints severely limited the functions and representations that could be learned.

These limitations were rigorously analyzed by Minsky and Papert in their 1969 book *Perceptrons*, whose formal results played a significant role in altering public perception of AI research and contributed to a temporary decline in funding.

1.1.2 Connectionism

Although Minsky and Papert's work provided a rigorous exploration of the limitations of single-layer perceptrons, their critique did not explore whether such constraints could be overcome by multi-layer networks equipped with a credit assignment mechanism. The subsequent era of neural network research, commonly referred to as *connectionism*, was defined by the efforts to address this challenge through the development of multi-layer architectures and the backpropagation algorithm. During this period neural networks also transitioned from primarily symbolic and biologically motivated abstractions toward increasingly general-purpose learning systems. As learning algorithms became more complex and models increased in scale, research

increasingly began to favor mathematical tractability over biological interpretability [144, 51].

In 1986, Rumelhart, Hinton, and Williams published *Learning Representations by Back-Propagating Errors* [144], demonstrating that multi-layer neural networks could be effectively trained through an general solution to the credit assignment problem. Although the mathematical foundations for efficiently computing gradients, known as reverse-mode automatic differentiation, had been previously derived by Seppo Linnainmaa [97] and applied to neural networks by Paul Werbos [187], Rumelhart, Hinton, and Williams popularized the method within the AI community.

The backpropagation algorithm is a gradient-based learning procedure that propagates error information from the network's output backward through successive layers, incrementally modifying each parameter. Using networks trained with backpropagation, the authors showed that non-linearly separable functions could be learned by multilayer networks without relying on hand-designed learning rules.

Backpropagation relies on the differentiability of both the loss function and the network's activation functions, which allow gradients with respect to all parameters to be computed efficiently via the chain rule. This formulation enabled learning signals to be propagated to all network layers, in contrast to earlier approaches like the perceptron, which required a manually specified learning procedure. The formulation of weight updates as an optimization problem allowed neural network training to be extended to more complex multilayer architectures. In addition to backpropagation, this work made several other contributions, such as an explicit learning rate, momentum-based updates, and gradient accumulation, all of which are core to modern neural network optimization.

Beyond their algorithmic contribution, Rumelhart, Hinton, and Williams provided a conceptual framework for understanding multilayer networks. They characterized networks as self-organizing systems that were capable of learning internal representations from data, positing that hidden units in multilayer networks encode features of increasing complexity as network depth increases. These concepts formed the basis

of future work in representation learning and motivated the exploration of deeper network architectures to model more complex structures in data.

The representational capabilities of neural networks were mathematically substantiated in 1989, when Cybenko and Hornik et al. independently published proofs of the Universal Approximation Theorem. These works demonstrated that a feed-forward network with a single hidden layer and a finite number of neurons can approximate any continuous function on a compact set [63, 30]. While Cybenko's work was restricted to networks utilizing sigmoidal activation functions, Hornik et al. extended this result, proving that universality holds for any bounded, non-constant, and monotonically increasing activation function. This work provided a definitive rebuttal to Minsky and Papert's critique of perceptrons [110], confirming that the limitations they identified were specific to single-layer architectures.

Despite these contributions, connectionist-era neural networks exhibited several limitations relative to modern deep learning models. Most notably, training was restricted by computational resources available at the time, which limited model size and large-scale experimentation. Neural networks were trained on general-purpose central processing units (CPUs), making it impractical to explore deeper architectures or large datasets.

In addition to computational constraints, optimization challenges further limited the effectiveness of early multilayer networks. The authors noted that the back-propagation algorithm could fail in the presence of local minima. With too small a learning rate, weight updates are not enough to traverse the loss landscape toward the global minimum. Furthermore, due to the use of activations such as sigmoid and tanh, gradients could become saturated during backpropagation. Excessively large or small values would fall near the asymptote of these functions, leading to vanishingly small values. As such, the resulting gradients would be small, multiplicatively reducing the learning signal in successive layers. These limitations prevented networks from increasing in depth and learning more complex representations, prompting the development of new methods to overcome these challenges.

1.1.3 Deep Learning

The transition from connectionism to deep learning emerged from convergent improvements in data, hardware, and algorithms during the late 1990s and early 2000s. These innovations addressed fundamental challenges of the connectionist era, such as vanishing gradients, computational bottlenecks, and convergence failures. Consequently, training networks with more layers became feasible and effective, leading to the era of deep learning, a term popularized by Geoffrey Hinton in 2006 [61]. The pursuit of depth was built on the theoretical foundations laid by Rumelhart, Hinton, and Williams, who demonstrated that hidden layers allow networks to learn more complex hierarchical representations. [144, 90].

A key innovation in the deep learning era was the shift from CPUs to Graphics Processing Units (GPUs). The physical architecture of the GPU is well-suited for homogeneous, parallelizable, and batched operations, specifically massive matrix multiplications central to neural network training. In 2009, Raina et al. published *Large-Scale Deep Unsupervised Learning Using Graphics Processors*, in which they trained Deep Belief Networks (DBNs) [61] and sparse coding networks [121] using GPUs [133]. By storing parameters in the GPU's global memory, parallelizing homogeneous thread operations in stream processors, and using efficient scheduling to reduce overhead times, the authors were able to achieve up to a 72x speedup in DBN training and a 15x speedup in sparse coding training. This acceleration rendered previously intractable problems computationally feasible; the reduction in training time enabled the exploration of deeper architectures and larger datasets, expanding the frontier of neural network training.

In order to fully leverage the benefits of GPU acceleration for neural network training, large, annotated datasets to were necessary to avoid overfitting and improve generalization. The earliest advances in dataset construction occurred in the image domain; in the early 2000s, the MNIST dataset of handwritten digits [92] served as the benchmark for neural network training. However, this dataset comprised

only 10 image classes within a narrow domain, meaning that models trained on this data failed to capture general image recognition capabilities. To overcome these limitations, datasets such as Caltech101 [39], PASCAL VOC [38], and MSRC [160] were developed, providing a wider variety of images and classes for training general-purpose computer vision models. Despite containing more subject diversity than MNIST, these datasets were still relatively small and lacked intra-class variability, with images often being standardized and lacking occlusions, translations, and other perturbations. In 2009, a breakthrough was made with the development and release of the ImageNet dataset [32]. The authors of ImageNet leveraged the increasing amount of data available on search engines and the advent of crowdsourcing platforms like Amazon Mechanical Turk to produce a dataset of 3.2 million annotated images following the hierarchical structure of WordNet [40]. The availability of large-scale datasets such as ImageNet encouraged the development of neural networks with more layers to capture increasingly abstract and general image representations. This paradigm shift was further catalyzed by the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) [145], which will be discussed further in Section 1.2.2.

As hardware capabilities improved and massive datasets became available, several algorithmic innovations enabled researchers to train deeper, larger-scale models. One notable breakthrough was the rectified linear unit, or ReLU, which supplanted the sigmoid and hyperbolic tangent (tanh) activation functions that were previously standard [144, 91]. Although ReLU was first utilized in Fukushima's *Cognitron* and *Neocognitron* architectures [41, 42], it became widely adopted only after Glorot et al. published *Deep Sparse Rectifier Networks* in 2011. Glorot et al. showed that ReLU was not only more biologically plausible than its counterparts but also empirically outperformed them in both computer vision and sentiment analysis, especially with deep architectures. ReLU activation enforced network sparsity by setting negative activations to zero and mitigated the vanishing gradient problem with strictly linear positive activations, preventing gradient saturation.

To fully leverage these properties, weight initialization techniques became widely adopted in deep networks. Building on the work of Glorot and Bengio, who introduced 'Xavier initialization' to control the activation variance in sigmoidal networks [49], He et al. introduced 'He initialization' to account for the rectification nonlinearity of ReLU [57]. These innovations facilitated the training of significantly deeper networks, cementing ReLU as a foundational component of the deep learning era.

To address the generalization challenges posed by these high-capacity models, Srivastava et al. introduced dropout in 2014 [164]. Although large datasets such as ImageNet were becoming available, many domains lacked access to millions of labeled training examples. Consequently, deep networks tended to overfit smaller datasets, fitting noise and exhibiting high variance. Dropout ameliorated this issue by regularizing the co-adaptations between neurons. Biologically inspired by sexual reproduction, which reduces complex co-adaptation to make genes more robust, dropout probabilistically removes neurons from the network during training. As a result, neurons in the network cannot co-adapt with each other and must instead learn to function with a randomly chosen sample of units. As the authors stated, "Complex co-adaptations can be trained to work well on a training set, but on novel test data they are far more likely to fail than multiple simpler co-adaptations that achieve the same thing." The utility of dropout is two-fold: it can enforce sparsity and considerably improves generalization error. Due to these factors, dropout has been ubiquitous in state-of-the-art architectures throughout the deep learning era.

In addition to ReLU and dropout, Ioffe and Szegedy introduced batch normalization in 2015, which allowed networks to be further extended [72]. The authors aimed to remedy internal covariate shift a phenomenon where the distribution of activations in hidden layers shifts continuously based on parameter updates in previous layers. To mitigate this, the authors proposed normalizing layer activations around the mini-batch mean and variance. They demonstrated that batch normalization accelerates training and acts as a regularizer while also achieving state-of-the-art performance across multiple deep learning benchmarks. While Santurkar et al. argued in 2018

that batch normalization helps optimization through smoothing the optimization landscape instead of reducing internal covariate shift [146], the technique remains a standard component in modern deep neural networks, facilitating the learning of increasingly complex representations.

Along with the architectural changes that made deep learning possible, the Adaptive Moment Estimation (Adam) optimizer was introduced to make weight updates for deeper networks more efficient [81]. Adam utilizes the first and second moments of the gradients to compute adaptive learning rates for each weight with little additional computation. The authors liken the resulting update rule to a "trust region" method: because the effective step size is bounded by the learning rate, optimization is effectively constrained to a neighborhood where the gradient estimate remains reliable. This promotes stability when gradients are sparse or noisy, allowing for faster convergence. Consequently, Adam has become the primary choice of optimizer in modern deep learning models.

The deep learning era was marked by the confluence of hardware improvements, data availability, and algorithmic innovation. These three factors enabled researchers to explore deep architectures that were previously impossible to train. As depth increased, a major paradigm shift occurred: the transition from *feature engineering* to *feature learning* [90]. While prior approaches relied on hand-crafted features to structure data, deeper networks allowed for complex representations to be learned directly in latent space, deriving hierarchical features with backpropagation instead of by hand.

With this shift away from manual feature engineering, research focus turned toward architectural and algorithmic design, with inductive biases tailored to specific data modalities. Computer vision assumed translation invariance, language models assumed sequentiality and contextual dependence, and graph deep learning models incorporated permutation invariance. These inductive biases align closely with the properties of biological data, making the history and current state of deep learning across modalities crucial for applications in phenomics, genomics, and systems biology.

The remainder of this review is organized as follows: we cover computer vision in Section 1.2, language modeling in Section 1.3, and graph learning in Section 1.4. Additionally, we will explore techniques to scale neural networks to massive modern datasets in Section 1.5. We will conclude with a review of how these techniques are applied to various biological subdomains in Section 1.6.

1.2 Computer Vision

Computer vision served as the primary proving ground for neural network techniques and architectures during the deep learning era. This domain was the first where deep learning achieved widespread consumer adoption [92] and later surpassed human performance [57]. Breaking these performance plateaus shifted the perception of deep learning from an esoteric field into a rapidly evolving discipline with disruptive applications. This success was not accidental; rather, the alignment between the structure of image data and the inductive biases of neural architectures, specifically convolutional neural networks (CNNs), allowed deep learning models to excel at learning complex representations. By integrating translation invariance, local connectivity, and hierarchy into neural architectures, researchers created biologically inspired vision models, grounded in Hubel and Wiesel's model of the mammalian visual cortex [67, 68], validating the connection between biological and artificial vision.

Several factors catalyzed rapid advancement of deep learning. The first was the establishment of standardized benchmarks that promoted competition and innovation in the computer vision community. The PASCAL VOC challenge [38], and subsequently the ILSVRC [145], provided annual venues to evaluate visual recognition algorithms. These datasets, distinguished by their massive scale and public availability, provided a robust stage to validate methods while fostering competition and connectivity. This created a feedback loop that encouraged research participation and innovation in computer vision, leading to rapidly dropping competition error rates (see Figure 1.2). Due to the challenging nature of the ILSVRC

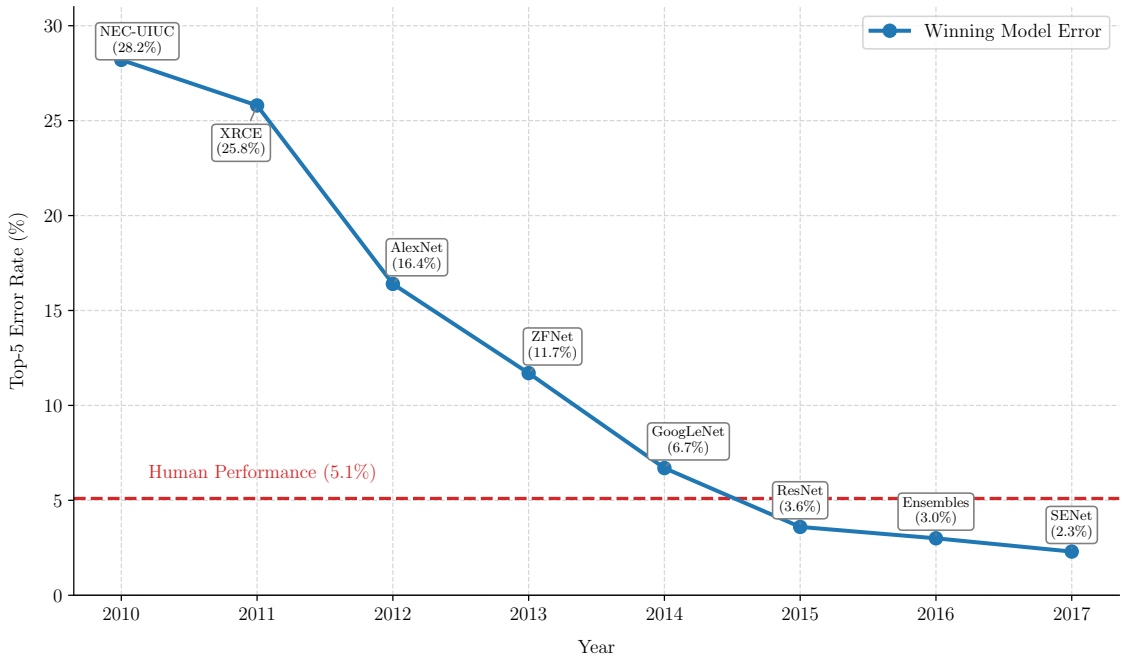

Figure 1.2: ILSVRC Top-5 Error Rate by Year (2010-2017)

and its popularity, the competition also served as a "test bed," where techniques that succeeded in the image domain became part of the standard toolkit for other deep learning subdomains, including biological modeling. Foundational methods widely adopted after their demonstration in the ILSVRC include GPU parallelism [86], residual connections [55], and data augmentation techniques [86].

The confluence of these factors enabled a transition from foundational benchmarking to biologically significant innovation. As performance on general image classification saturated, focus shifted to higher-order tasks, such as object detection, semantic segmentation, and automated feature extraction. These methodologies proved directly applicable to spatial and visual biological data, catalyzing innovation in image-based phenotyping, geospatial modeling, and multimodal artificial intelligence. Consequently, examining the evolution of computer vision is essential to understanding the technological development of modern biological modeling.

1.2.1 Origins

Much like general deep learning research (Section 1.1.1), computer vision research began as a biologically inspired field grounded in neuroscience principles. In the late 1950s and early 1960s, Hubel and Wiesel provided the theoretical underpinnings for modern architectures with their work on the mammalian visual cortex [67, 68]. They showed that vision is a hierarchical process built on detecting and combining local features. The visual cortex does not respond to the retinal input holistically; instead, neurons respond to stimuli within small, specific regions called receptive fields. Additionally, Hubel and Wiesel identified two types of cells involved in processing this visual information: simple cells and complex cells. While simple cells fire only when a specific feature, like a horizontal or vertical line, appears in a precise location, complex cells activate when the feature appears anywhere in the receptive field. Based on these fundamental units, they proposed a hierarchical model of visual processing: simple cells connect to complex cells, which connect to hypercomplex cells, allowing the visual cortex to build a layered representation that abstracts from lines to shapes and eventually to objects.

Kunihiko Fukushima built on this work by introducing the cognitron in 1975 and the neocognitron in 1980 [41, 42]. A precursor to modern CNNs, the neocognitron operationalized the concepts of receptive fields, simple cells, and complex cells from Hubel and Wiesel. Instead of having fully connected layers like the perceptron [142], the neocognitron featured a shared weight mechanism. This simulated simple cells (referred to as "S-cells" by Fukushima) by applying the same filter to each part of the image, meaning a cell would activate any time its filter matched the input pattern in a region. Multiple S-cells could be applied to the image in parallel to detect multiple features. These features were then propagated to complex cells, or C-cells, for transformation-invariant feature detection. S-cells are equivalent to convolution operations, and C-cells are equivalent to pooling operations. The combination of multiple S-C layers resulted in a similar hierarchical structure to the model of

visual processing introduced by Hubel and Wiesel [67, 68]. Although Fukushima made foundational contributions to CNN architecture, the neocognitron relied on unsupervised competitive learning rather than backpropagation, which hindered its standalone success for complex supervised tasks.

In 1989, Yann LeCun combined the architectural foundations of Fukushima [41, 42] with the learning rules of Rumelhart et al. [144] to train the first modern CNNs [93]. LeCun used backpropagation to train a CNN for recognizing digits from the United States Postal Service. The model achieved a 1% error rate and 9% rejection rate, demonstrating state-of-the-art performance on a real-world task. This result showed that utilizing convolutions for computer vision tasks was a promising approach; because the model learned hierarchical features, the need for manual preprocessing was minimized. Furthermore, the computational burden of these networks was lower than fully-connected networks due to their shared-weight design. Additionally, LeCun argued that the reduction in free parameters caused by the architectural choice reduced overfitting, providing more evidence for the potential of CNNs in computer vision.

LeCun produced multiple iterations on this approach, introducing LeNet-5 in 1998 [92]. This work expanded the results from LeCun's original paper by scaling the dataset and model size while providing a comparison to other gradient-based models. LeNet-5 achieved performance superior to most existing methods and comparable to Support Vector Machines (SVMs), while offering significantly faster inference speeds. LeCun noted that "better pattern recognition systems can be built by relying more on automatic learning and less on hand-designed heuristics" [92]. The combination of scale and performance in these experiments solidified CNNs as one of the primary methods for image classification. Additionally, the LeNet-5 paper introduced a modular paradigm for neural network programming, introducing various forward and backward propagation functions that could be composed while still storing a computational graph. This framework provided a predecessor to popular

deep learning libraries such as Caffe [76], TensorFlow [105], Pytorch [126], and Jax [14].

Despite CNNs showing successful performance at scale, training was still constrained by hardware and data limitations. LeNet-5 took three days to train for 20 epochs, and the MNIST dataset was still small by modern standards, containing only around 60,000 images. However, as discussed in Section 1.1.3, innovations in these realms promoted more successful computer vision research, with the ILSVRC [145] serving as the catalyst for rapid progress.

1.2.2 The ImageNet Challenge

As introduced in Section 1.1.3, the ImageNet dataset [32] revolutionized computer vision research, providing a massive, open-source dataset for researchers to benchmark their methods. Due to its popularity, the creators of ImageNet introduced the ILSVRC, which created a standardized benchmark for the deep learning community. As noted earlier, many seminal contributions used the ILSVRC as their primary benchmarking dataset. We will discuss the impact of key ILSVRC participants in this section.

The first major breakthrough related to the ILSVRC was the introduction of AlexNet [86], which won the competition in 2012. AlexNet achieved a reduction in top-5 error rate of over 10 percentage points compard to the previous winner, producing "by far the best results ever reported on these datasets" at the time [86]. The AlexNet architecture revolutionized deep learning with several contributions. Primarily, the authors were the first to utilize model parallelism with GPUs to train larger models more efficiently. They used two GPUs, partitioning convolutional filters between the two to parallelize processing. Despite being one of the largest CNNs ever at the time, training took a relatively modest five to six days. Furthermore, AlexNet was the first network to utilize dropout [164], ReLU [50], and data augmentation at scale. The adoption of these techniques, combined with the competition-winning

performance they yielded, solidified AlexNet as a foundational work and exemplified the superiority of deep learning for image recognition over other techniques, like SVMs, which were popular at the time. It is also important to note that AlexNet's success was a precursor to modern deep learning scaling laws, which will be discussed further in Section 1.3; the authors demonstrated that using more compute and data had a direct relationship with performance, a trend that continued to be explored in the following years across various deep learning subdomains.

In 2014, two innovative architectures achieved first and second place in the ILSVRC. GoogLeNet [167], developed by researchers at Google, won the competition, while VGGNet took second place. GoogLeNet utilized an architecture called "Inception," which enabled the training of even deeper CNNs while using fewer parameters. Each inception block balanced receptive field focus by using 1×1, 3×3, and 5×5 convolutional kernels in parallel in a single layer; the varied kernel sizes created an implicit balance between hierarchy and localization. Crucially, to reduce the number of parameters in the network, and thus overfitting, GoogLeNet utilized 1 global average pooling to transform the inception outputs for classification rather than the dense layers used in prior work. Due to the shared weights of convolutions, this greatly reduced the parameter burden compared to using fully-connected layers, which previous architectures like AlexNet had relied on for reshaping outputs [86]. Finally, GoogLeNet leveraged auxiliary classification to increase network depth. The authors added outputs at intermediate layers in the network; by performing backpropagation from these auxiliary objectives, they ensured a strong gradient signal throughout the network, even with increasing depth. The combination of these innovations produced the most performant model of the time.

While GoogLeNet's Inception architecture showed that a complex, highly engineered CNN could yield state-of-the-art performance, VGGNet took an alternate approach, demonstrating that a homogeneous network with many layers of small convolutions could yield comparable performance [161]. The authors tested CNNs of various depth while exclusively using 3×3 convolutional filters, arguing that

stacks of smaller filters were equivalent to a single, larger filter while fitting a more discriminative function with fewer parameters. They demonstrated this result empirically, achieving second place in the ILSVRC and first place in the ImageNet localization competition. This homogeneous, stacked architecture provided an easily scalable structure, and its simplicity led to its adoption across multiple other subdomains.

In 2015, superhuman performance was formally achieved in the ILSVRC with the introduction of ResNet [55]. The primary focus of the ResNet architecture was combating the vanishing gradient problem, which, despite being partially addressed by innovations such as ReLU [50], was still prevalent as network depth exceeded 30 layers. The ResNet team introduced residual learning, where instead of fitting a function, $h(x)$, its residual, $h(x) - x = f(x)$, was learned. Consequently, layer mappings could be reformulated as learning $f(x) + x$, including a direct connection from the input to the output of a layer. This directly addressed gradient saturation by providing a residual stream without exposure to a nonlinearity, stabilizing gradient flow. By adding residual connections to an architecture inspired by VGGNet, the authors were able to train networks with a depth of up to 152 layers. The innovations introduced in ResNet have been adopted by numerous state-of-the-art architectures across subdomains, showing the impact of this work.

Over its life cycle from a difficult, open research competition to a saturated benchmark, the ILSVRC yielded many innovative approaches to image classification. Model parallelism [85], dropout [164], ReLU [50], homogeneous block architectures [161], and residual connections [55] are all used in various architectures across subdomains today. In the next section, we will discuss how these methods were incorporated to address the challenges of semantic segmentation and object detection.

1.2.3 Semantic Segmentation

As performance on the ILSVRC improved, the benchmark became saturated; state-of-the-art models reached an accuracy plateau where meaningful improvements became diminishingly marginal. Due to this saturation, focus shifted to more complex tasks such as semantic segmentation, where specificity and localization carried greater weight. In biology, semantic segmentation is critical; for example, pixel-level classification enables researchers to extract intensive phenotypes from images, facilitating population-scale studies. Therefore, it is important to understand advances in semantic segmentation to understand how best to apply it to biological data.

While semantic segmentation was historically strongly tied to the ImageNet dataset, one of the first state-of-the-art segmentation models was developed for biomedical image segmentation. U-Net, developed in 2015, is a fully convolutional architecture that produces pixel level classifications to create image segmentation maps. The architecture features a contractive path to capture hierarchical and contextual features followed by a symmetric expansive path to provide specific localizations within the image. The contractive and expansive paths are also linked via skip connections. Functionally similar to residual connections [55], skip connections improve gradient flow in the model while also enabling the combination of hierarchical features with localized features via concatenation in the expansive feature map [141]. U-Net achieved state-of-the-art performance on both *Drosophila* nerve membrane segmentation and cellular segmentation, solidifying its role as a biologically useful deep learning architecture.

Despite the success of U-Net, its supervised nature meant that ground-truth pixel-level segmentation data was necessary for training. Such data is expensive and tedious to collect, so naturally, researchers began to explore other architectures to reduce the burden of collecting hand-labeled segmentations. One of the first attempts to explore deep, unsupervised image segmentation was W-Net [192]. This architecture combined

two U-Nets to produce a segmentation map without supervision. The first U-Net produced a k-channel representation of the image, with k representing the number of classes for segmentation. This segmentation map was then propagated through another U-Net, which reconstructed the input. The intermediate segmentation map was evaluated with a soft normalized cut loss function, which maximizes association between pixel groups, while the final reconstruction was evaluated with a reconstruction loss function. By jointly optimizing these two functions, the W-Net architecture could extract segmentations in an unsupervised manner. After training, conditional random field smoothing was applied to improve segmentation quality. W-Net produced results comparable to the state-of-the-art unsupervised segmentation models, providing an alternative to the labor-intensive process required to produce datasets for supervised semantic segmentation.

In 2017, He et al. introduced Mask R-CNN, an alternative architecture that unified object detection, instance segmentation, and classification into a multi-task objective [58]. Expanding on the work of the Fast R-CNN [47] and Faster R-CNN [137], which explored only classification and object detection, the Mask R-CNN added a semantic segmentation path that was trained simultaneously with the classification and bounding box objectives. Unlike semantic segmentation, which labels all pixels of a class identically, this approach enabled instance segmentation, distinguishing between individual objects of the same class. This architecture achieved state-of-the-art performance on the common objects in context (COCO) dataset [96] and human pose estimation, showing the transferability and generality of the Mask R-CNN method.

Despite the success of Mask R-CNN and U-Net, the explosive success of transformer architectures in the language modeling domain (detailed in section 1.3) inspired novel approaches to computer vision. Dosovitskiy et al. introduced the vision transformer (ViT) in 2020; instead of relying on convolutional layers, which were the standard for computer vision architectures, the authors leveraged the attention mechanism. By dividing images into 16×16 pixel "patches," and encoding these

patches into tokens with a linear layer, the ViT architecture allowed images to be processed as sequences. This solution addressed many concerns in the original W-Net paper regarding data efficiency; while the original ViT was supervised, it established the architectural framework that later enabled powerful self-supervised masked imaged modeling (such as MAE), reducing reliance on expensive labeled data [192]. Although not strictly a segmentation model, the ViT created a framework that enabled transformer-powered segmentation.

Building on the ideas introduced by the ViT, researchers at Meta released the Segment Anything Model (SAM) for semantic and instance segmentation [82]. SAM is a foundation model built with a ViT architecture, trained on over 11 million images and 1 billion segmentation masks with a model-in-the-loop paradigm. Newer versions, SAM 2 [135] and SAM 3 [16] were released in 2024 and 2025, respectively, to enable video segmentation, improve inference speed, and enhance text prompting.

- unet: first fully convolutional semantic segmentation architecture; utilized skip connections, and reduction/reconstruction strategy to produce segmentation maps. allows precise localization by combining high-level semantic info with low-level texture info.
- wnet: unsupervised variant of U-Net; built on unet by taking using one u-net to produce a segmentation map and another to recreate the image, and then
- mask r-cnn
- ViT: first prominent vision architecture to borrow from language modeling, produces image-patch tokens and performs next-token predictions on those tokens for more effective unsupervised training. When paired with a decoder, semantic segmentation could be performed.
- SAM: scaled version of ViT by meta, foundational model trained on millions of images for general segmentation. more variants appeared for better segmentation, then video segmentation

1.2.4 Representation Learning and Transfer Learning in Vision

- SimCLR/ Contrastive learning
- CLIP: used contrastive learning to align language and image representations in vector space, matching image-caption pairs to each other with a contrastive loss function. core concept that applies itself to multimodal modeling, not only with images, but also with graphs, language, and other domains, as will be discussed.
- ViT: the unsupervised training process allowed for representation learning and fine-tuning without the burden of manually labeled datasets
- SAM: performed this representation learning on a massive scale, creating a foundation model that had learned general image representations for millions of examples

1.2.5 Transformer-Based Computer-Vision

- ViT
- SAM
- CLIP
- BLIP-2
- DINO
- MAE

1.3 Language Modeling

1.3.1 N-Gram

1.3.2 Neural Language Modeling

1.3.3 RNN/LSTM

1.3.4 Transformer

1.3.5 Scaling Laws

1.3.6 Multi-Modal Language Modeling

1.4 Graph Learning for Biology

1.4.1 Spectral vs. Spatial Methods

1.4.2 Message Passing Neural Networks (MPNNs)

1.4.3 Graph Attention Networks (GATs)

1.5 High-Performance Computing for Deep Learning

Explain why scaling is crucial for modern deep learning

1.5.1 Parameter Efficient Fine-Tuning (PEFT)

1.5.2 Distributed Data Parallel

1.5.3 Fully-Sharded Data Parallel

1.5.4 DeepSpeed ZeRO-3

1.5.5 Tensor Parallel

1.5.6 Pipeline Parallel

1.6 Deep Learning for Biology

While deep learning has achieved notable success in domains such as computer vision and natural language processing, its application to biological data remains comparatively less mature. Unlike image or text modalities, biological data exhibit complexity, hierarchical structure, and long-range dependencies, all with limited or noisy supervision. These characteristics challenge many of the assumptions that underlie standard deep learning architectures and training paradigms. As a result, biological problems provide compelling settings to extent how advances in representation learning, model scaling, and computational infrastructure generalize beyond areas where deep learning has been most successful. Understanding of prior work in vision, language modeling, and scalable training is therefore essential for situating and motivating the approaches explored in this thesis.

1.6.1 Vision-Based Phenotyping

1.6.2 Transformer-Based Genomic Prediction

1.6.3 LLM-Based Mechanistic Exploration

Chapter 2

A High-Throughput Deep Learning Pipeline for Image-Based Phenotyping and Genetic Interpretation of Pennycress Seed Pods

2.1 Introduction

Pennycress (*Thlaspi arvense L.*) is a winter annual in the *Brassicaceae* family that has shown potential as a bioenergy feedstock and cover crop due to its winter-annual habit, growth in the fallow period, and amenability to genetic modification [115, 152]. Its short generation time, established transformation protocols, sequenced reference genome [119] and species-wide pangenome [13] make pennycress a viable candidate for breeding and domestication [152]. However, domesticating pennycress requires improving yield, seed retention, and cover-crop/feedstock traits [152]. In *Brassicaceae*, silicle morphology is directly tied to seed yield and pod shattering [118].

Pods play an active role in development and resource allocation rather than passively containing seeds [11]. Pod walls are photosynthetically active and supply assimilate to the developing seed [65, 198]. Seeds per pod is one of three multiplicative components of seed yield, alongside pods per plant and seed weight, and in rapeseed its natural variation is under maternal and embryonic genetic control [199]. Therefore, tissue-specific pod traits such as wing area, envelope area, seed count, and tissue-to-seed ratios are biologically meaningful targets for breeding and phenotyping.

However, realizing the potential of pennycress as a biofuel feedstock requires population-scale studies characterizing variation in feedstock-relevant traits, which are currently limited by phenotyping bottlenecks, most notably the manual collection of pod-level traits [94, 175]. Although high-throughput methods exist for measuring seed composition [172], single-seed weight [23], and seed shape [168], each operates on individual seeds and extracts a limited set of traits. These techniques do not resolve distinct pod tissues (wing, envelope, and seed) or support the depth of morphological and inter-tissue measurements that pod-level phenotyping requires. Despite their relevance to pennycress improvement, pod traits have not been measured at tissue resolution across large pennycress panels.

The central bottleneck to measuring pod traits is tissue segmentation, which requires separating wing, envelope, and seed regions, and counting seeds in each pod [175]. This process is labor-intensive because each pod requires boundary tracing, object separation, and visual inspection [94]. Consequently, manual measurement limits both sample size and trait depth, forcing researchers to measure either fewer accessions or fewer traits [175]. As a result, detailed pod traits are systematically omitted from population-scale studies. Automated segmentation addresses this bottleneck by converting raw images into tissue-specific masks that can be used for scalable trait extraction [94, 87].

In other domains, image-based phenotyping has enabled measurements at scales and resolutions that are infeasible by hand, from cell-level microscopy to satellite imagery [94, 87]. A single imaging modality can yield many traits simultaneously,

allowing researchers to extract complex topological and morphological features that are inaccessible with manual measurement [94]. For pod-level phenotyping, the operative task is segmentation: distinguishing wing, envelope, and seed regions within each pod. Classical segmentation approaches require controlled imaging conditions and manually tuned thresholds, which do not generalize across natural variation in raw scans.

Unlike classical segmentation, deep learning models operate on raw images directly, learning relevant features end-to-end; deep learning has achieved state-of-the-art performance across computer vision tasks [86, 55]. In the plant phenotyping domain, deep learning models have been applied to disease detection [111], organ counting [173], seed-per-pod estimation [175], and population-scale trait extraction [181, 87]. In particular, convolutional neural networks (CNNs) learn hierarchical filters over local image neighborhoods [90], giving them a spatial inductive bias well suited to segmentation. U-Net is a CNN-based segmentation architecture that shows strong performance on small, specialized biological and biomedical segmentation tasks, making it a natural baseline for tissue-level pod segmentation [140]. More recent architectures can improve performance in some settings but may require larger datasets, different preprocessing, or task-specific adaptation. Because pennycress pod segmentation is a small, specialized biological task with fine seed/envelope boundaries, benchmarking modern architectures against U-Net is necessary.

Once tissue-resolved pod traits can be measured at scale, the next challenge is linking trait variation to candidate genetic mechanisms. High-throughput phenotyping can be used in conjunction with genome-wide association studies (GWAS) [174] to reveal the genetic architecture of phenotypes, providing candidate genes and markers for breeding. A network-based framework that leverages multiple lines of evidence in pennycress or related species utilizes GRIN (Gene set Refinement through Interacting Networks) [165], MENTOR (Multiplex Embedding of Networks for Team-Based Omics Research) [166], and MENTOR-IA (MENTOR Interpretation Agent) [178] to prioritize these candidates and resolve them into interpretable modules,

yielding candidate mechanisms. This framework has been applied to other pennycress traits [24], but not to intensive, tissue-resolved pod phenotypes in pennycress.

In this work, we developed a deep learning pipeline for pennycress seed pod segmentation, enabling deep, tissue-resolved phenotyping at population scale. We benchmarked a suite of modern computer vision models, spanning CNNs, vision transformers, and foundation models. We found that a custom U-Net provided the most efficient pod tissue segmentation performance with comparable accuracy to modern models and a 30- to 45-fold speed increase over manual phenotyping. We applied this pipeline at population scale to over 12,000 pods from 768 accessions and used it to identify candidate loci, genes, and mechanisms underlying pod traits.

2.2 Materials and Methods

2.2.1 Experimental Design

We designed this study to explore whether it was possible to use deep learning to accelerate image-based pennycress pod phenotyping and connect the extracted phenotypes to candidate genetic mechanisms for pennycress improvement. Following this objective, our study contains two main components. In the first stage, we trained and benchmarked a U-Net segmentation model to extract 52 per-pod phenotypes, applied the model at population scale to 12,253 pods, and examined the resulting trait matrix for biological coherence with a predictive phenotype network. In the second stage, we filtered traits by heritability ($H^2 \geq 0.05$), connected them to loci via GWAS (significant at $p < 1 \times 10^{-6}$), and mapped loci to candidate genes. We then pruned the candidates with GRIN and resolved the survivors into modules with MENTOR. With the assistance of MENTOR-IA, we interpreted the modules to generate exploratory mechanistic hypotheses for genetic targets. Our data were sourced from a 768-accession panel of pennycress pod images, collected from field sites in the midwestern United States. Broad-sense heritability was estimated on

$n = 25$ replicated genotypes, and GWAS was performed on a subset of 189 genotyped accessions from dryland environment treatments.

2.2.2 Dataset

Data Collection

Our raw data consisted of 768 scanned images of pennycress (*Thlaspi arvense L.*) seed pods collected from field sites in the Midwest, including those at Western Illinois University (WIU), Illinois State University (ISU), and University of Minnesota (UM) . After collecting seed pods, we imaged them using an Epson V39 Perfection scanner. Each scanned image contained multiple seed pods due to their size. Figure 2.1A displays an example raw image scan. The 768 scans represented 768 pennycress accessions, with each scan typically containing approximately 10–20 seed pods.

Data Annotation

For supervised model training, we created pixel-level annotations using GNU Image Manipulation Program (GIMP). We labeled three areas of each pennycress seed pod: the wing was annotated with red pixels, the envelope was annotated with green pixels, and the seeds were annotated with blue pixels. We annotated these areas due to the biological significance of their phenotypes. Figure 2.1B and C show an example of a separated pennycress seed pod and its associated segmentation. We manually annotated 21 scanned images, amounting to 347 annotated seed pods. We assigned 16 scans containing 282 pod crops to the development set and reserved 5 scans containing 65 pod crops as an independent test set used only for final benchmarking. Within the development set, we randomly allocated the 282 pod crops in an 80:20 ratio to training and validation subsets, respectively. The validation subset was used for checkpoint selection and early stopping; the independent test set was not used during training or checkpoint selection and was evaluated only for the final architecture benchmark.

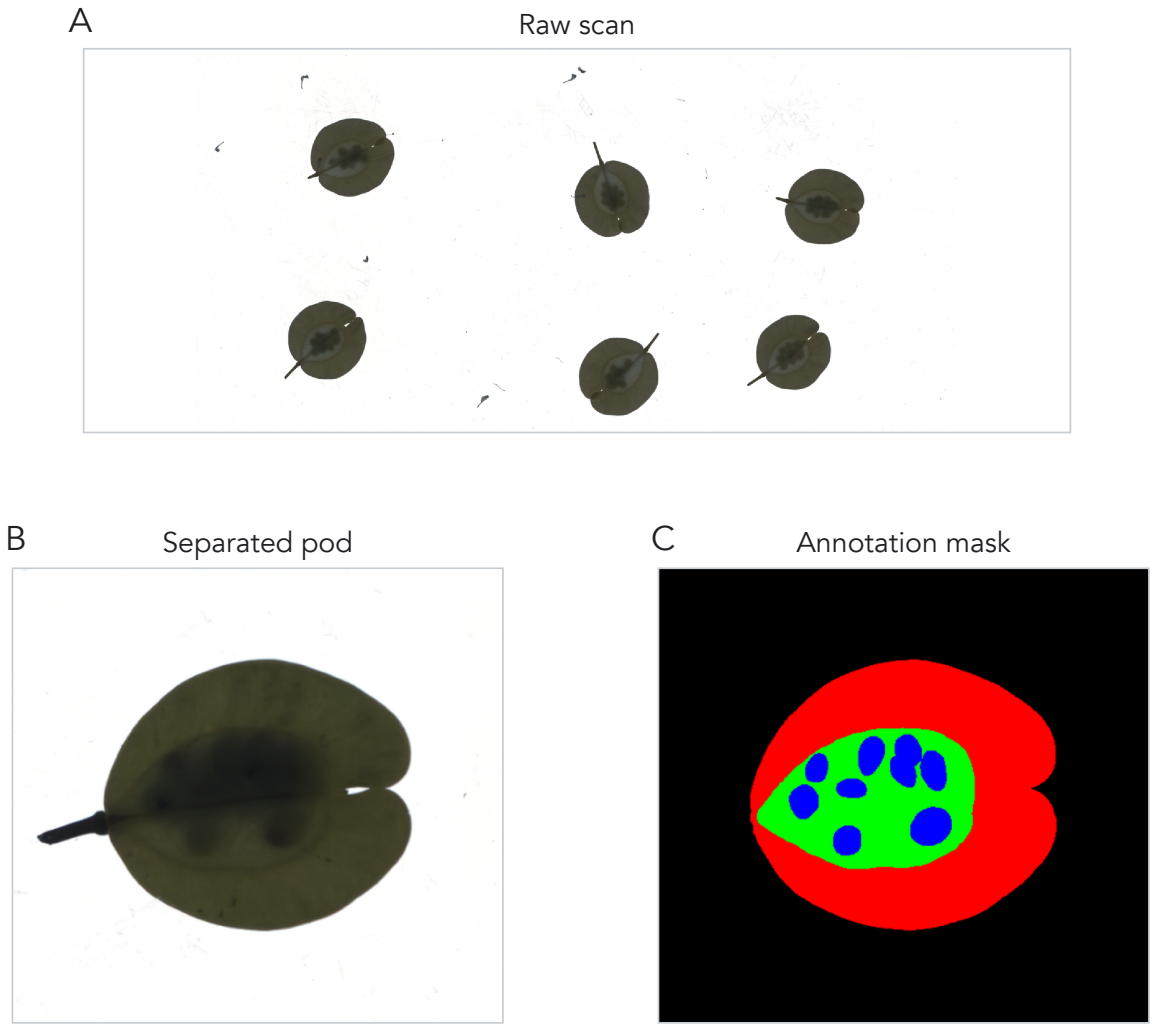

Figure 2.1: Raw pennycress seed-pod scan, separated pod crop, and pixel-level annotation. (A) Flatbed scan excerpt showing multiple *Thlaspi arvense* seed pods on a white scanner background before object separation, annotation, and segmentation preprocessing. (B) Separated *T. arvense* seed pod extracted from a raw scan. (C) Corresponding semantic annotation mask, with wing tissue in red, envelope tissue in green, seeds in blue, and background in black.

Data Preprocessing

Before model input, we separated each pennycress seed pod in the raw image. We created rectangular bounding boxes around each seed pod object in Python, and we saved each bounding box as a separate image, yielding a dataset of 12,253 individual pods across the 768 scans. We used these separated images for model input.

Image Tile Generation

We implemented a tile generation scheme to increase the amount of training data available and build an invariance to common imperfections that occur during the image collection process. Tiling is a form of image augmentation that divides each image into small, fixed-size patches that serve as training examples.

Prior to tiling, we padded each separated image with 128 pixels in all directions. Image padding enables us to generate tiles near the edge of images, allowing the model to see a greater context. The red box in the leftmost image in Figure 2.2 highlights the padding boundary, and the remaining panels show the corresponding semantic and class-specific masks used during tile sampling. While we also used a tile size of 128 pixels, the padding and tile size are independent parameters.

After padding, we selected a tile size of 128 pixels. Pixels on the seed pod were randomly initialized as tile centers. After the algorithm selected center pixels, it used the surrounding 128 × 128 pixels to create each tile.

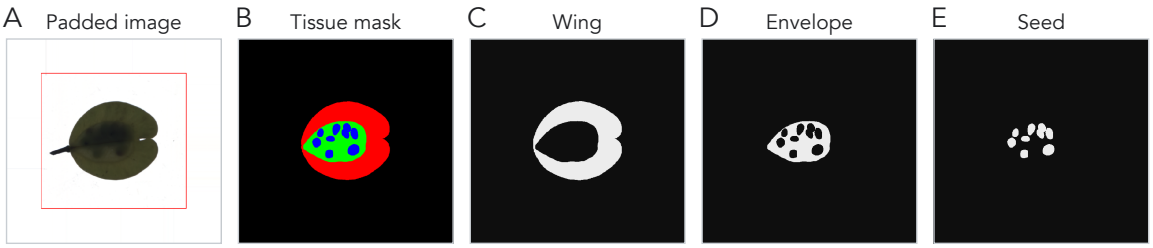

Figure 2.2: Preprocessing outputs for model training. (A) Padded separated pod image; the red rectangle marks the original crop boundary after adding 128 pixels of context on each side. (B) Full semantic tissue mask. (C–E) Binary masks for wing, envelope, and seed classes, respectively.

During validation, we left tiles unchanged to prompt the most accurate predictions possible. However, during training, we applied various image augmentations and transformations to build an invariance to common data collection issues. We applied rotation, blur, saturation, brightness, flips, and transposes with a 50% chance of occurrence each, greatly increasing the number of unique types of tiles that could be generated.

We selected each type of transformation to combat certain issues that occur during the scanning process. For example, blur transformations simulate potential smudges on the scanner screen. Rotations model the differences in angle between scanned leaves, and brightness transformations capture the potential for shadows underneath folds in an image. By using these transformations to simulate real imaging issues, the model learns to become invariant to such features and instead focus on biologically relevant information. Therefore, the purpose of the tile generation scheme is twofold: not only does it increase the amount of available training data, but it also builds an invariance to common scanning issues. We show examples of generated training tiles, their corresponding labels, and representative augmentation outputs in Figure 2.3.

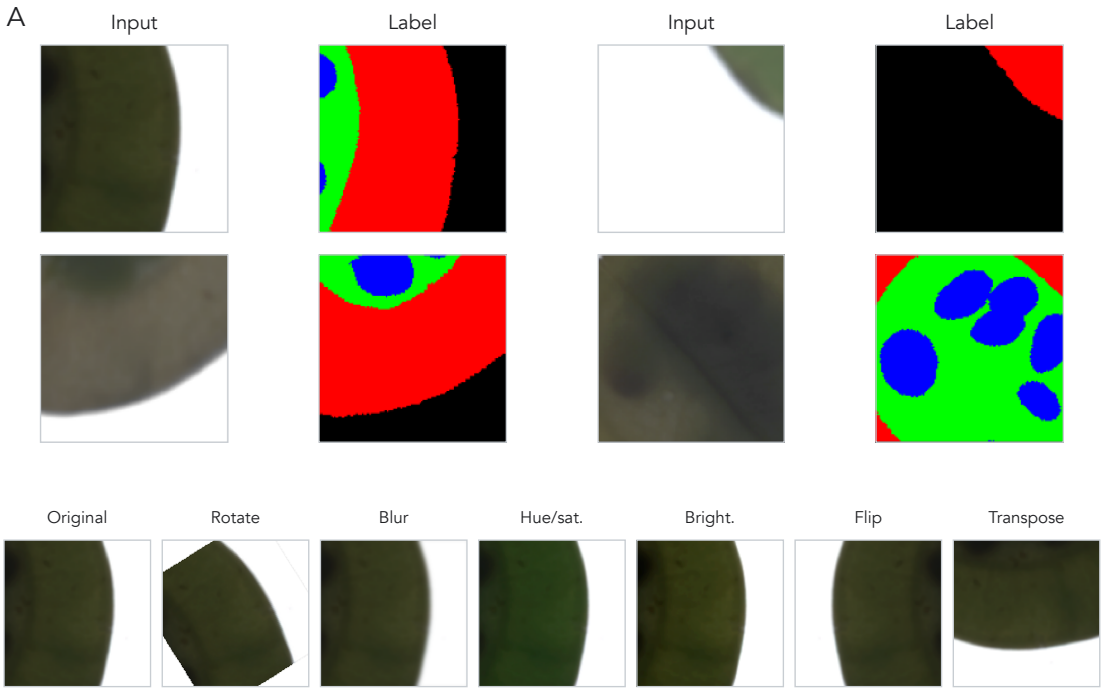

Figure 2.3: Example training tiles and augmentation outputs. (A) Input and label pairs show 128×128 tiles sampled from padded seed-pod images and their corresponding tissue-label masks. Tiles include background, pod-boundary, tissue, and seed contexts used for model training. (B) Representative image-tile augmentations show the transformation families applied during training, including affine rotation, blur, hue/saturation shift, brightness/contrast adjustment, flip, and transpose operations.

2.2.3 U-Net Baseline

Architecture

We used U-Net [140], a fully convolutional encoder-decoder architecture in which a contractive path captures hierarchical context and a symmetric, expansive path recovers spatial localization. We adapted the original architecture with a modified block structure, SELU activations, and tuned hyperparameters for tissue-level pod segmentation.

Because raw seed pod images were too large and computationally expensive to fit into model memory while training, we used the generated tiles created in Section 2.2.2 for training.

The model's input is a 3-channel RGB tile of size 128 × 128, and its output is also 128 × 128 with four channels corresponding to background, wing, envelope, and seed classes. We applied a softmax activation function to create a pixel-wise segmentation map, where the model gives each pixel a class label. We then compared segmentation maps to the corresponding hand-annotated segmentations detailed in Section 2.2.2 and minimized the weighted cross-entropy loss between them to train the model.

The main component in the U-Net architecture is a convolutional block. Each block contains three convolutional layers with 3 × 3 kernels; each convolution is followed by batch normalization, SELU activation, and dropout with a rate of 0.1.

The network comprises nine convolutional blocks (Figure 2.4), beginning with a 32-kernel block at the full 128×128 tile resolution. The four contractive blocks are separated by 2×2 max-pooling that halves spatial resolution while increasing the channel depth from 32 to 256. A 512-kernel bottleneck operates at 8×8 resolution. The four expansive blocks reverse this with upsampling-convolution steps that restore spatial resolution and reduce channel depth, returning to 128×128. Skip connections concatenate contractive and expansive feature maps, improving gradient flow and fusing features across scales. A final 1×1 convolution produces the four-channel segmentation map.

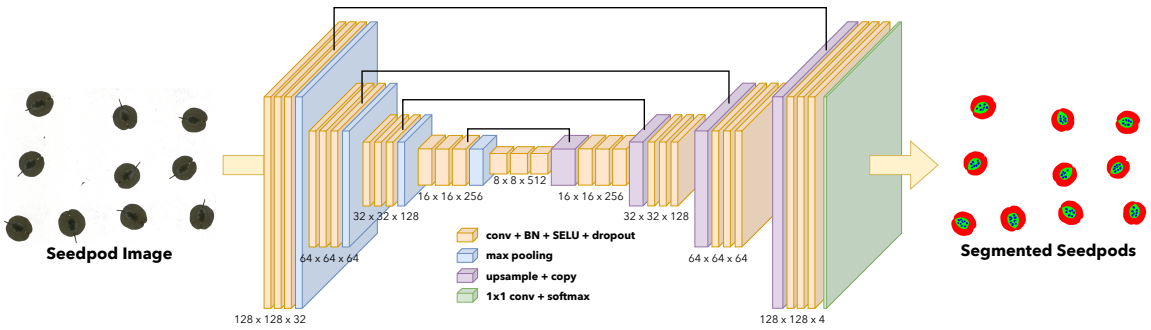

Figure 2.4: Pennycress U-Net architecture. The model takes a 128×128 RGB seed pod tile as input and returns a 128×128 four-channel semantic segmentation map for background, wing, envelope, and seed classes. Orange blocks denote 3×3 convolution, batch normalization (BN), SELU activation, and dropout; blue blocks denote max pooling; purple blocks denote upsampling with skip-copy connections; and the green block denotes the final 1×1 convolution with softmax output. Feature-map labels are shown as height × width × channels.

Boundary-Weighted Cross Entropy Loss

To account for uncertainty in the exact boundaries of our ground-truth labels, we implemented a weighted cross-entropy loss that scales the per-pixel loss by a boundary-aware weight map.

We first compute a raw weight map from the eroded seed mask. We binarize the seed channel, subtract its binary erosion to isolate the boundary ring, and then calculate the Euclidean distance from each pixel to the closest border pixel to produce a distance map. We clip distances at a maximum of $b = 5$ pixels and normalize to the range $[0, 1]$. We evaluated two configurations parameterized by a border weight w_b.

In the **boundary down-weighted** configuration ($w_b < 1$), we min-max normalize the raw weight map so that pixel values $W_{i,j} \in [0, 1]$, where i and j index the row and column of a pixel within a tile. We then invert the weight map so that pixels with a smaller distance to the boundary have a lower weight value. Finally, we scale the weight map so that its range is $W_{i,j} \in [w_b, 1]$. This configuration treats boundary annotations as the noisiest signal and lets the model rely more on confidently-labeled interior pixels.

In the **boundary up-weighted** configuration ($w_b > 1$), we instead clip the raw weight at a minimum of 1, yielding $W_{i,j} \in [1, w_b]$ so that distant pixels carry weight 1 and boundary pixels carry weight up to w_b. This configuration treats boundaries as the most informative regions to learn.

We then multiply the weight map pixelwise with cross-entropy loss and average over pixels:

$$L_{WCE}(t, p(\theta)) = -\frac{1}{\mid I \mid\mid J \mid} \sum_{j}^{J} \sum_{i}^{I} \sum_{k}^{K} W_{i,j}(t_{i,j,k} \log(p(\theta)_{i,j,k})), \tag{2.1}$$

where t is the ground truth segmentation map and $p(\theta)$ is the predicted segmentation map as a function of U-Net parameters. The indices i and j run over the I rows and J columns of a tile, so $\mid I \mid\mid J \mid$ is the number of pixels the loss is averaged over, and k runs over the $K = 4$ classes (background, wing, envelope, and seed). For a given pixel, $t_{i,j,k}$ is 1 if that pixel is labeled class k in the ground truth and 0 otherwise, while $p(\theta)_{i,j,k}$ is the softmax probability the model assigns to class k at that pixel. The inner sum over k therefore reduces to the log-probability the model placed on the correct class, and $W_{i,j}$ rescales that pixel's contribution according to its distance from a seed boundary.

Model Training

We trained the U-Net for up to 150,000 mini-batch updates using the Adam optimizer with a batch size of 32. Rather than partitioning training into fixed epochs over the tile dataset, we sampled tiles continuously from the train split (Section 2.2.2) and tracked progress by iteration count. We evaluated validation loss every 1,000 batches and applied an early stopping criterion of 50 evaluation intervals: if the best validation loss did not improve over 50 consecutive checkpoints (50,000 batches), training terminated.

We used a learning rate schedule with linear warmup and cosine decay. The learning rate ramped from 0 to 1×10^{-3} over the first 1,000 batches to stabilize early gradients, then decayed via a cosine schedule to 1×10^{-5} over the subsequent 90,000 iterations and held constant thereafter.

2.2.4 Phenotype Extraction

We developed functions to extract 52 features from each seed pod segmentation map, grouped into three categories: absolute, ratio, or color.

Absolute Features

We calculated seven absolute features from each segmentation map: wing area and perimeter, envelope area and perimeter, seed area and perimeter, and seed count. Areas for each tissue were calculated by summing the pixels assigned to that class. Perimeters were calculated using Scikit-Image [179] on nested binary masks. Wing perimeter was calculated from the union of wing, envelope, and seed pixels, envelope perimeter was calculated from the union of envelope and seed pixels, and seed perimeter was calculated on the raw (pre-watershed) seed mask, giving the total boundary length of all seed pixels in the pod. Pixel counts were converted to physical area by dividing by DPI^2 ($600^2 = 360,000$) and multiplying by 6.4516 cm^2/in^2.

In addition to measuring area and perimeter features, we utilized the watershed algorithm to count the number of seeds in each segmented image. First, we isolated the seed channel and applied a morphological opening to suppress noise. Then, we calculated the Euclidean distance from each seed pixel to the nearest non-seed pixel. We applied a threshold to this distance map, setting any pixel that was less than 60% of the maximum distance value in the map to 0. The thresholded foreground was labeled into connected components and passed as markers to the OpenCV [15] watershed algorithm, which returned a per-seed segmentation; seed count is defined

as the number of unique markers in the map. The watershed process is visualized in Figure 2.5.

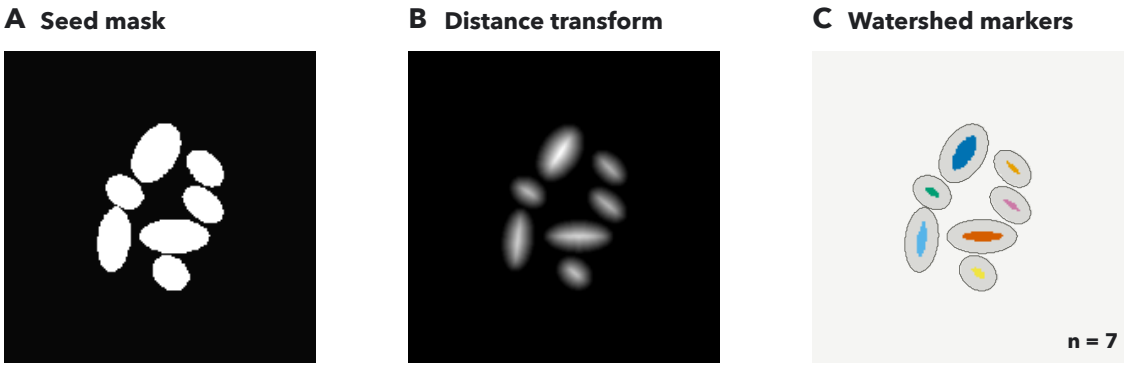

Figure 2.5: Seed-count extraction by watershed segmentation. (A) Binary seed mask isolated from the U-Net segmentation map. (B) Euclidean distance transform used to identify seed interiors. (C) Connected-component markers after thresholding the distance map at 60% of its maximum value; colors denote marker identities used to initialize watershed segmentation, and the resulting marker count defines seed count.

Ratio Features

We calculated 18 ratio features from the segmented seed pod masks, with six to-total ratios and twelve between-tissue ratios, evenly split between area and perimeter-based variants. The six to-total ratios divide each tissue's area or perimeter by the total area or perimeter, yielding wing-to-total, envelope-to-total, and seed-to-total ratios for both measurement types.

The 12 between-tissue ratios cover all pairs of distinct tissues for both area and perimeter. These include ratios into seed (envelope/seed, wing/seed), into envelope (wing/envelope, seed/envelope), and into wing (seed/wing, envelope/wing), computed for both measurement types. Reciprocal ratios are retained (e.g., seed/envelope and envelope/seed) so that downstream analysis is not restricted.

Color Features

We extracted nine color features from each tissue: red, green, blue (RGB); hue, saturation, brightness (HSV); and lightness, a^* (green-red), and b^* (blue-yellow) from CIELAB. We converted each input image into all three color spaces using OpenCV [15]. We then created a tensor of these features with size $\texttt{height} \times \texttt{width} \times 9$. We used the segmentation masks to subset this tensor by tissue, yielding three sub-tensors covering wing, envelope, and seed features separately.

We aggregated each color channel across spatial dimensions by taking the mean over the relevant tissue's pixels, producing a nine-dimensional color vector per tissue. Across the three tissues, this yielded 27 features in total.

2.2.5 Segmentation Architecture Benchmarking

To ensure that our pipeline extracts the most accurate downstream phenotypes possible, we identified the most effective segmentation architecture. We tested six architectures across three families: CNNs, transformers, and foundation models.

We compared two CNN architectures to the baseline U-Net: nnU-Net [73] and DeepLabV3+ [17]. nnU-Net is an approach that configures a U-Net systematically through a three-stage pipeline. The selection process includes fixed parameters that generalize across many datasets; rule-based parameters that are determined by unique dataset and pipeline characteristics; and empirical parameters that are learned through trial and error. Because nnU-Net derives its own patching, batch size, and augmentation scheme through fingerprinting and rule-based selection, the benchmarking for this architecture did not include our custom tile generation scheme (Section 2.2.2). DeepLabV3+ combines an encoder-decoder architecture with atrous depthwise separable convolutions, balancing the use of long-range spatial context with fine-grained boundary detection.

We selected SegFormer [193] and Mask2Former [20] as exemplar transformer architectures for benchmarking. SegFormer uses a hierarchical transformer encoder backbone that generates both high-resolution, fine-grained features and low-resolution, coarse features, and a lightweight MLP decoder to fuse these features for the final semantic segmentation task. Unlike SegFormer, which optimizes a per-pixel classification objective, Mask2Former adopts an alternate paradigm: set prediction. Mask2Former uses a multi-scale encoder backbone to extract features, which are then upsampled to high-resolution per-pixel embeddings with a pixel decoder. Additionally, Mask2Former utilizes a transformer decoder with learnable query inputs to predict mask candidates. These queries are refined through cross-attention with the multi-scale features, and the transformer decoder output and pixel decoder output are combined via dot product to produce region predictions. Although Mask2Former is designed to be a general segmentation architecture, supporting panoptic, instance, and semantic segmentation, we trained only for a semantic segmentation objective due to our task formulation.

We evaluated SAM 3 [16] as a candidate foundation model for benchmarking. SAM 3 is a foundation model designed for promptable concept segmentation, taking input as either text or image exemplars. SAM 3 was trained with a model-in-the-loop data engine on over 1 billion images, generating segmentation examples using human input while shifting towards more model reliance during the annotation process. Although SAM 3 is capable of segmenting video and using concept prompts, we focused on the benefits that its pretrained encoder provides, adopting an approach similar to ClassWise-SAM-Adapter [128] by using a lightweight convolutional mask decoder for classwise semantic segmentation. Furthermore, we applied Low-Rank Adaptation (LoRA) [64] to the image encoder to enable parameter-efficient fine-tuning and limit drift from pretrained representations.

We trained DeepLabV3+, SegFormer, and Mask2Former from default pretrained backbones with the same tile generation and preprocessing scheme that we used for our custom U-Net to ensure consistency. Since nnU-Net's data-aware preprocessing

approach is emphasized as one of its strengths, we did not apply our custom preprocessing pipeline to it, instead using nnU-Net's adaptive fingerprinting setup for preprocessing and model training. As a consequence, nnU-Net scored predictions under its own pipeline at the level of individual seed pod crops (64 crops), whereas all other architectures were evaluated on the five full images from the same test split. For comparability, we grouped nnU-Net crop-level predictions by their source full image before computing the per-image metrics reported below.

For all models except nnU-Net and Mask2Former, we trained variants with unweighted, up-weighted, and down-weighted boundary loss. Due to nnU-Net's adaptive pipeline and Mask2Former's mask-classification objective, boundary weighting was not compatible with these architectures. These model families provide coverage across architectures (CNN and transformer), training objectives (per-pixel prediction and set prediction), and learning scenarios (supervised pretraining and foundation model pretraining).

We used intersection over union (IoU), also known as the Jaccard index, as the primary evaluation metric for this project, following its established use as the standard measure for semantic segmentation benchmarks [38]. Given a ground truth pixel-wise segmentation set A and a predicted pixel-wise segmentation set B for a single class on a single image, the IoU is defined as:

$$\mathrm{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}. \tag{2.2}$$

IoU lies in $[0, 1]$, with a value of 1 indicating identical sets and 0 indicating disjoint sets. We report class-wise IoU for each tissue class (wing, envelope, seed), reflecting the differing biological importance and segmentation difficulty of each class. We additionally report the mean IoU (mIoU) as a single-number summary used to rank models during benchmarking. We compute IoU separately for each test image and summarize it as a mean and standard deviation across images, rather than pooling pixels over the whole test set, because per-image scores are more informative than

a single dataset-level score for segmentation evaluation [29]. For consistency, we compute IoU identically across all architectures.

2.2.6 Predictive Phenotype Networks

To explore the relationships between measured phenotypes, we produced predictive phenotype networks (PPNs) using iRF-LOOP [22]. Given an input set of extracted phenotypes for each pod, we computed the average value for each accession, which we used as input to the model. iRF-LOOP uses iterative random forest [9] to predict a single held-out phenotype from all others. Iterative random forest modifies the probability that each feature is sampled to create splits in the decision trees, weighting it by the feature's importance in the previous iteration. Because all iRF-LOOP tasks in our phenotype set are regression tasks, importance is measured by Mean Decrease in Impurity (MDI), which captures the normalized total reduction in mean squared error attributable to each feature across all trees, normalized to sum to one. Using this process, informative features are selected more frequently as the model converges. This process is repeated by holding out each phenotype, with the entire model output yielding pairwise importance values between phenotypes.

Using the iRF-LOOP output, we constructed PPNs, which take the form of a directed graph, where nodes represent phenotypes and edges represent predictive relationships. An edge from phenotype A to phenotype B indicates that A is predictive of B, with edge weight equal to the iRF-LOOP importance of A in predicting B. Using the raw iRF-LOOP output produced a dense network with all phenotypes connected to all others; however, we induced sparsity and highlighted only strong predictive relationships by retaining only edges whose weight falls in the top 5%. For interpretation, we also visualized the subset of retained edges that connect traits measured from different pod tissues. PPN outputs allowed us to perform an exploratory analysis of relationships between phenotypes before downstream analysis.

2.2.7 Genome-Wide Association Study

After measuring seed pod phenotypes and their relationships on a population scale, we performed a GWAS to find significant associations between SNP loci and variation in phenotypic traits.

Phenotype Curation and Heritability Estimation

Before running association tests, we assessed whether each phenotype had sufficient genetic signal to warrant GWAS by estimating broad-sense heritability in the dryland environment, in which the pod-morphology phenotypes analyzed here were measured. Because most accessions in our panel are unreplicated, H^2 was estimated on the subset of $n = 25$ replicated genotypes. For each phenotype, we fit a linear mixed model with genotype as a random effect and computed

$$H^2 = \sigma_G^2/(\sigma_G^2 + \sigma_E^2),$$

where σ_G^2 and σ_E^2 are genotypic and residual variance components. Phenotypes with $H^2 < 0.05$ were removed from downstream association analyses due to lacking sufficient heritable signal. We additionally restricted our analysis to phenotypes derived from the U-Net segmentation pipeline described in Section 2.2.3, excluding manually measured traits to maintain methodological consistency. Per-trait H^2 for the screened traits is reported in Table 2.1.

Genotypic Data

SNP genotypes were drawn from the imputed, lightly filtered whole-genome re-sequencing dataset of *Thlaspi arvense* natural accessions generated by Wu et al. [191]. We applied additional quality-control filtering: SNPs with severe departures from Hardy–Weinberg equilibrium ($p < 1 \times 10^{-150}$) were removed, genotypes with more than 10% missing calls were dropped, and the highly differentiated Armenian

Table 2.1: Broad-sense heritability (H^2) of the 34 U-Net-derived pod traits in the dryland environment that passed the $H^2 \geq 0.05$ screen and were carried into GWAS. H^2 was estimated from a linear mixed model with genotype as a random effect, fit on 25 replicated genotypes (2–3 replicate images per genotype; ∼15 pods per replicate image). Significance p is from a restricted likelihood-ratio test on the genotype variance component ($H_0 : \sigma^2_G = 0$). Traits marked † yielded significant GWAS associations (Table 2.4); note that heritability was necessary but not sufficient for a GWAS hit.

Trait	H^2	Significance (p)
envelope-to-wing area ratio	0.461	< 0.0001
envelope-to-total area ratio	0.423	0.0002
wing-to-envelope area ratio	0.409	0.0003
envelope perimeter	0.390	0.0016
envelope area	0.389	0.0017
wing-to-total area ratio	0.387	0.0003
wing area†	0.366	0.0010
wing HSV saturation	0.321	0.0022
wing perimeter	0.302	0.0060
seed area	0.297	0.0214
envelope-to-wing perimeter ratio	0.292	0.0006
wing-to-envelope perimeter ratio	0.287	0.0009
wing-to-seed area ratio†	0.285	0.0077
seed perimeter	0.263	0.0334
seed-to-wing area ratio	0.247	0.0117
envelope-to-total perimeter ratio	0.230	0.0051
wing CIELAB b^*	0.227	0.0525
envelope HSV saturation	0.220	0.0418
seed count	0.218	0.0319
envelope CIELAB b^*†	0.217	0.0315
seed RGB red	0.204	0.0881
envelope-to-seed area ratio†	0.200	0.0501
seed CIELAB b^*	0.190	0.0627
seed-to-envelope area ratio	0.188	0.0681
seed-to-total area ratio	0.188	0.0453
seed HSV hue	0.185	0.0796
seed HSV saturation	0.155	0.1048
wing RGB blue	0.131	0.1770
seed HSV brightness	0.120	0.1518
seed CIELAB lightness	0.097	0.1672
envelope RGB green	0.089	0.2169
envelope CIELAB lightness	0.086	0.2213
seed RGB blue	0.083	0.1234
envelope HSV brightness	0.068	0.2530

subpopulation was excluded. The resulting dataset comprised 423 genotypes and 1,975,204 SNPs. Association testing was restricted to the 189 accessions with matched U-Net-derived pod phenotypes.

Association Testing

We fit four single- and multi-locus association models implemented in GAPIT (Genome Association and Prediction Integrated Tool) [180]: MLM [197], MLMM [153], FarmCPU [98], and BLINK [66]. We further restricted the marker set to the 1,759,624 SNPs with a minor allele frequency (MAF) ≥ 0.03. We ran GWAS independently per phenotype and retained the union of SNPs reaching $p < 1 \times 10^{-6}$ under any of the four models. Because extensive linkage disequilibrium makes the per-SNP tests non-independent, a Bonferroni correction is overly stringent; we therefore treated raw $p < 1 \times 10^{-6}$ as a suggestive association cutoff and deferred false-positive control to GRIN (Section 2.2.8).

SNP-to-Gene Assignment

After finding significant loci, we mapped SNPs to candidate genes using a two-pronged approach. For each significant SNP, we first fit a local linkage disequilibrium (LD) decay curve and defined a data-driven window around the SNP based on fitted decay distance. We then identified all genes whose coordinates fall within this window on the pennycress TAV2 reference genome. When LD decay could not be reliably fit for a SNP, we instead assigned the two nearest flanking genes (one upstream and one downstream) by physical distance. Because functional annotation of the pennycress genome is limited, we annotated each assigned pennycress gene by its best BLAST-hit Arabidopsis ortholog from TAIR [44] to enable downstream biological interpretation.

Candidate Gene Set Construction

We aggregated candidate genes by taking the union of genes mapped from significant SNPs, regardless of which GWAS method produced the original association. We restricted our downstream analysis to traits with considerable heritability ($H^2 \geq 0.05$) and to phenotypes derived from the U-Net segmentation pipeline. Under these restrictions, four pod-related phenotypes (envelope CIELAB b^*, envelope-to-seed area ratio, wing area, and wing-to-seed area ratio) yielded significant GWAS hits, mapping to 6 unique SNPs and 69 pennycress genes, 64 of which had annotated Arabidopsis TAIR orthologs. We note that these four phenotypes are highly correlated, as they are geometric ratios computed from a common set of pod measurements. They therefore do not represent four biologically independent signals. These 64 orthologs constitute the candidate ortholog set, which we filtered with GRIN (Section 2.2.8) before MENTOR module derivation (Section 2.2.8).

2.2.8 MENTOR Module Derivation

GRIN Filtering

Before using our candidate genes as input to MENTOR for module construction, we applied GRIN [165] to reduce potential false-positive genes. Both GRIN and MENTOR operate over a nine-layer *Arabidopsis thaliana* multiplex network spanning a pool of 26,605 genes, in which each layer encodes a distinct line of biological evidence: protein–protein interaction, transcriptional regulation from two independent sources, gene co-expression, co-evolution, knockout phenotype similarity, methylation-derived predictive relationships, metabolic pathway membership, and a diversity-derived layer. All nine layers are undirected and unweighted, and each contributes equally to network propagation; per-layer sources and sizes are given in Table B.2. Given a candidate gene set, whose members serve as the seed nodes from which network propagation begins, and this multiplex network [77], GRIN operates in two stages. First, it runs random walk with restart (RWR) from the seed nodes, yielding a

rank vector of the genes in the network most closely related to them; in parallel, it runs RWR on 100 randomly sampled gene sets of the same size, creating a null distribution rank vector. Next, GRIN uses a sliding-window Mann-Whitney U test to evaluate the likelihood of observing the seed-node RWR ranks at random. With this sliding window, a threshold can be defined where the ranks become insignificant. The resulting significant genes are retained, while all others are dropped. GRIN retains genes that are more tightly connected in the multiplex network to the rest of the candidate set; this network proximity is taken as a proxy for shared biological function. In this paradigm, GRIN assumes that well-connected genes may jointly contribute to the associated phenotype. We refer to the genes GRIN retains as the network-supported gene set.

Module Construction

We applied MENTOR, a multi-step framework for deriving functionally related genetic modules, using the multiplex network and the network-supported gene set as input [166]. We used the *Arabidopsis thaliana* multiplex because this close relative has richer functional annotation and network resources than pennycress.

Starting from the network-supported gene set (Section 2.2.8), MENTOR applies RWR as a diffusion method to measure the probability of traveling to each gene in the multiplex from all genes in that set. Each seed node produces its own score vector over all multiplex nodes, with indices representing the probability of reaching each node from that seed. The score vectors are then rank-ordered, truncated at the elbow of the mean score-versus-rank curve, and converted into a dissimilarity matrix by taking $(1-\rho)$, with ρ representing Spearman correlation, between each pair of genes' rank-ordered score vectors. With the resulting dissimilarities, MENTOR creates a dendrogram using agglomerative clustering with complete linkage as our module distance measure. This dendrogram is then thresholded to examine modules of a certain size.

2.2.9 MENTOR-IA Interpretation

After deriving a set of modules with MENTOR, we used MENTOR-IA to explore the biological mechanisms associated with these modules [178]. MENTOR-IA is a multi-agent LLM framework for mechanistic interpretation, in which specialized agents each contribute to a distinct line of evidence. These agents include a mygene interpreter, which utilizes gene summaries, GO terms, and pathway annotations to explore each gene's role individually; a literature agent, which explores mechanistic connections in related literature via Europe PMC; an enrichment agent, which tests whether a module is enriched for pathways in GO, KEGG, or Reactome; a network agent, which uses subgraph evidence to refine the interpretation; and a meta aggregator, which synthesizes evidence to produce a coherent mechanistic summary.

Using the evidence provided by MENTOR-IA as a starting point, we manually examined pennycress and *Brassicaceae* literature to assess whether the proposed mechanisms were consistent with known biology.

2.2.10 Statistical Analysis

This section summarizes the statistical analyses applied at each stage of the pipeline. Models, sample sizes, and significance thresholds are specified in the referenced sections, and the tables cited below report the resulting statistics.

Segmentation accuracy was evaluated with per-class IoU and mIoU (Section 2.2.5), computed per test image and summarized as mean ± standard deviation in Table 2.2. We additionally profiled the computational efficiency of each benchmarked architecture—parameter count, mean per-image inference time, and peak allocated GPU memory on the test set—reported in Table 2.3 and Figure 2.6.

Predictive phenotype networks were constructed with iRF-LOOP, using Mean Decrease in Impurity as the edge-weight statistic (Section 2.2.6). Broad-sense heritability was estimated from a linear mixed model with genotype as a random effect (Section 2.2.7); per-trait H^2 and the significance of the genotype variance component

are reported in Table 2.1. Association testing used four models implemented in GAPIT (Section 2.2.7); retained associations, lead SNPs, and their unadjusted p-values are reported in Table 2.4. Candidate genes were then filtered on network connectivity using the sliding-window Mann-Whitney U test implemented in GRIN (Section 2.2.8).

2.3 Results

We present our results in two stages: the first converts raw pod scans into a per-accession trait matrix, and the second carries that matrix toward candidate genetic mechanisms. In the first stage, we benchmarked segmentation architectures and selected U-Net, extracted per-pod phenotypes from the predicted masks, and applied the trained model at population scale to assemble the trait matrix. We then examined the trait matrix for biological coherence using a predictive phenotype network; this analysis is exploratory and validates the extracted phenotypes rather than selecting which traits enter the downstream analysis. In the second stage, we filtered the trait matrix by heritability, combined the retained traits with genome-wide resequencing genotypes for GWAS, and mapped the associated SNPs to candidate genes. We then pruned those candidates with GRIN, resolved the survivors into modules with MENTOR, interpreted the modules with the assistance of MENTOR-IA, and assessed whether the prioritized candidates are expressed in pod tissue.

2.3.1 Segmentation Benchmark

We evaluated our models' segmentation performance on 5 raw images containing 65 seed pods in total. Per-class IoU and across-class mIoU are reported in Table 2.2. The boundary down-weighted U-Net achieved the highest mIoU at 85.58%, narrowly ahead of the baseline U-Net at 85.53% and Mask2Former at 85.03%. These top scores were close relative to per-image variation, so we interpret the leading architectures as

Table 2.2: Segmentation benchmark results. Per-class IoU (wing, envelope, seed) and mean IoU (mIoU), reported as mean ± standard deviation (%) across the five full test images; mIoU is the mean of the three tissue-class IoUs. Standard deviations are population values (ddof = 0) computed over the five test images; nnU-Net was scored on pod crops under its own adaptive pipeline (Section 2.2.5) and grouped back to the source full images before summary. Bold values denote the best mean score in each column.

Model	Wing	Envelope	Seed	mIoU
U-Net	95.60 ± 0.98	82.84 ± 2.31	78.14 ± 3.83	85.53 ± 1.45
U-Net-Down	95.53 ± 0.92	**82.94** ± 2.35	**78.28** ± 3.59	**85.58** ± 1.46
U-Net-Up	95.58 ± 0.95	82.24 ± 1.79	77.60 ± 3.81	85.14 ± 1.22
nnU-Net	**95.77** ± 0.93	81.78 ± 1.44	75.40 ± 4.65	84.32 ± 1.39
DeepLabV3+	95.19 ± 0.90	81.53 ± 2.80	75.80 ± 3.93	84.17 ± 1.72
DeepLabV3+-Up	95.28 ± 0.92	81.70 ± 2.72	75.64 ± 3.76	84.21 ± 1.64
DeepLabV3+-Down	95.14 ± 0.91	81.31 ± 2.69	75.89 ± 3.79	84.11 ± 1.52
SegFormer	95.03 ± 0.99	80.45 ± 2.24	75.07 ± 3.95	83.52 ± 1.24
SegFormer-Up	94.99 ± 0.99	80.22 ± 2.18	75.23 ± 4.07	83.48 ± 1.23
SegFormer-Down	95.31 ± 1.04	81.68 ± 2.13	76.36 ± 4.29	84.45 ± 1.15
Mask2Former	95.65 ± 1.15	82.80 ± 3.67	76.65 ± 4.41	85.03 ± 2.05
SAM 3	95.15 ± 1.54	76.42 ± 4.30	65.62 ± 9.19	79.07 ± 4.04
SAM 3-Up	95.35 ± 1.54	76.78 ± 4.06	68.15 ± 7.33	80.09 ± 3.10
SAM 3-Down	94.95 ± 1.82	73.49 ± 3.92	57.61 ± 9.39	75.35 ± 3.32

Table 2.3: Computational efficiency of the benchmarked segmentation architectures. Parameter count (millions), mean inference time per full test image ($n = 5$, ROCm), and peak allocated GPU memory are summarized once per architecture. For architectures with boundary-weighting variants, inference time is averaged across the measured variants because parameter counts and peak allocated memory are architecture-level quantities. For nnU-Net, inference time and peak memory were computed by grouping its pod-crop predictions back to their five source full images. SAM 3 is adapted with LoRA, so only 5.70 M of its parameters are trainable. Bold values denote the lowest cost in each column.

Model	Params (M)	Inference time (s/image)	Peak memory (GB)
U-Net	**12.57**	**43.1**	**0.06**
nnU-Net	65.21	202.2	0.65
DeepLabV3+	45.67	122.7	0.18
SegFormer	81.97	412.1	0.39
Mask2Former	215.45	377.0	0.90
SAM 3	459.74	400.7	1.81

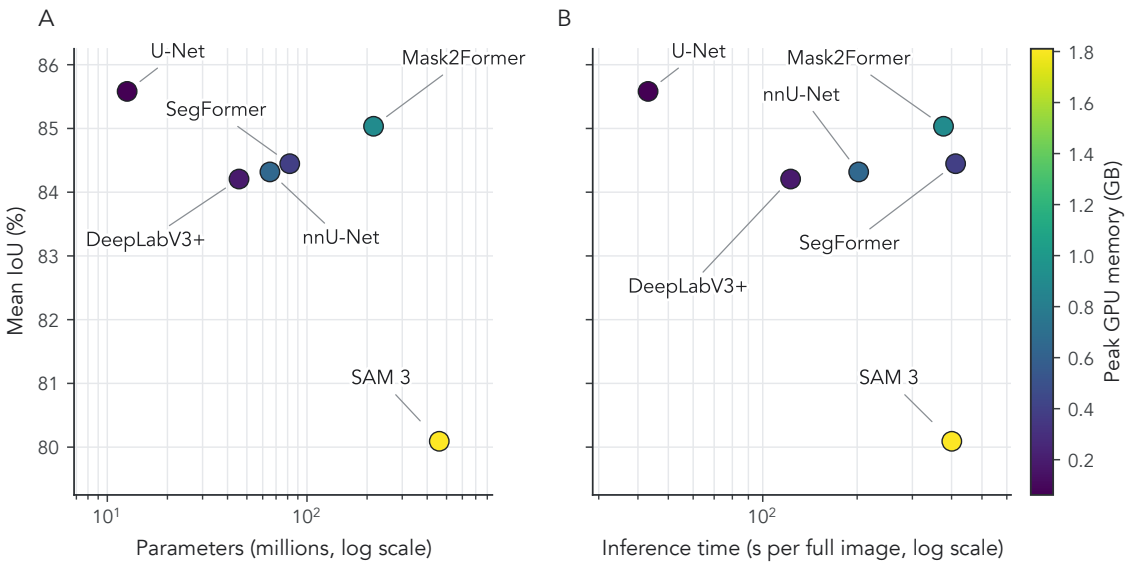

Figure 2.6: Accuracy and computational efficiency of benchmarked segmentation architectures. (A) Best mean IoU (mIoU) achieved by each architecture across its boundary-weighting variants plotted against total parameter count on a log scale. (B) Best mIoU plotted against mean full-image inference time on a log scale, where lower values indicate faster segmentation. Point color and the shared color bar encode peak allocated GPU memory during full-image inference; nnU-Net values were grouped from pod-crop predictions back to the source full images.

comparable in accuracy rather than as statistically separable. SAM 3 was the clearest laggard, especially for seed segmentation, where IoU fell as low as 57.61%.

Efficiency separated the leading architectures more strongly than accuracy (Table 2.3; Figure 2.6). U-Net used 12.57 million parameters, required 43.1 s per full test image, and peaked at 0.06 GB allocated GPU memory. nnU-Net, DeepLabV3+, SegFormer, Mask2Former, and SAM 3 required more parameters and longer inference times; Mask2Former had a similar mIoU but used 215.45 million parameters, required 377.0 s per full test image, and peaked at 0.90 GB allocated GPU memory. Thus, U-Net provided the most favorable accuracy-efficiency tradeoff for population-scale phenotyping in this small, specialized pod-segmentation task. We found no prior work measuring semantic segmentation of pennycress to compare with our model benchmark suite.

2.3.2 Population-Scale Phenotyping

Having selected U-Net as the most favorable benchmark on accuracy and efficiency, we applied it at population scale to characterize phenotypic structure across the pennycress accession panel. We measured phenotypes from 12,253 seed pods across 768 accessions. Each image yielded approximately 16 retained pod crops on average and took around 2 minutes of model inference time to segment, compared to approximately 1.5 hours of hand segmentation per image, yielding an estimated 30- to 45-fold throughput gain depending on annotator pace.

We used the feature extraction pipeline described in Section 2.2.4 to construct an open-source dataset of over 50 phenotypes for all 12,253 seed pods. For visualization and correlation summaries, we excluded detections for which the watershed step returned zero seeds. Figure 2.7 shows the distributions of three key phenotypes: wing area, envelope area, and seed area. These traits correspond directly to the three annotated tissue classes, making their distributions a useful biological plausibility check on segmentation output. Each trait was right-skewed and heavy-tailed,

consistent with the log-normal-like distributions commonly observed in quantitative plant traits [79].

Consistent with the wing-seed regulatory association described in Section 2.1, wing area was positively correlated with seed area and seed count (Pearson $r = 0.51$ and 0.30, respectively; Figure 2.8). The resulting per-accession phenotype matrix served two roles: as input to iRF-LOOP for PPN construction (Section 2.2.6), and as the candidate trait pool for heritability filtering and GWAS (Section 2.2.7).

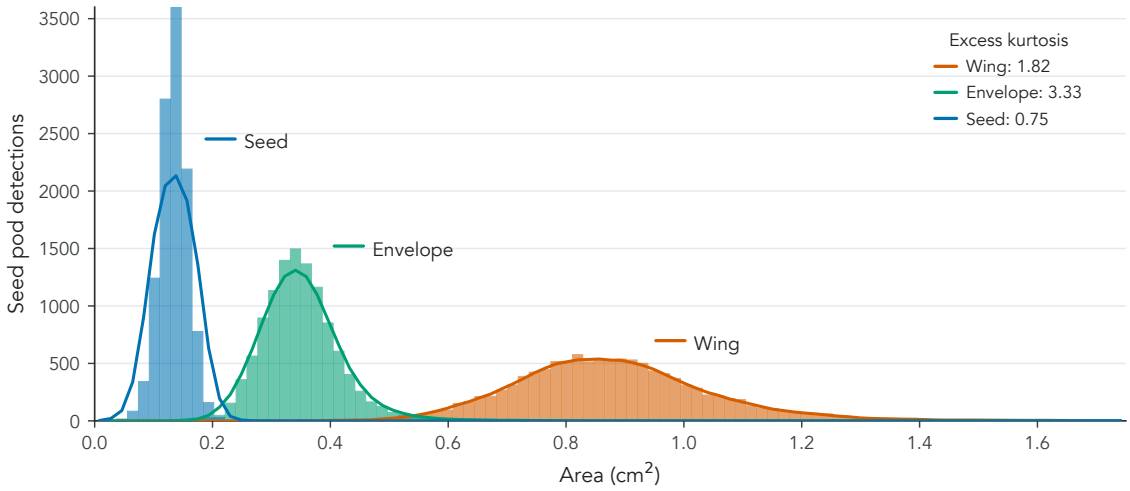

Figure 2.7: Distribution of seed-pod component areas across non-empty population-scale detections. Histograms show the measured areas (cm^2) of wing, envelope, and seed tissues after excluding detections with zero watershed seed markers. Colored lines show five-bin moving averages of histogram counts, and inset values report excess kurtosis, quantifying heavier-than-normal distribution tails. Seed area is concentrated at smaller values, envelope area at intermediate values, and wing area at larger values; all three traits are right-skewed and log-normal-like, consistent with natural variation in plant morphology.

2.3.3 Predictive Phenotype Network

After extracting phenotypes from our dataset, we used iRF-LOOP to construct PPNs to check the biological plausibility of extracted phenotypes. Running iRF-LOOP on the entire dataset yielded a PPN with 48 nodes and 108 directed edges after filtering the network to include only the top 5% of relationships (Figure 2.9). Despite being

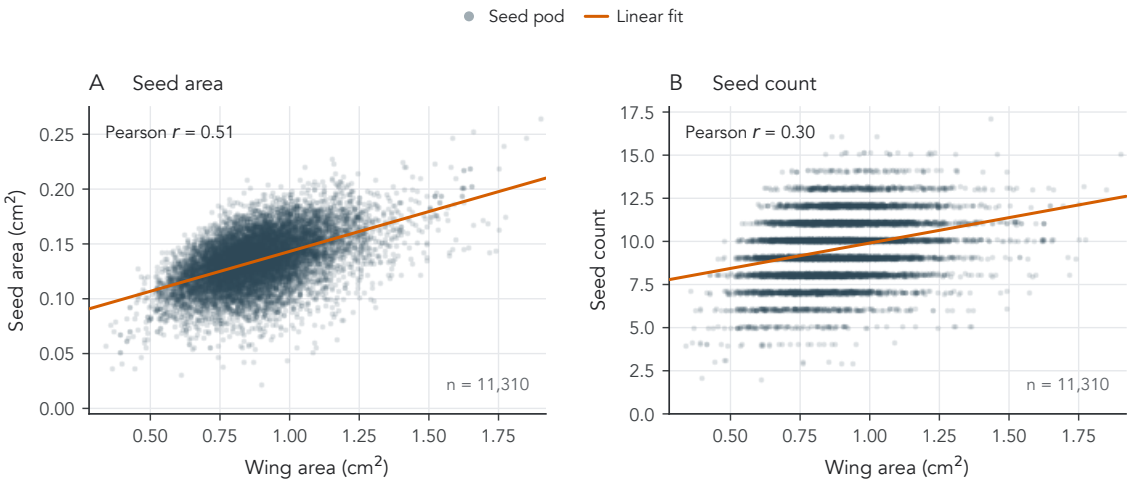

Figure 2.8: Relationship between wing area and seed traits across non-empty population-scale detections. Wing area is plotted against (A) seed area and (B) seed count, with each point representing one seed pod and the solid line showing the linear least-squares fit. Points in (B) are vertically jittered by less than 0.1 seed for visibility; correlations and fitted lines use the unjittered seed-count values. Wing area is positively correlated with both seed area (Pearson $r = 0.51$) and seed count ($r = 0.30$), consistent with wing–seed regulatory association in *Brassicaceae*.

a directed network, the PPN exhibited high reciprocity, indicating that predictive relationships tend to be mutual, which is expected for many geometrically derived traits.

The PPN separated into three connected components after filtering, each dominated by a single feature type: a 27-node component containing all color features across all three tissues, a 12-node component containing absolute geometry and area ratios, and a 9-node component containing all perimeter ratios. This separation by measurement type is expected given how tightly color and geometric features covary within each class.

To interpret the cross-tissue relationships more directly, we extracted the subset of PPN edges that connect traits across pod tissues (Figure 2.10). This focused subset separates two recurring patterns: pigment coupling among color traits and size coupling among envelope, seed, and wing geometry traits.

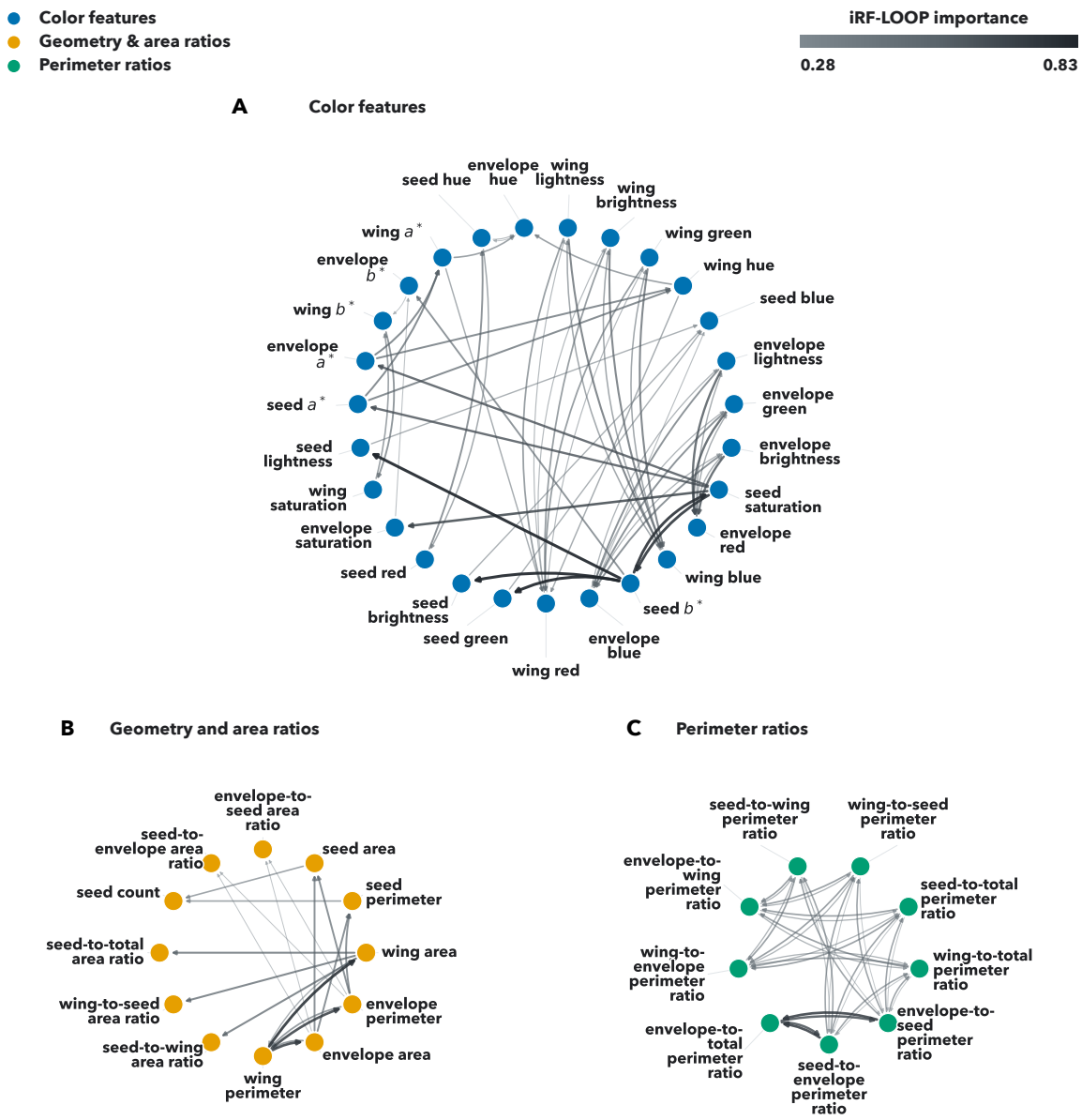

Figure 2.9: Predictive phenotype network (PPN) for pod-related traits after retaining the top 5% of iterative Random Forest Leave-One-Out Prediction (iRF-LOOP) relationships. Nodes are measured phenotypes; node color and panel membership denote feature classes: (A) color features, (B) geometry and area ratios, and (C) perimeter ratios. Directed arrows indicate predictor-to-response relationships, and edge width and darkness encode iRF-LOOP importance. The filtered network contains 48 nodes and 108 directed edges. In color-feature labels, a^* and b^* are CIELAB color channels.

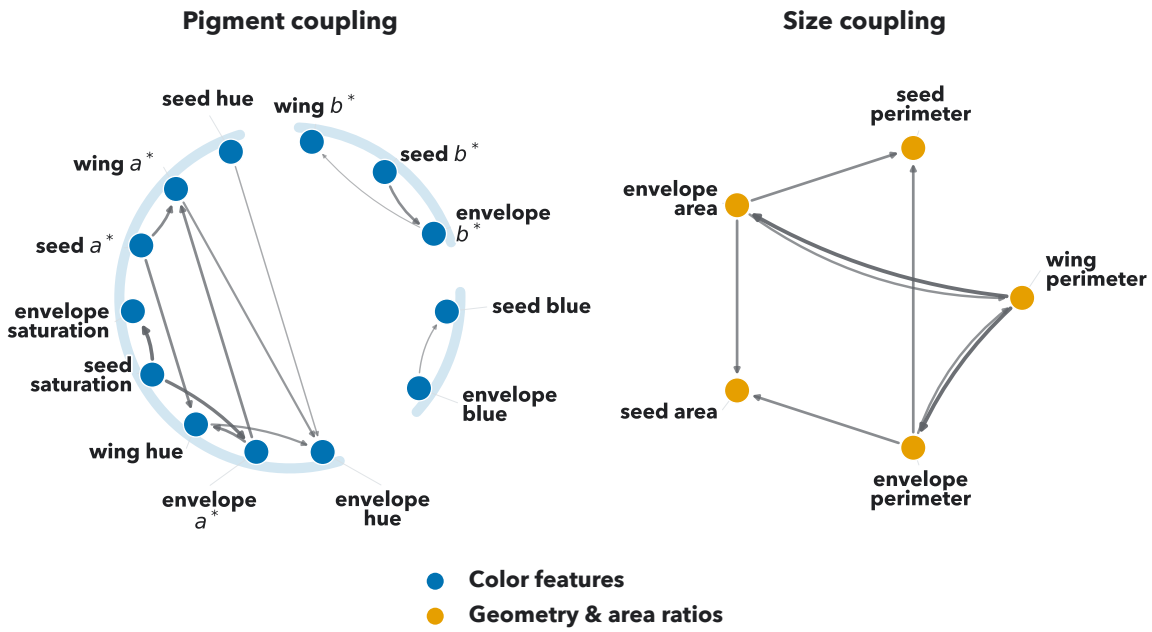

Figure 2.10: Focused subset of cross-tissue predictive phenotype network relationships. The subset highlights PPN edges connecting traits across pod tissues, separated into pigment coupling and size coupling groups. Nodes are measured phenotypes, with blue nodes denoting color features and orange nodes denoting geometry and area-ratio features. Directed arrows indicate predictor-to-response relationships, and edge width and darkness encode iRF-LOOP importance. Pale blue arcs visually group related pigment-feature relationships. In color-feature labels, a^* and b^* are CIELAB color channels.

Within this focused subset, several informative relationships emerged that could not be explained by geometric identities alone. Most notably, both envelope area and perimeter were predictive of seed area and perimeter (iRF-LOOP importance ≈ 0.56), consistent with the pod's active role in seed development and resource allocation in *Brassicaceae* [11]. This relationship mirrors the envelope-to-seed area ratio, which was the most recurrent of our significant GWAS signals (Section 2.2.7). Similarly, seed and envelope color features were predictive of each other in CIELAB b^*, RGB blue, and HSV hue spaces. This suggests that color features, which provide a proxy for pod maturity and pigmentation, may be co-regulated across tissues. These relationships

are correlative and do not imply causality, but they indicate that extracted phenotypes capture biologically coherent structure beyond trivial geometric relationships.

2.3.4 Heritability of U-Net-Derived Pod Traits

Of the 149 phenotypes available for this panel, 52 were extracted by our U-Net pipeline. Screening these for heritable genetic signal retained 34 traits, with H^2 ranging from 0.068 to 0.461 (Table 2.1); four of the 34 ultimately yielded significant GWAS associations.

Heritability was strongly structured by trait class. The 19 retained geometry and ratio traits spanned H^2 0.188–0.461, whereas the 15 retained color traits spanned 0.068–0.321, and all seven of the most heritable traits were geometric. The most heritable color trait, wing HSV saturation ($H^2 = 0.321$), fell below every one of them. This separation anticipates the association results below, in which three of the four traits with significant hits are geometry or area ratios.

Heritability was necessary but not sufficient for a GWAS hit, and the retained set includes traits whose genotype variance components were weakly resolved. Fourteen of the 34 retained traits did not reach nominal significance on the restricted likelihood-ratio test ($p > 0.05$), and 12 of those 14 were color traits. Envelope-to-seed area ratio, which produced the most recurrent association signal, was itself borderline ($H^2 = 0.200$, $p = 0.0501$). We therefore treat the downstream candidates as exploratory rather than as well-powered genetic findings. Per-trait H^2 for all 52 U-Net-derived traits, including the 18 that fell below the screen, is reported in Table B.1.

2.3.5 Genome-Wide Association and Gene Mapping

We performed GWAS on the U-Net-derived pod traits on a panel of 189 accessions using four GAPIT models (MLM, MLMM, FarmCPU, and BLINK). We retained candidate SNPs with raw-$p < 1 \times 10^{-6}$ (a suggestive association cutoff; a Bonferroni threshold is overly stringent given linkage disequilibrium among SNPs) and MAF

≥ 0.03, deferring false-positive control to GRIN filtering in the module derivation stage.

GWAS yielded six unique SNPs across four phenotypes: envelope CIELAB b^*, envelope-to-seed area ratio, wing area, and wing-to-seed area ratio. The most recurrent signal came from envelope-to-seed area ratio. Our GWAS results are summarized in Table 2.4.

Table 2.4: Summary of retained GWAS associations for heritable, U-Net-derived pod traits (dryland environment), tested across 189 genotyped accessions. Associations are reported at a suggestive raw $p < 1 \times 10^{-6}$ cutoff; linkage disequilibrium among SNPs makes per-SNP tests non-independent, so a Bonferroni threshold is overly conservative. H^2, broad-sense heritability; SNPs, number of retained single-nucleotide polymorphisms; Models, GAPIT models (MLM, MLMM, FarmCPU, BLINK) producing the associations; Lead SNP, most significant SNP (chromosome_position); Lead raw p, its unadjusted p-value.

Trait	H^2	SNPs	Models	Lead SNP	Lead raw p
envelope CIELAB b^*	0.217	1	MLMM	`4_8127381`	8.71×10^{-7}
envelope-to-seed area ratio	0.200	3	BLINK, FarmCPU, MLMM	`5_963442`	1.04×10^{-7}
wing area	0.366	1	MLMM	`7_63065229`	4.14×10^{-7}
wing-to-seed area ratio	0.285	1	MLMM	`2_2343404`	6.06×10^{-7}

We mapped the six retained SNPs from GWAS to pennycress genes using the mixed LD-decay and distance-based approach described in Section 2.2.7. For each SNP, we defined a window centered on the SNP with a width set by local LD-decay distance and assigned the annotated genes falling in that window, retaining any gene whose body directly contained that SNP. Windows spanned approximately 24.6–49.6 kb, with strong LD-decay fits ($R^2 = 0.944$–0.989), reflecting the different recombination contexts of each chromosomal region. Because all six SNPs yielded reliable LD-decay fits, every candidate gene was assigned from an LD window, and the distance-based fallback was not used.

This procedure produced 69 candidate pennycress genes, with half of the loci tracing to the envelope-to-seed area trait and the remaining loci tracing to the other

three GWAS traits. We mapped these genes to their nearest Arabidopsis orthologs in TAIR [44] to enable cross-species functional annotation and module derivation. This mapping yielded 64 unique orthologs, as several pennycress genes shared a single Arabidopsis ortholog. This ortholog set constitutes the candidate ortholog set carried forward to GRIN filtering and module derivation (Section 2.2.8).

2.3.6 MENTOR Module Derivation

GRIN filtering reduced the candidate ortholog set of 64 genes to 33, pruning 31 that lacked sufficient network support; these 33 constitute the network-supported gene set. We applied MENTOR to this set, using the process and standard settings described in Section 2.2.8, resolving it into three modules of 5, 11, and 17 genes (Figure 2.11). We then used these modules for mechanistic interpretation (Section 2.2.9).

2.3.7 MENTOR-IA Exploratory Interpretation

We used MENTOR-IA to conduct an exploratory interpretation of the three modules derived by MENTOR (Section 2.2.8). Module 1, which contained 17 Arabidopsis orthologs, was run with the enrichment agent, literature agent, mygene interpreter, and meta aggregator. Module 2, which contained 11 orthologs, was run with the same agents as Module 1. Module 3, which contained 5 orthologs, was run with only the mygene interpreter and meta aggregator, as the small gene count limited the power of enrichment analyses. Because our modules did not have an associated network, we did not use the network agent for analysis.

MENTOR-IA produced per-agent summaries for each module, along with a final interpretation that synthesized evidence across agents. This interpretation included keyword themes, enrichment statistics from GO and KEGG, and a mechanistic summary with candidate supporting citations. Across modules, the proposed mechanisms recovered known *Brassicaceae* pod and seed biology, demonstrating our pipeline's ability to connect measured phenotypes to candidate mechanistic

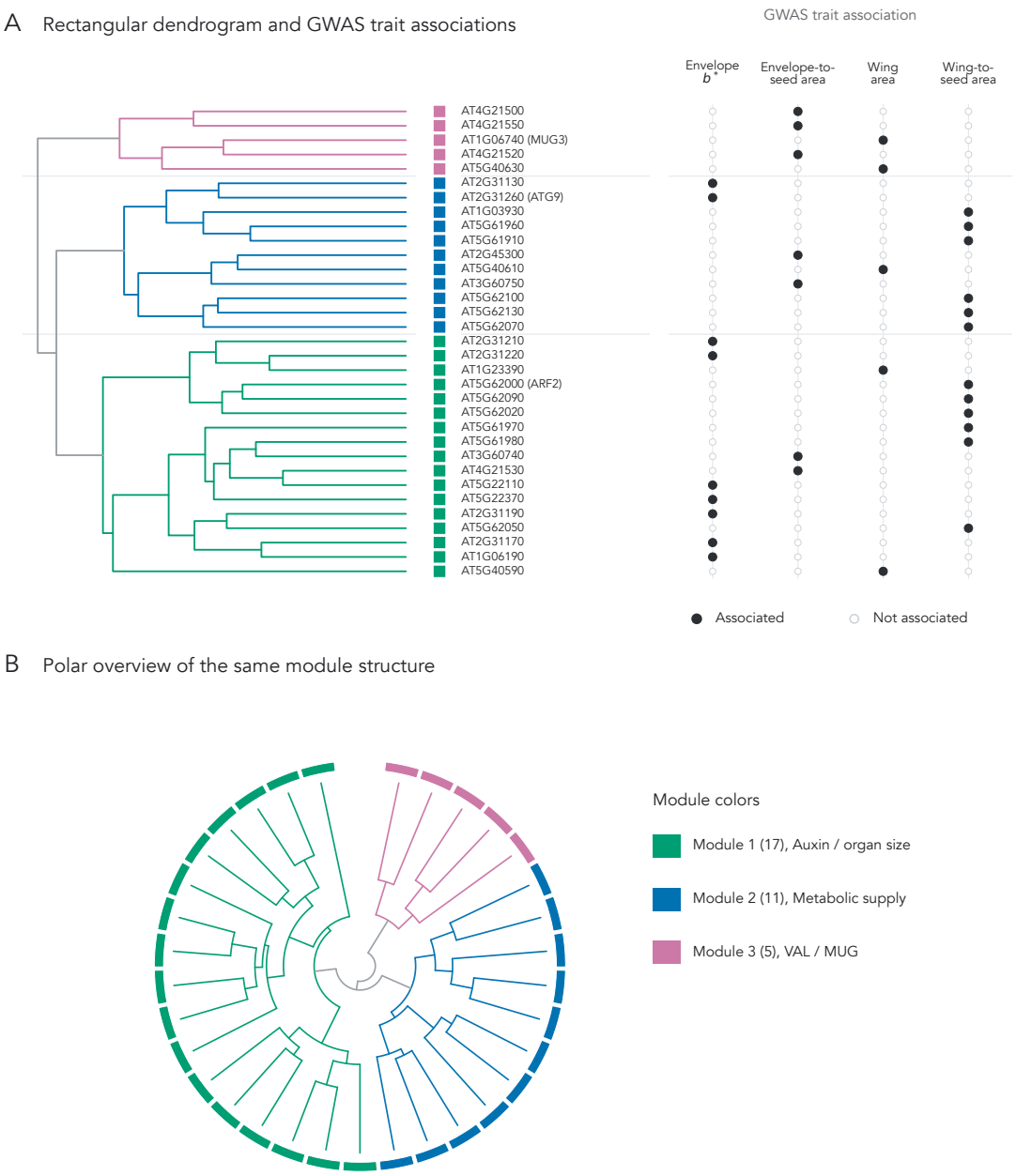

Figure 2.11: MENTOR dendrogram of the 33 GRIN-refined Arabidopsis orthologs retained from U-Net-derived pod-trait GWAS candidates. (A) Complete-linkage clustering of random-walk-with-restart (RWR) dissimilarities, with the adjacent trait matrix indicating which of the four GWAS traits—envelope CIELAB b^*, envelope-to-seed area ratio, wing area, and wing-to-seed area ratio—each gene was associated with. (B) Polar view of the same topology, emphasizing the three MENTOR modules. Branch and outer-ring colors indicate Module 1 (17 genes; auxin, organ-size, and reproductive development), Module 2 (11 genes; metabolic supply, autophagy, and nutrient remobilization), and Module 3 (5 genes; VAL/MUG transposon-derived regulators).

explanations. We manually cross-checked the agent output against pennycress and *Brassicaceae* literature, finding the mechanistic summaries to be consistent with reported biology.

Because functional interpretation used Arabidopsis orthologs, the mechanisms below should be read as candidate hypotheses rather than validated pennycress-specific effects. For clarity, gene-symbol mentions below refer to Arabidopsis orthologs; parenthetical IDs give the Arabidopsis locus followed by the corresponding pennycress TAV2 locus or loci.

Module 1 Interpretation

We confirmed that the MENTOR-IA module 1 analysis recovered mechanistic themes centered around auxin-mediated control of organ size and reproductive development, with supporting themes connecting to auxin and cell division more broadly. [149] demonstrated that Arabidopsis *ARF2* (`AT5G62000`; pennycress `TAV2_LOCUS7857`) represses auxin-responsive transcription, limiting integument cell proliferation, and loss of *ARF2* function leads to enlarged seeds and organs. [125] found that *ARF2* genetically interacts with *SEEDSTICK* and *GORDITA* during seed development, linking this auxin-response factor to integument growth and seed-coat differentiation. These findings provide the strongest candidate bridge to envelope-to-seed area ratio, while any connection to wing area remains more indirect through broader auxin-mediated organ-growth biology.

Module 1's link to auxin and cell division is further supported by the presence of Arabidopsis *APC4* (`AT4G21530`; pennycress `TAV2_LOCUS24640`), which encodes a subunit of anaphase-promoting complex (APC/C). [183] demonstrated that Arabidopsis *apc4* mutants exhibit defects in female gametogenesis and embryogenesis, with aberrant embryo cell division and distorted auxin distribution. [99] also implicated *ARF2*, together with *ARF4* and *ARF5*, in gametophyte development, reinforcing the reproductive-development theme. Arabidopsis *RUS2*/*WXR1* (`AT2G31190`; pennycress `TAV2_LOCUS13944`) provides a second auxin-related but less direct line

of evidence: [45] showed that *RUS2*/*WXR1* is required for normal polar auxin transport and PIN/AUX1 abundance in roots, while [89] placed *RUS2* with *RUS1* in a root UV-B-sensing pathway. Thus, the proposed *RUS2* connection to wing growth is an inference from auxin-distribution biology rather than a demonstrated pod-morphology effect. The module also surfaced a possible connection to color phenotypes such as envelope CIELAB b^*; *arf2* mutants show delayed senescence and altered chlorophyll retention in leaves [35]. Because this evidence comes from leaves and developmental timing rather than pod-envelope color directly, the connection to pod color remains tentative.

In addition to the literature-supported themes that we manually verified, the MENTOR-IA enrichment agent returned several significant GO and KEGG terms consistent with reproductive development, such as floral whorl development ($p = 9.8 \times 10^{-5}$), ovule development ($p = 3.6 \times 10^{-3}$), and embryo development ending in seed dormancy ($p = 3.6 \times 10^{-3}$), providing additional support for the proposed developmental theme.

Despite the strong theme and supporting evidence around auxin-mediated control of organ size and reproductive development, the remaining orthologs in module 1 had only gene-level annotation and no clear trait-linking literature in the MENTOR-IA output, so we leave their exploration for future work and specify them as additional candidates. The module 1 interpretation is consistent with known biology of pod and seed development in *Brassicaceae*.

Module 2 Interpretation

Module 2 is centered around metabolic supply, autophagy, nutrient remobilization, and chloroplast carbon metabolism. Unlike module 1, which provides evidence of a direct developmental regulator in *ARF2*, module 2 provides an indirect hypothesis in which metabolic supply from the pod to the seed may influence pod growth. The strongest candidate connection is to envelope-to-seed area ratio and pod filling

because the literature links these pathways to seed resource allocation rather than directly to pod morphology.

Arabidopsis *ATG9* (AT2G31260; pennycress TAV2_LOCUS12330) anchors the module's autophagy signal. [157] showed that *ATG9* contributes to autophagic flux, while [71] found that *atg9* mutants retain constitutive autophagy in root tips, supporting a modulatory rather than strictly essential role. [78] and [103] place *ATG9* in autophagosome formation and closure pathways, and [88] provides structural evidence for *ATG9*'s role in autophagy via cryo-EM, showing that it is the core integral membrane autophagy protein. [52] observed that *atg* mutants show reduced nitrogen remobilization to seeds under both low- and high-nitrate conditions, and [34] demonstrated that autophagy affects resource allocation and storage-protein accumulation during seed development. These findings suggest that variation in autophagy may affect pod filling or envelope-to-seed nutrient allocation, providing a candidate link to phenotypes like envelope-to-seed area ratio.

Arabidopsis *TKL1* (AT3G60750; pennycress TAV2_LOCUS16016) provides a second line of evidence for metabolic supply and chloroplast carbon metabolism. [138] found that *TKL1* is phosphorylated at Ser428 in a light- and chloroplast-dependent manner, suggesting that it may be a regulatory point for carbon flux in the Calvin-Benson-Bassham (CBB) cycle and oxidative pentose phosphate pathway (OPPP). Additionally, [80] demonstrated that altered plastid transketolase dosage disrupts growth through thiamine- and TPP-related metabolic constraints. Because carbon supply is important to seed growth, we infer that *TKL1* may provide a second, indirect link to pod growth and envelope-to-seed area, supporting our mechanistic supply hypothesis.

Additionally, [156] adds a redox-homeostasis line of evidence, showing that Arabidopsis *GPDHc1* (AT5G40610; pennycress TAV2_LOCUS25599) controls plant $NADH/NAD^+$ ratios through the mitochondrial glycerol-3-phosphate shuttle. Although the OPPP is not directly implicated in this work, it supports a broader

interpretation in which module 2 captures genes involved in carbon and redox balance during pod and seed development.

Outside of the autophagy and metabolic supply themes, the remaining module 2 genes had only gene-level annotation and no clear trait-linking literature, so we specify them as additional candidates for future exploration. Overall, the module 2 interpretation is consistent with known biology of seed filling and envelope-to-seed biomass allocation, providing a plausible but indirect mechanistic hypothesis connecting our phenotypes to metabolic supply and autophagy.

Module 3 Interpretation

Since module 3 yielded only sparse gene-level annotation and lacked enrichment and literature support, we did not advance a mechanistic interpretation for it; its genes remain candidates for future analysis.

Expression-Based Plausibility Check

We further evaluated the plausibility of our candidate mechanisms by examining gene expression in pennycress leaf, pod, and root tissues using the dataset from [25]. For the purpose of connecting candidate genes to pod phenotypes, we focused on expression in pod tissues. The strongest literature-supported candidate loci were expressed in pods. These included the module 1 candidates *ARF2* (`TAV2_LOCUS7857`), *APC4* (`TAV2_LOCUS24640`), and *RUS2* (`TAV2_LOCUS13944`), and the module 2 candidates *ATG9* (`TAV2_LOCUS12330`), *TKL1* (`TAV2_LOCUS16016`), and *GPDHc1* (`TAV2_LOCUS25599`). By contrast, several lower-priority candidate loci were not highly expressed in pod tissues. The *BAG2* orthologs were `TAV2_LOCUS6658/26197`. The *bHLH010* and *bHLH091* orthologs were `TAV2_LOCUS13948` and `TAV2_LOCUS13945/13946`, respectively. These lower expression patterns support our focus on the literature-backed mechanisms. The sparse expression of many module 3 paralogs also supports treating that module as a future-work candidate set. Overall, expression results

indicate that prioritized candidates are active in pod tissues, supporting their plausibility as candidate mechanisms and providing a basis for future experimental validation.

2.4 Discussion

We found that a custom U-Net provided the most favorable accuracy-efficiency trade-off among the benchmarked CNN, transformer, and foundation model architectures when training and testing on our open-source, hand-annotated ground truth dataset of 347 seed pod images. These results suggest that for small, specialized biological datasets like ours, a task-specific CNN with tailored inductive biases can match or modestly exceed more complex architectures while requiring less computation. However, we note that our small dataset size and tiling scheme may have limited the performance of transformer- and foundation model-based architectures, which rely on large receptive fields and pretraining priors. Future work could explore whether larger datasets and different preprocessing strategies can unlock the potential of these architectures for pennycress phenotyping.

Our pipeline produced 52 biologically plausible, tissue-resolved phenotypes at population scale (12,253 seed pods over 768 accessions). Deep learning methods provided a 30- to 45-fold speedup over manual annotation while achieving 85.58% mIoU. Extracted seed, wing, and envelope areas exhibited a log-normal distribution, wing area was positively correlated with seed area and count, and the resulting PPN separated into coherent color, geometry, and perimeter sub-networks while highlighting cross-tissue pigment and size coupling. These results indicate that our pipeline may enable new avenues for research on pennycress seed pod biology through accurate and fast high-throughput measurements.

Starting from our image-derived pod traits, our pipeline surfaced candidate genes and mechanisms that recover known *Brassicaceae* pod and seed biology. GWAS yielded significant associations for four pod traits, which we mapped to 69 pennycress

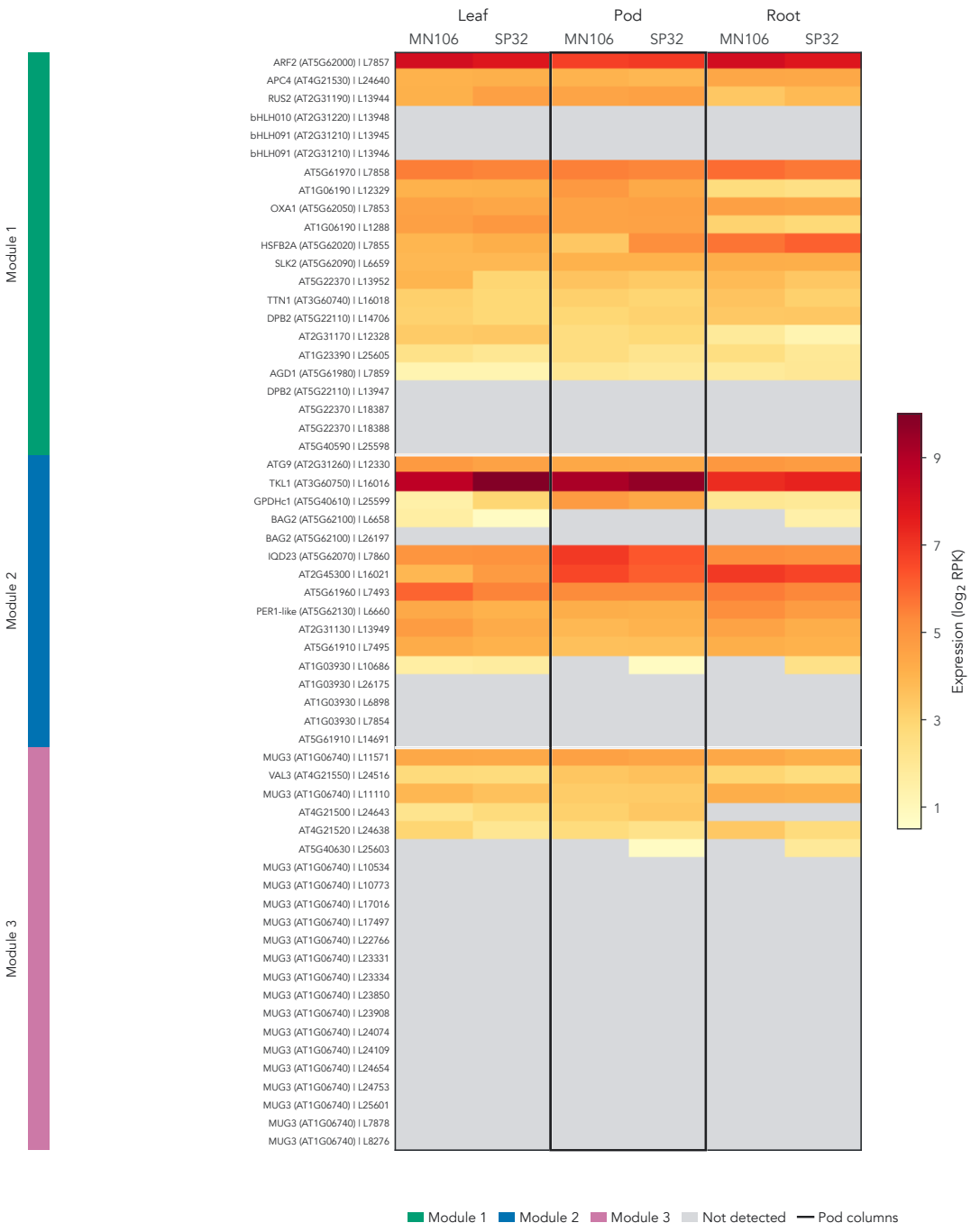

Figure 2.12: Expression of MENTOR-derived candidate loci across pennycress tissues. Rows are pennycress loci grouped by MENTOR module; repeated Arabidopsis orthologs reflect paralogous pennycress loci. Cell color encodes $\log_2$-transformed expression (reads per kilobase) in leaf, pod, and root tissue for MN106 and SP32, with gray cells denoting values below the detection threshold (≤ 0.5). Pod columns are grouped and outlined because the candidate-gene plausibility assessment focuses on pod expression, and bold row labels mark literature-supported candidates emphasized in the text.

genes and 64 Arabidopsis orthologs. GRIN narrowed these to 33 network-supported candidates, which MENTOR resolved into three modules; two were related to pod and seed biology. Module 1 was the strongest, centered on auxin-mediated organ growth, reproductive development, and cell division. Its key gene, *ARF2*, offers the clearest candidate bridge to envelope-to-seed area. Module 2 was more indirect: autophagy, nutrient remobilization, and redox balance via *ATG9*, *TKL1*, and *GPDHc1* support a plausible metabolic-supply hypothesis for pod filling and envelope-to-seed allocation. However, these candidates rest on Arabidopsis orthologs, as pennycress network and literature data remain limited.

We compared the MENTOR-IA results to pennycress gene expression data, using the expression heatmap to assess hypothesis plausibility in pod tissues. Prioritized Module 1 and 2 candidates were expressed in pods, while some lower-priority candidates were weakly expressed or not detected, supporting our focus on the literature-backed mechanisms. Furthermore, Module 3 expression is sparse, supporting our decision to treat it as a candidate set for future work. It is important to note that expression was measured across the whole pod and is correlative, so it does not establish causality or eQTL regulation. However, our expression evidence supports prioritization of our literature-backed candidates for future studies.

Despite this progress, our work has some limitations. Collecting and scanning seed pods to use for segmentation is still a labor-intensive process; this could become a bottleneck in future population-scale studies, and it only yielded a small training set. Additionally, the model's performance is dependent on the quality of label data due to its supervised nature. Accurately annotating seed border pixels was difficult due to the low brightness and contrast levels in image scans, and this limitation was reflected in the lower accuracy for seeds than other surfaces. Furthermore, benchmark compatibility with nnU-Net is imperfect due to its built-in adaptive preprocessing and evaluation pipeline.

Additionally, GWAS results are limited as they used only 189 accessions and retained SNPs with a permissive threshold, although GRIN did help filter candidates

further. Due to the limited availability of pennycress data, we relied on Arabidopsis orthologs for module derivation and interpretation, which may not fully capture pennycress biology. Finally, the MENTOR-IA interpretation is exploratory and based on literature evidence from Arabidopsis, so it must be experimentally validated in pennycress to draw strong conclusions.

We present several avenues for future work to extend this pipeline. Improving imaging quality and scanning throughput could yield higher-quality datasets, which can also be expanded through more hand annotation for ground truth. Additionally, performing GWAS on a larger panel of accessions can provide higher-powered association results. Finally, experimental validation of the candidate mechanisms, such as *ARF2*/*APC4*/*RUS2* and *ATG9*/*TKL1*/*GPDHc1*, can test their relevance to pennycress pod phenotypes and provide a deeper understanding of the underlying biology.

This work has several implications for future pennycress research. We found no prior work that extracts intensive phenotypes from pennycress seed pods due to the labor-intensive nature of this task. Therefore, this article provides a novel pipeline for extracting several biologically important phenotypes at the population scale for pennycress and linking them to genetic variants. The framework allows scientists to perform experiments on pennycress that were previously impossible; for example, researchers can examine how treatments and experimental conditions affect entire populations by measuring seed pod phenotypes across treatment groups. Furthermore, scientists can test hypotheses about phenotypic relationships on a broad scale, leading to higher-impact research. More broadly, this work shows how deep learning-derived phenotypes can be carried into genetic association and module-level interpretation to prioritize testable biological hypotheses.

Chapter 3

Deep Learning for Genotype-by-Environment Prediction of Maize Grain Yield

3.1 Introduction

Genotype-by-environment (G×E) interactions are a key mechanism underlying crop stability and performance [100, 139]. Accurately modeling these interactions has important practical implications, including increased food security and optimized farming practices [136]. Because the G×E contribution to grain yield variation is substantial, yield rankings are not stable across environments; a hybrid superior in one site or year may be inferior in another [139, 100]. A breeder who selects on cross-environment mean performance will systematically mis-rank hybrids for the environment of interest. Therefore, predicting within-environment hybrid ordering is critical for improved selection.

An increase in publicly available data has brought both renewed focus and challenge to G×E prediction. The Genomes to Fields (G2F) Initiative curated a public dataset of genotypes and environments, which spans multiple years, hybrids,

and field sites [100]. Each yield observation carries both the genotype and the environmental covariates of the plot it was grown in, so a model can represent G×E interactions directly. The 2024 G2F prediction competition built a benchmark on this dataset and standardized the task with a common dataset, a held-out target year, and a single evaluation metric [18]. This design makes independently developed methods directly comparable. The competition scores predictions by the Pearson correlation within each environment, averaged across environments, which gives relative ranks priority over absolute scale [18]. Additionally, since this dataset was collected in real-world field conditions, it contains heterogeneous data sources with varying quality and extensive missingness. By addressing these constraints, models developed on this dataset are more likely to generalize to real-world breeding scenarios.

A variety of methods have been developed for G×E prediction. Early genomic prediction approaches relied on statistical models such as genomic best linear unbiased prediction (GBLUP) [176] and Bayesian linear mixed models [108] to predict breeding values purely from genotypes. These methods were later extended to add genotype-by-environment covariance structure, either through explicit modeling of environmental covariates [28, 60] or kernel-based approaches [75]. Although these methods have proven effective for G×E prediction in some scenarios, they are limited by several inductive biases. The interaction structure must be specified a priori in the case of kernel methods, which prevents flexible modeling of complex relationships. Furthermore, these methods assume a linear relationship between inputs and outputs, which limits the depth of interactions that can be learned. In the case of G×E prediction, this is particularly limiting because the relationship between genotype and environment is often nonlinear [114].

By contrast, deep learning models are a natural extension for multimodal genomic prediction because they can learn nonlinear feature hierarchies from heterogeneous inputs [5, 113]. Prior studies have applied multilayer perceptrons (MLPs) to compute structured interaction components [112], combined genotype and weather data to

predict yield directly [159], and used sequence models, including state-space and transformer architectures, to model marker sequences [148, 36, 190].

However, when a wide range of modeling strategies were evaluated head-to-head in the first public G2F prediction competition, which used the 2022 season as its held-out target, statistical and deep learning models both produced satisfactory yield estimates with no family separating decisively from the rest [185]. This result suggests that no single model class guarantees better accuracy on this task. Multimodal architectures for genomic prediction commonly encode each modality with its own encoder and combine the resulting representations late, through concatenation, a weighted sum, or a dedicated fusion layer [113, 159]. Under this paradigm, marker–environment interactions can only be formed after each modality has been processed independently, so the interactions available to the model are limited to those that survive compression. This design leaves an open question: whether direct interactions between individual marker and environmental tokens improve G×E prediction.

We designed this study to test whether a unified marker and environment representation could outperform branch-based late fusion for G×E prediction. We trained the FullTransformer with the 2024 G2F competition data and encoded marker and environmental tokens in one transformer sequence [18, 177]. A sparse MoE block supported token-level specialization, and an affine head corrected absolute scale while it preserved within-environment rank. We reviewed 427 run records and selected completed comparisons of fusion architecture, objective choice, and affine calibration. The selected model achieved a macro environment PCC of 0.423 and MSE of 55.40; this result corresponds to sixth place in the retrospective comparison. Future work will test whether new data and feature representations can close the gap to the competition-winning PCC of 0.45.

3.2 Methods

3.2.1 Data

Data for this model were obtained through the 2024 Genomes to Fields (G2F) Maize Prediction Competition [18]. The dataset included trait, metadata, soil, weather, genotype, and environmental covariate files spanning the 2014-2024 growing seasons, with yield labels for 2014-2023 released prior to the competition, and 2024 labels made publicly available after the competition's end. The training dataset contained 173,960 trait rows across 272 environments; 164,921 of those rows had finite yield. The 2024 test dataset contained 10,057 submission-template rows across 23 environments. Raw genotype data contained 5,899 hybrid records with 2,425 single nucleotide variant (SNV) marker loci. Environmental data contained 654 numeric environmental covariates (ECs), along with 16 daily weather variables and separate metadata and soil tables.

The data were collected across many sites, years, and management regimes, so they are heterogeneous and carry extensive missingness. The competition organizers deliberately released much of the data in raw form to capture the difficulty of modeling real-world data. Extensive preprocessing, including filtering and imputation, was necessary to address inconsistencies and ensure reliable performance. We detail the steps taken to clean our marker and environmental data in Sections 3.2.1 and 3.2.1 respectively.

Marker Data

The raw numerical genotype table contained 5,899 hybrid records and 2,425 candidate SNV loci. The raw marker files stored these states as 0, 0.5, and 1, so at each locus, we encoded alleles by dosage according to the scheme in Table 3.1 to make them compatible with tokenization. A value of 2 represents two major alleles, a value of 1

represents a heterozygous pair of alleles (major-minor), and a value of 0 represents two minor alleles.

Allele Combination	Model Token
Major-Major (AA)	2
Major-Minor (Aa)	1
Minor-Minor (aa)	0

Table 3.1: Genomic Input Marker Encodings. Each diploid marker is encoded as a single token representing the dosage of the major allele by multiplying the raw dosages (0, 0.5, 1) by two.

Marker data preprocessing consisted of filtering and imputation, which we applied to the released all-hybrid genotype table. We removed loci with more than 10% missing values. This process removed 201 loci, leaving 2,224 locus inputs to our final model. Additionally, phenotype rows without a corresponding genotype were dropped, decreasing the training set from 164,921 to 163,026 rows.

For the reported model, missing marker calls were processed sequentially in genotype-file order. For each missing call, we calculated the locus-specific median among genotype records sharing either parent with the target hybrid. Missing loci were replaced with the corresponding median value. Because records were processed sequentially, values imputed for earlier records could contribute to the donor medians used for later records. In total, this procedure filled 104,820 missing calls. After imputation, dosage values were multiplied by two and rounded to the nearest integer to produce model tokens. This conversion affected 2,516 imputed calls whose values fell outside the raw $\{0, 0.5, 1\}$ dosage grid; ordinary heterozygous calls at 0.5 converted exactly to token 1.

Environmental Data

In addition to genotypic data, we cleaned environmental data in a multi-step process. First, we focused on the output column, which measured grain yield in megagrams

per hectare (Mg/ha) and served as the prediction target. We excluded all rows which lacked either yield measurements or complete environmental data. Of the 173,960 raw trait rows, 9,039 lacked a finite yield value and were dropped. From the 163,026 rows that retained both a finite yield and a matching genotype record (Section 3.2.1), we further removed rows without seasonal weather data, yielding 161,803 records, and rows without EC coverage, yielding 143,050 records. These plot records corresponded to 238 unique environments. Because 34 environments lacked complete weather or EC blocks, they were excluded from the final cohort.

After applying these filters, we performed imputation on the weather station location data included in the competition dataset. For weather stations with partially missing data, we replaced missing observations with the station's mean latitude/longitude coordinates over all existing years. One research station, location TXH4, had no location data present. We used the Texas Tech New Deal Research farm [170] as a proxy for this weather station to retrieve the coordinates for this station, since both are located in Lubbock, Texas. We used the facility's official website to retrieve the New Deal Research Farm's latitude and longitude.

In addition to imputing weather-station locations, we compressed the daily weather data into 46 summary features. First, we indexed the 16 daily weather variables by environment and date, and dropped five variables, reducing the daily weather set to 11. We then aggregated variables starting on the planting date, and ending 14 days after harvest for historical environments and on the last date supplied in the seasonal-weather record for 2024 environments. For each retained weather variable, we calculated the minimum, maximum, mean, and accumulated sum over the aggregation window, yielding 44 summary features. We added the starting month and number of included days as additional features.

Following filtering and imputation, we integrated our location, weather, and environmental covariate datasets into a unified set of 705 variables, which consisted of 654 ECs, 46 weather summaries, two coordinates, and three categorical fields. In the best run, the three categorical fields (`Irrigated`, `Treatment`, and `Previous_Crop`)

were excluded, leaving 702 numeric environmental covariates as model inputs. Continuous variables were standardized to zero mean and unit variance, and the scaler was fit only on the training set to prevent data leakage.

We applied the same coverage requirements to the 2024 benchmark rows. Of the 10,057 submission-template rows across 23 environments, 9,486 rows across 22 environments entered the benchmark cohort. The filters removed all 385 rows of environment `SCH1_2024`, so that environment does not appear in the cohort or in any reported benchmark result. The remaining 186 excluded rows came from 13 environments that were retained.

3.2.2 Model Architecture

Overview

We developed the FullTransformer, a full-sequence transformer model that predicts maize yield as a function of genotype and environment. The model represents marker dosages and environmental covariates in one token sequence, allowing self-attention to model genotypic, environmental, and G×E structure in a shared representation space. A sparse mixture-of-experts (MoE) feed-forward sublayer [155] adds capacity through token-level expert routing, and a learned `[CLS]` summary token [33] provides the representation used for rank prediction. An environment-conditioned affine-calibration branch converts the rank prediction into the final yield estimate. The architecture is summarized in Figure 3.1.

Input Representation

For the reported model, each sample was represented by a sequence of length 2,927: one learned `[CLS]` summary token, 2,224 marker tokens, and 702 environmental tokens. Marker dosages in $\{0, 1, 2\}$ were embedded with a shared learned lookup table of vocabulary size three and width 256. The same learned `Linear(1,256)` projection was applied to every standardized environmental scalar, producing one

token per environmental feature. The [CLS], marker, and environmental tokens were concatenated in that order, and a static sinusoidal positional encoding [177] was added to the sequence. This produced a per-sample encoder input of shape 2,927 × 256.

Because yield prediction depends on the complete input rather than autoregressive next-token prediction, no causal attention mask was applied. Bidirectional attention [33] allowed every marker and environmental token to attend to every other position, enabling direct marker–environment interactions within the shared representation.

Transformer Encoder with Mixture-of-Experts

The reported FullTransformer used one encoder block with hidden width 256 and four bidirectional attention heads. The block followed the pre-layer-normalized residual organization used in GPT-2 [131], with layer normalization applied before each sublayer [6]. The normalized token representations were passed through multi-head self-attention, and dropout with rate 0.15 was applied to the attention output before it was added to the block input through a residual connection [56]. The updated token representations were normalized again and passed through the MoE sublayer, whose output received the same dropout before the second residual addition.

The reported encoder replaced the conventional dense feed-forward sublayer with a sparse MoE layer [155]. A learned linear router produced eight logits, one for each routed expert, for every sequence token, including [CLS], marker, and environmental tokens. The two routed experts with the highest logits were selected independently for each token, and a softmax over those logits produced weights used to combine their outputs. Each routed expert was a two-layer FFN with hidden width 256, GELU activation, and output width 256. An additional shared expert with the same structure processed every token, and its output was added to the weighted sum of routed-expert outputs. During training, an auxiliary load-balancing loss with weight 0.01 discouraged routing collapse by encouraging routing mass to remain distributed across the eight routed experts. This design increased conditional capacity while activating only two routed experts, in addition to the shared expert, for each token.

Prediction and Calibration Heads

After the final transformer block, layer normalization was applied to the full sequence. The normalized representation at the `[CLS]` position was passed through a `Linear(256,1)` head to produce a scalar rank prediction. This rank head was optimized directly against the environment PCC loss described in Section 3.2.3.

The selected model additionally applied an environment-conditioned affine calibration to transform the rank prediction into an absolute-yield prediction. For each sample we took the mean of its 702 projected environmental input tokens, computed before positional encoding and transformer processing. Every plot record within one environment carried the same environmental covariates, so this mean was constant within an environment; we therefore write it as $\bar{\mathbf{z}}_E$ for environment E. Two learned `Linear(256,1)` heads, f_s and f_b, mapped $\bar{\mathbf{z}}_E$ to a scalar raw-scale value and a scalar shift, respectively. The calibrated total prediction was computed as

$$\begin{aligned} s_E^{\text{raw}} &= f_s(\bar{\mathbf{z}}_E), & b_E &= f_b(\bar{\mathbf{z}}_E), \quad f_s, f_b : \mathbb{R}^{256} \to \mathbb{R}, \\ \hat{y}_{\text{total}} &= \left[\text{softplus}(s_E^{\text{raw}}) + 10^{-4}\right] \hat{y}_{\text{rank}} + b_E. \end{aligned} \tag{3.1}$$

The weights of both calibration heads and the bias of the shift head were initialized to zero. The scale-head bias was initialized to the inverse softplus of one, so the initial scale was approximately 1.0001 after adding the 10^{-4} offset, while the initial shift was zero. The softplus activation and offset constrained the scale to be strictly positive. Every hybrid in the same environment shared the same positive scale and shift, so calibration preserved both the within-environment rank ordering and Pearson correlation, leaving the macro environment PCC unchanged while allowing adjustment of absolute yield scale.

The model also exposed the mean of the encoded environmental tokens after the transformer and final normalization for use in the auxiliary environment-similarity

loss (Section 3.2.3). This pooled representation was not itself a prediction head; it shaped the learned environmental representation as described below.

3.2.3 Training

Training Objective

The training objective was built around an environment PCC loss designed to mirror the competition's macro environment PCC metric. Within each local per-GPU microbatch, we computed the Pearson correlation between observed yield and the model's rank prediction for every environment $e \in \mathcal{E}_B$ containing at least four observations. Here, $\hat{y}_i$ denotes the rank prediction for sample i. Each GPU computed correlations from its local microbatch of 256 samples, and the sufficient statistics were not pooled across GPUs. The raw environment correlations were uniformly averaged without Fisher z transformation or sample-count weighting. The environment PCC loss was defined as

$$r_e = \frac{\sum_{i \in e}(y_i - \bar{y}_e)(\hat{y}_i - \bar{\hat{y}}_e)}{\sqrt{\sum_{i \in e}(y_i - \bar{y}_e)^2}\sqrt{\sum_{i \in e}(\hat{y}_i - \bar{\hat{y}}_e)^2}}. \tag{3.2}$$

$$L_{\text{envpcc}} = 1 - \frac{1}{|\mathcal{E}_B|}\sum_{e \in \mathcal{E}_B} r_e. \tag{3.3}$$

In addition to optimizing within-environment ranking, we implemented an auxiliary environment-similarity objective to align internal environmental representations with the raw environmental data. We mean-pooled the encoded environmental tokens into an embedding $\mathbf{z}_i$ for each sample i. The cosine similarity between $\mathbf{z}_i$ and $\mathbf{z}_j$ defined the learned similarity S^z_{ij}, while the cosine similarity between their standardized raw environmental vectors defined the target similarity S^E_{ij}. We computed the loss as the mean-squared difference between these similarities over all off-diagonal pairs in a local microbatch of size B:

$$L_{\text{env-sim}} = \frac{1}{B(B-1)} \sum_{\substack{1 \le i,j \le B \\ i \ne j}} (S_{ij}^{z} - S_{ij}^{E})^2. \tag{3.4}$$

The maximum weight of this loss was 0.1. The term was inactive until epoch 50 and then ramped linearly to its maximum weight by epoch 100.

For absolute-yield calibration, the calibrated total prediction was optimized with an environment concordance correlation coefficient (CCC) loss [95] and a mean Huber loss [69]. For each environment e, the concordance correlation was

$$\begin{aligned} \rho_{c,e} &= \frac{2\ \operatorname{cov}\left(y_e, \hat{y}_e^{\text{total}}\right)}{\sigma_{y,e}^2 + \sigma_{\hat{y}^{\text{total}},e}^2 + \left(\mu_{y,e} - \mu_{\hat{y}^{\text{total}},e}\right)^2}, \\ L_{\text{envCCC}} &= 1 - \frac{1}{|\mathcal{E}_B^{(2)}|} \sum_{e \in \mathcal{E}_B^{(2)}} \rho_{c,e}, \end{aligned} \tag{3.5}$$

where $\mathcal{E}_B^{(2)}$ contained environments in the local microbatch with at least two observations and nonzero yield and prediction variance. The environment CCC loss had maximum weight 0.05. The Huber loss used mean reduction, $\delta = 1.0$, and maximum weight 0.02. Both calibration terms were inactive until epoch 150 and ramped linearly to their full weights by epoch 250. Until epoch 250, the rank prediction was detached from calibration gradients, allowing the calibration heads to learn without altering the rank head. Beginning at epoch 250, calibration gradients reached the rank prediction at 10% of their ordinary magnitude, while the environment PCC loss on the rank prediction remained fully active throughout training.

Combining all of these objectives, the total loss function was defined as

$$L(t) = L_{\text{envpcc}} + 0.1\, r_E(t) L_{\text{env-sim}} + r_C(t)\, [0.05 L_{\text{envCCC}} + 0.02 L_{\text{Huber}}] + 0.01 L_{\text{MoE}}, \tag{3.6}$$

where $r_E(t)$ increased linearly from zero to one between epochs 50 and 100, and $r_C(t)$ increased linearly from zero to one between epochs 150 and 250. The final term was the MoE load-balancing loss described above.

Training Details

The reported model contained 1,552,259 trainable parameters. We trained it on Oak Ridge National Laboratory's Frontier supercomputer using distributed data parallel training across four compute nodes, with eight GPU devices per node. Training used automatic mixed precision with the `bfloat16` data type. Each GPU processed a microbatch of 256 samples, yielding a global batch size of $4 \times 8 \times 256 = 8192$. The reported run reached early stopping after 53 minutes and 47 seconds of wall-clock time, which corresponds to approximately 29 GPU-hours.

We used the AdamW optimizer [101] with weight decay 1×10^{-5}. The learning rate increased linearly to 1×10^{-4} during an approximately five-epoch warmup and then followed cosine decay to a floor of 1×10^{-5} over a 400-epoch horizon, defined as $\min(2 \times 200, 3000)$. Training was configured for at most 3000 epochs and stopped early after 200 epochs without improvement in the validation criterion. The model used dropout 0.15, four attention heads, embedding dimension 256, one transformer block, and base model and split seeds of 1.

Because the primary objective computed within-environment correlations, we used environment-stratified minibatches with at least 32 samples per represented environment when possible. As described above, correlations were computed within each 256-sample per-GPU microbatch rather than across the global batch. For leave-environment-out validation, we randomly held out 15% of entire pre-2024 environments from model fitting. Checkpoints and early stopping were determined by the macro environment PCC of the rank prediction on this validation subset, with validation loss used to break ties. The 2024 data were excluded from model fitting and checkpoint selection and were subsequently used as a retrospective competition benchmark rather than as an untouched external validation set.

3.2.4 Evaluation

We used the 2024 evaluation data from the G2F competition [18] as our retrospective competition benchmark. The competition metric was the macro environment PCC between predicted and observed yield across the 2024 benchmark environments. We retained only rows with finite observed and predicted yields. An environment was eligible for PCC evaluation if it contained at least two observations and both its observed and predicted values were nonconstant. We computed the Pearson correlation within each eligible environment and averaged these correlations uniformly without weighting by environment size. Calibrated total predictions were used to compute the primary reported PCC; rank-head PCC was also calculated for comparison and was equal up to numerical precision because the calibration head applied a constant positive affine transformation within each environment. The evaluation procedure is summarized in Algorithm 1.

Algorithm 1 Macro Environment PCC Evaluation

for each eligible environment $e \in \mathcal{E}_{2024}^{\text{valid}}$ in the retrospective benchmark **do**
  $y_e \leftarrow$ observed yields for e
  $\hat{y}_e^{\text{total}} \leftarrow$ calibrated total predictions for e
  $PCC_e \leftarrow \text{Pearson}(y_e, \hat{y}_e^{\text{total}})$
end for
$PCC_{\text{macro}} \leftarrow \frac{1}{|\mathcal{E}_{2024}^{\text{valid}}|} \sum_{e \in \mathcal{E}_{2024}^{\text{valid}}} PCC_e$

In addition to the macro environment PCC evaluation, we implemented a macro environment MSE evaluation to assess absolute-yield calibration. We defined the macro environment MSE as

$$\begin{aligned} \text{MSE}_e &= \frac{1}{n_e} \sum_{i \in \mathcal{I}_e} \left(y_i - \hat{y}_i^{\text{total}} \right)^2, \\ \text{MSE}_{\text{macro}} &= \frac{1}{|\mathcal{E}_{2024}^{\text{MSE}}|} \sum_{e \in \mathcal{E}_{2024}^{\text{MSE}}} \text{MSE}_e, \end{aligned} \tag{3.7}$$

where $\mathcal{I}_e$ is the set of finite observed-prediction pairs in the environment e, n_e is the number of such pairs, $\mathcal{E}_{2024}^{\text{MSE}}$ is the set of environments containing at least one finite

pair, and $\hat{y}_i^{\text{total}}$ is the calibrated total prediction for sample i. In our reported results, $\text{MSE}_{\text{macro}}$ is the macro environment MSE stated alongside the macro environment PCC.

3.2.5 Model Development Comparisons

To evaluate the effectiveness of critical architectural and training choices, we examined run records from the model-development process. We highlight three comparisons: unified versus branch-based processing of markers and environmental covariates, environment PCC versus global MSE training objectives, and uncalibrated versus affine-calibrated predictions. Each comparison was performed on the same 2024 retrospective benchmark and used the same evaluation procedure. Within each comparison, we held the principal settings constant. All six runs used one transformer block, four attention heads, embedding dimension 256, a mixture-of-experts feed-forward layer with eight routed experts, top-2 routing and one shared expert, dropout 0.15, global batch size 8192, microbatch size 256, environment-stratified batching, LEO validation, and the learning rate schedule described in Section 3.2.3. The three comparisons were assembled at different stages of development, however, and both the auxiliary losses and the checkpoint-selection criterion changed between those stages. Comparisons therefore hold within a pair and not across pairs.

The first comparison evaluated the performance of a unified transformer block that processed marker and environmental tokens together (Section 3.2.2) against a branch-based late-fusion model that processed markers and environmental covariates separately before it combined their representations. The branch-based late-fusion model applied a transformer encoder and a parallel LD-oriented CNN to the marker sequence, and an MLP to the environmental covariates. The environment representation was appended to the marker token sequence as a single token, the LD representation was aligned to that sequence and added elementwise, and a layer normalization combined them. A final G×E transformer block processed the fused

sequence, and a linear head read the rank prediction from the `[CLS]` position. Figure 3.2 shows this architecture. The branch-based late-fusion model contained 2,886,404 trainable parameters. These divided into 1,452,288 for the marker encoder, 246,784 for the environment encoder, 396,288 for the LD encoder, and 791,044 for the G×E encoder. The unified FullTransformer contained about half as many parameters, so this comparison did not favor the unified model on capacity. At this stage in development, neither model used the environment-similarity auxiliary loss or the affine calibration branch. Both runs in this pair also selected checkpoints on a target-weighted validation score, which reweighted validation locations toward the 2024 site distribution, stabilized each per-location correlation with a Fisher z transformation, weighted it by sample count, and took a pessimistic bootstrap quantile across locations.

The remaining two comparisons came from a later stage of development. Both used the environment-similarity auxiliary loss at weight 0.1 and selected checkpoints on the macro environment PCC of the rank prediction. After comparing unified and branch-based architectures, we examined the effect of training objective choice. We trained two FullTransformer controls (Section 3.2.2) with identical architectures and principal hyperparameters, but optimized one model with the environment PCC loss defined in Section 3.2.3 and the other with a global MSE loss.

Finally, we compared the effect of the affine calibration head on absolute yield prediction. The reported FullTransformer used the environment PCC loss together with the affine calibration branch, while its control used the environment PCC loss alone. These two runs came from adjacent development commits, so this pair is near-matched rather than exactly matched.

Two runs of the environment PCC configuration completed at the same commit under identical settings. We report both and treat their difference as an estimate of run-to-run variation. Every other comparison arm is a single run, so this study cannot estimate variation across random initializations. The complete experiment export contained 427 run records, of which 374 finished successfully. The accompanying

machine-readable manifest records every exported run, its provenance, completion status, reported metrics, and manuscript disposition. Only completed comparisons from the legacy data scheme entered the primary Results; experiments that used the later data-generation or tokenization schemes are designated as future work.

3.3 Results

3.3.1 Performance on the 2024 Retrospective Benchmark

On the 2024 retrospective benchmark, the reported affine-calibrated FullTransformer model achieved a macro environment PCC of 0.423 and a macro environment MSE of 55.40. These metrics came from the calibrated total prediction of the checkpoint selected by pre-2024 LEO validation. The result came from one run using seed 1, so no across-seed uncertainty estimate was available.

3.3.2 Internal Model Comparisons

We evaluated architectural and training variants during model development to determine how performance varied across design choices. Section 3.2.5 defines the three comparisons and the settings that differ between them. In the architecture comparison, the unified FullTransformer control achieved a macro environment PCC of 0.411 and a macro environment MSE of 96.34, whereas the branch-based late-fusion model achieved a macro environment PCC of 0.308 and a macro environment MSE of 116.47. These results indicate that unified processing of markers and environmental covariates was associated with better within-environment correlation and lower absolute error in this comparison.

Next, we examined how matching the primary training objective to the competition's evaluation metric affected performance. Both models were evaluated with macro environment PCC and macro environment MSE, although one optimized global MSE during training. The model trained with the environment PCC loss achieved

a macro environment PCC of 0.423 and a macro environment MSE of 100.57. The model trained with the global MSE loss achieved a macro environment PCC of 0.305 and a macro environment MSE of 14.54. These results indicate that optimizing within-environment correlation substantially improved macro environment PCC but increased macro environment MSE, suggesting that the resulting predictions were less well-calibrated for absolute yield.

Motivated by this rank–scale trade-off, we compared a near-matched uncalibrated rank model with the reported affine-calibrated model. Both models achieved a macro environment PCC of 0.423, but their macro environment MSE differed: 101.06 for the uncalibrated model and 55.40 for the affine-calibrated model. Calibration improved macro environment MSE by approximately 45.2%, while macro environment PCC changed by only 0.00004. This behavior was expected because an environment-specific affine transformation with a positive scale preserves within-environment Pearson correlation while allowing the absolute scale and offset of the predictions to be adjusted.

The replicate pair described in Section 3.2.5 bounds how much of this variation is run-to-run noise. Its two runs agreed to three decimals on macro environment PCC at 0.423, and gave macro environment MSE values of 100.57 and 101.06. These differences, 0.0000039 in PCC and 0.50 in MSE, provide the study's only estimate of run-to-run variation. On that scale, the 45.66 MSE improvement from calibration is about 90 times the replicate difference. The corresponding PCC difference of 0.0000378 amounts to 0.009% of the score. Every other comparison arm is a single run. These results should therefore be interpreted as descriptive model-development comparisons rather than replicated estimates of individual-component effects. Table 3.2 reports the highlighted comparisons at full precision.

Comparison	Variant	Macro environment PCC	Macro environment MSE
Architecture	Branch-based late-fusion model	0.30819	116.46976
	Unified FullTransformer control	0.41066	96.33830
Training objective	Global MSE	0.30504	14.54242
	Environment PCC	0.42316	100.56502
Calibration	Uncalibrated rank model	0.42317	101.06046
	Affine-calibrated model	0.42321	55.40464

Table 3.2: Highlighted internal model-development comparisons on the retrospective 2024 benchmark. Comparisons should be made within, rather than across, the three labeled groups. Each row reports a single run. The architecture and objective pairs used matched principal settings; the calibration pair used near-matched runs from adjacent development commits. Values are given to five decimals so that the small differences discussed in the text remain visible. The complete run-level inventory is provided in `supplement/ablation_run_manifest.csv`.

3.3.3 Performance Across Benchmark Environments

To better understand the model's performance across environments, we examined variation in PCC among the 2024 benchmark environments (Figure 3.3). All 22 environments in the benchmark cohort met the eligibility criteria described in the Evaluation section and yielded defined PCC values. Across these environments, the model achieved a mean PCC of 0.423 and a median PCC of 0.434. The interquartile range of environment-level PCC values was 0.364–0.480, and the full range was 0.247–0.567. The model achieved positive PCC in all 22 environments, with 16 of 22 (72.7%) having PCC greater than 0.40. Thus, performance was consistently positive but varied meaningfully among benchmark environments.

We also examined the upper and lower ends of the environment-level PCC distribution. The environment with the highest PCC was `IAH4_2024`, with a PCC of 0.567. The second-highest PCC occurred in `OHH1_2024`, with a PCC of 0.532, and the third-highest occurred in `MOH1_2024`, with a PCC of 0.522. In contrast, the environment with the lowest PCC was `NYH3_2024`, with a PCC of 0.247. The second-lowest was `IAH2_2024`, with a PCC of 0.304, and the third-lowest was `GAH1_2024`, with

a PCC of 0.306. The difference between the highest and lowest environment-level PCC values was 0.319, demonstrating considerable heterogeneity in model performance across environments.

3.3.4 Competition Context

To contextualize the reported FullTransformer result, we compared it with the leading competing entries on the 2024 G2F G×E Prediction Competition EvalAI leaderboard [37]. The competition ranked submissions using the macro environment PCC, the same primary metric used in our retrospective benchmark [18]. Several leaderboard entries marked as "not competing" were omitted from this comparison. The FullTransformer achieved a PCC of 0.423, which would have ranked sixth among the competing entries shown in Table 3.3. This score was 0.003 lower than the fifth-place score and 0.027 lower than the winning score. Because the FullTransformer was evaluated after the competition and was not submitted, this placement is a retrospective comparison rather than an official result. Its proximity to the fifth-place score nevertheless indicates that the architecture was competitive with the leading competition entries.

Rank	Entry	Competition PCC
1	Naaa	0.45005
2	NJUST_KMG1	0.44402
3	PARaBra	0.43710
4	transform(Base)	0.42605
5	The Cornquerors	0.42574
(6)	FullTransformer (retrospective)	0.42321

Table 3.3: Comparison of the FullTransformer result with the five leading competing entries on the 2024 G×E Prediction Competition EvalAI leaderboard. Entries explicitly marked as not competing were omitted. The FullTransformer score is a retrospective comparison and was not an official competition submission; its rank is given in parentheses to mark the position the score would have taken.

3.4 Discussion

This study tested whether a unified representation of marker and environmental tokens could support maize G×E yield prediction more effectively than branch-based late fusion. Our results support early cross-modal interaction as a useful design choice and show that prediction rank and absolute scale require distinct treatment. The final FullTransformer combined these choices and achieved a competitive retrospective macro environment PCC of 0.423 on the 2024 competition benchmark.

Unlike a late-fusion architecture, the FullTransformer places marker and environmental tokens in one sequence before either modality enters a separate encoder. This design gives each marker token a direct attention connection to every environmental token; thus, the model can form cross-modal representations before separate encoders can discard information needed for later interactions. Prior multimodal G×E methods often use separate encoders and late fusion [113, 159], so the comparison addresses a common design choice. The higher PCC and lower MSE of the unified control were consistent with this rationale. The unified model also reached the higher score with about half the trainable parameters of the branch-based late-fusion model, so capacity does not explain the difference. However, one seed and several component differences prevent a causal attribution to early fusion alone.

Objective choice exposed a practical trade-off between within-environment selection and absolute yield prediction. Macro environment PCC emphasizes whether the model preserves relative yield variation within each environment. By contrast, macro environment MSE penalizes errors in yield units, including errors in scale and offset. These metrics therefore support different uses: PCC supports hybrid selection within a trial, whereas MSE supports yield estimates on an absolute scale. The environment-conditioned affine head recovered part of the scale gap through a positive scale and shift that preserved within-environment PCC. In the near-matched comparison, this calibration reduced macro environment MSE by 45.2%, which supports separate correlation and scale targets. However, the calibrated model's macro environment

MSE was still 3.8× higher than that of the global MSE-trained model. That model in turn reached a macro environment PCC of only 0.305, 0.118 below the calibrated model, so it would rank hybrids poorly within a trial. Calibration therefore narrowed the trade-off rather than removing it.

The mean benchmark PCC concealed substantial variation among the eligible 2024 environments. Although every environment had positive PCC, the model did not provide uniform reliability across environments. This variation could reflect differences in environmental complexity, data quality, sample composition, or other unmeasured factors, but this study did not test these explanations. These results show that model assessment requires both aggregate and environment-level metrics.

The retrospective placement below the five leading entries supports competitiveness, but it does not constitute an official competition result. Public leaderboard records do not describe each submission method, so score differences cannot identify which architecture choices caused the placement. The 2022 G2F competition also found no clear advantage for one model family [185]. This broader pattern suggests that the FullTransformer score establishes method viability, not architecture superiority.

3.4.1 Limitations and Future Work

Each highlighted comparison used one seed, so the study could not estimate variation across random initialization. Future work should evaluate multiple seeds to quantify variation across model initializations.

The architecture comparison matched principal training settings, but its two designs differed in several components. The calibration comparison also used near-matched runs from adjacent commits. These comparisons therefore provide descriptive evidence rather than isolated causal effects. Future work should use controlled single-component ablations across multiple seeds.

The study used the 2024 labels for several model comparisons, so the benchmark was not an untouched external test set. The reported scores therefore describe retrospective benchmark performance, not a prospective competition submission. After method development, we will evaluate the FullTransformer on a future G2F season or another untouched benchmark.

Leave-environment-out validation holds out whole environments from the pre-2024 years, so it measures spatial interpolation for hybrids that the model has already seen. The 2024 benchmark posed a different problem. All 22 of its environments were new to the model, and 87.6% of its hybrids were crosses absent from the training data. We therefore selected the reported checkpoint with a criterion that does not track benchmark performance. An improvement measured under leave-environment-out validation need not transfer to novel crosses in a novel year. Future work should build a validation design that reproduces the novel-cross and novel-year structure of the benchmark.

The tester composition of the benchmark was also highly asymmetric. PHP02 accounted for 5,119 of the 9,486 rows and LH287 for 3,716, which left 651 rows for all other testers. The reported scores therefore describe performance on crosses to two tester backgrounds. They provide little evidence about hybrids with other testers.

Access to compute constrained the design of the study. Training the reported model required 32 GPUs across four nodes of a shared supercomputer, and every run depended on an allocation award and on queue scheduling. This constraint, rather than the cost of any single run, is the practical reason the highlighted comparisons use single runs rather than replicated seeds. The reported result also rests on one checkpoint and its captured run configuration. We will archive that checkpoint, its configuration, the benchmark predictions, and the metric output with the manuscript.

In addition to the future work that is necessary to address the study's limitations, we outline several directions for further model improvement. First, we plan to evaluate the new reproducible data pipeline and feature tokenization options. The pipeline creates traceable and repeatable data from the raw competition files, including

fixed imputation, filters, feature order, and data splits. This structure will also support controlled tests of alternative preprocessing choices. The tokenization options will support feature-aware representations for temperature, precipitation, soil, management, and other environmental covariates.

Furthermore, we plan to add temporal detail to the weather representation. The current model uses seasonal weather summaries, which can hide the date and crop stage of a weather event. Instead, we plan to test a weather encoder that can process daily weather sequences from planting to harvest. This approach will align weather features by days after planting rather than calendar dates. Development measures, such as growing degree days, could provide a closer link to crop stage. A transformer, RNN, or LSTM will convert the daily sequence into one or more weather tokens.

The model can then use attention to combine these tokens with soil, management, and marker tokens. We will compare seasonal weather summaries, temporal weather tokens, and their combination for G×E prediction.

In addition to improving data and environmental representations, we plan to test whether external parental representations improve predictions for unseen hybrids. Currently, the model uses a shared learned dosage embedding across markers, with positional codes that identify loci. We plan to evaluate external parental information, such as parent genotypes, pedigree or relationship data, and known tester identity. Parent genotype data may overlap with hybrid marker data, but direct parental representations could expose inheritance and relationships more clearly.

We will compare hybrid markers alone with parent identity tokens, parent genotype representations, pedigree data, and combining-ability features. We will calculate any combining-ability features from the fit partition only to prevent target leakage. We will report performance on novel crosses, known crosses, and major testers.

Two further directions follow from the results reported above. The affine calibration head was trained as a low-weight auxiliary objective, so we will test whether a stronger calibration objective reduces macro environment MSE further

without harming macro environment PCC. We will also test whether environmental complexity, data quality, or sample composition explain the variation in environment-level PCC, and whether environment-specific adaptation or calibrated uncertainty estimates improve reliability across environments.

All future comparisons will use multiple paired seeds and report mean scores, standard deviations, and paired differences to measure effects beyond seed variation.

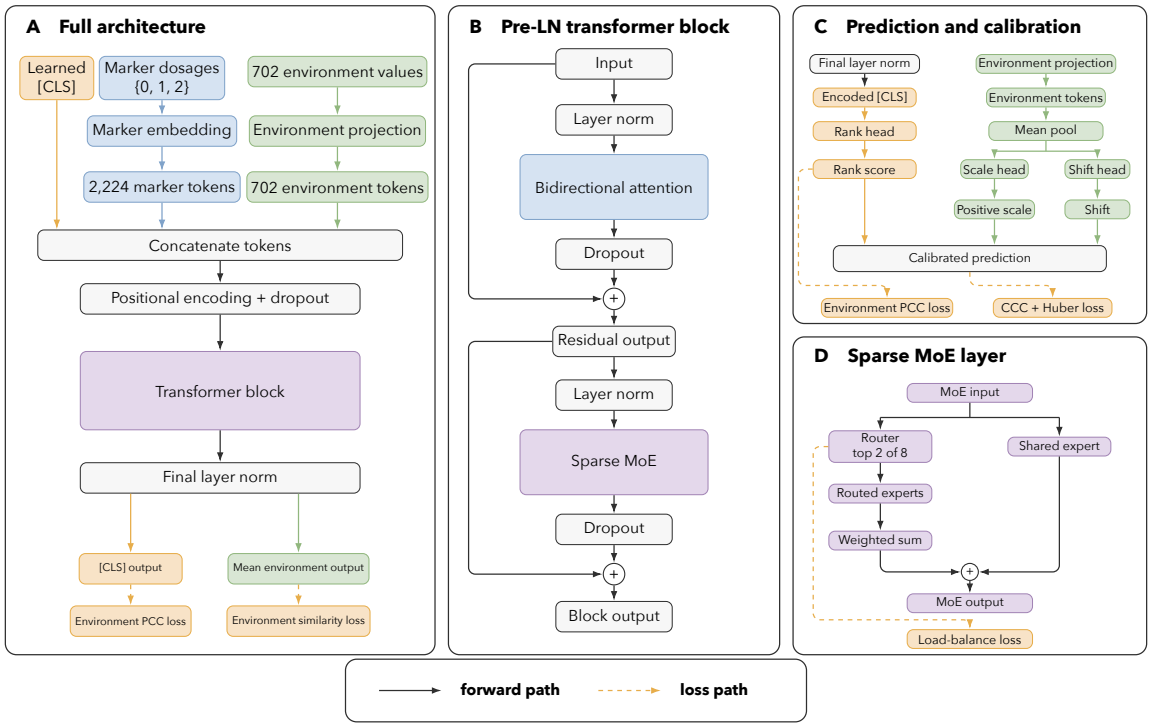

Figure 3.1: FullTransformer architecture for maize yield prediction. (A) The model embeds 2,224 marker dosages and projects 702 environmental values into one sequence with a learned `[CLS]` token. One transformer block produces the normalized `[CLS]` and environmental representations used by the primary and auxiliary losses. (B) The pre-layer-normalized block applies bidirectional attention and a sparse MoE layer with dropout and residual connections. (C) The rank head predicts relative yield, while the mean projected environmental representation supplies a positive scale and shift for affine calibration. The environment PCC loss acts on the rank score, and the CCC and Huber losses act on the calibrated prediction. (D) The MoE router selects two of eight routed experts per token and combines their weighted output with one shared expert. A load-balance loss discourages routing collapse. Blue blocks show marker and attention operations, green blocks show environment operations, purple blocks show MoE operations, and orange blocks show `[CLS]` and losses. Solid arrows show forward paths, and dashed arrows show loss paths.

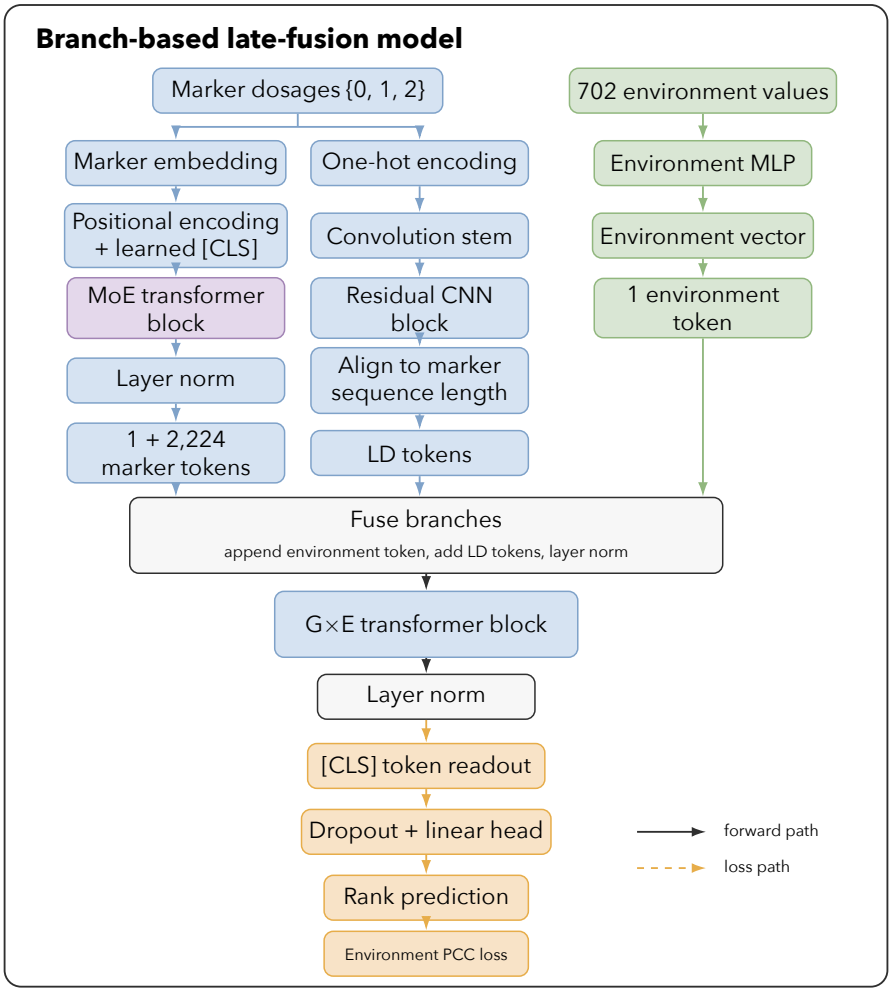

Figure 3.2: Branch-based late-fusion model used in the architecture comparison. Marker dosages enter a transformer encoder with a mixture-of-experts feed-forward layer and, in parallel, a one-hot convolutional encoder that captures local linkage-disequilibrium structure. The 702 environmental values enter a multilayer perceptron that produces a single environment token. The environment token is appended to the marker token sequence, the aligned LD representation is added elementwise, and a layer normalization combines them. One G×E transformer block then processes the fused sequence, and a linear head reads the rank prediction from the `[CLS]` position. Blue blocks show marker operations, green blocks show environment operations, purple blocks show mixture-of-experts operations, and orange blocks show `[CLS]` and loss operations.

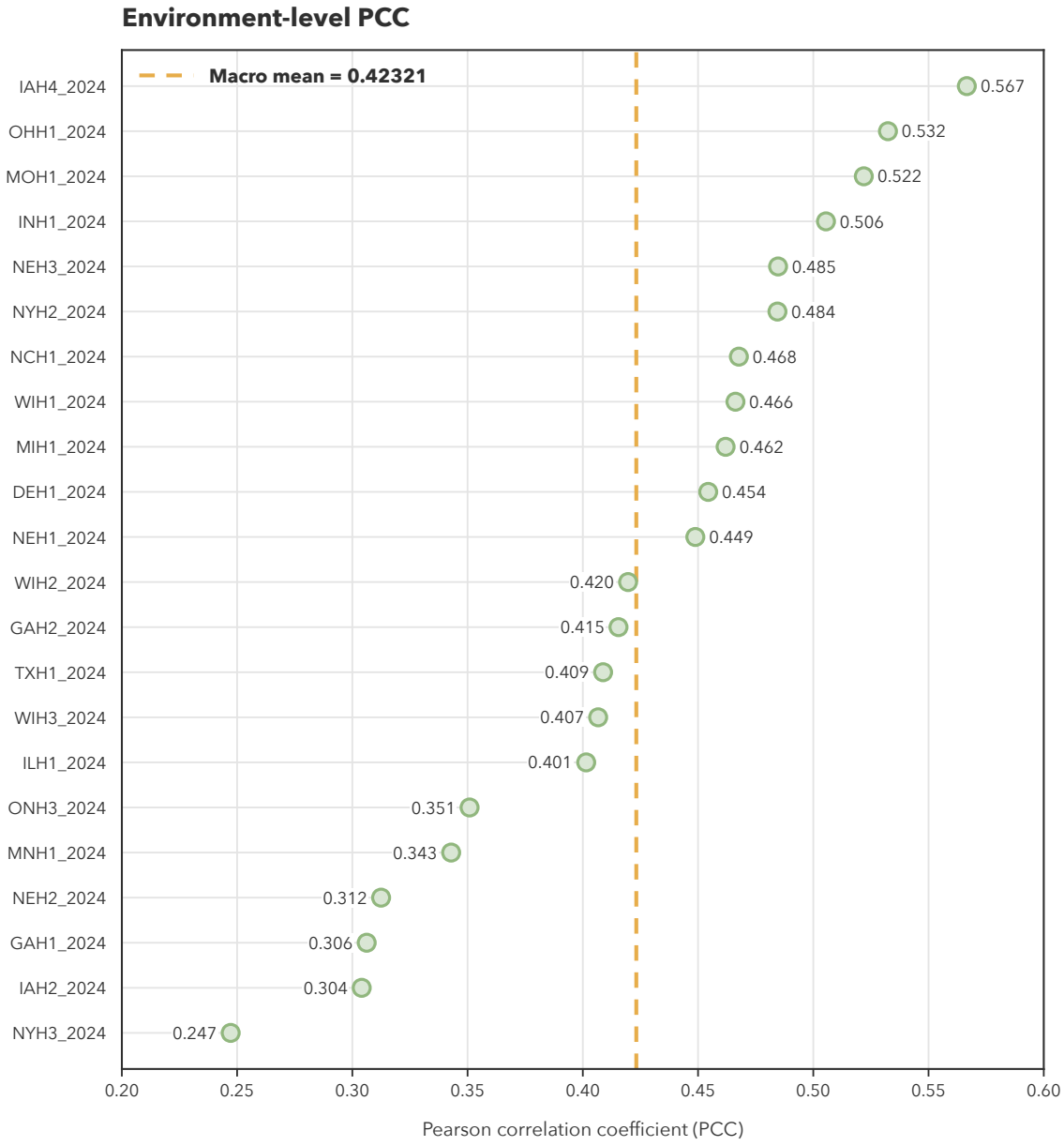

Figure 3.3: Environment-level Pearson correlation coefficients (PCCs) for the selected FullTransformer checkpoint on the 2024 retrospective benchmark. Green points show PCC for each of the 22 environments in the benchmark cohort, ordered from highest to lowest. The dashed orange line marks the macro environment PCC of 0.423. Point labels show values rounded to three decimals.

Chapter 4

MENTOR-RL

4.1 Introduction and Motivation

4.1.1 Biological Interpretation as Mechanistic Discovery

Biological interpretation involves the development and discovery of mechanistic relationships between biological entities such as genes, proteins, RNA, and other molecular components. Interpretation is essential for discovering context-specific relationships and developing models of disease, phenotypes, pathways, and regulatory systems. Recently, machine learning (ML) and artificial intelligence (AI) techniques such as iterative random forest (iRF) [10, 22] and related approaches have accelerated research through the construction of biologically informed networks, which scientists explore to generate mechanistic and functional hypotheses.

As these methods increase the resolution and scope of networks, biological interpretation has become increasingly mediated by large, multi-scale graph structures. To comprehensively examine these large biological networks and supporting databases, mechanistic exploration requires iterative querying and multi-step reasoning. Recent agentic interpretation systems such as eubiota [102] and Mentor-IA [178] demonstrate that provenance-preserving retrieval and tool-mediated synthesis can

yield high-fidelity mechanistic narratives. However, these pipelines depend on frontier model inference and are not fully optimized as a trainable policy.

4.1.2 Biological Interpretation is Bottlenecked by Human Cognition and Frontier Model Cost

Although ML/AI methods such as MENTOR [166] have refined mechanistic focus into groups of functionally related modules of genes, these modules can still be massive and intractable for individual researchers to systematically explore. Additionally, most research emphasizes a deep-but-narrow focus on a small subset of functionally related genes, which can overlook broader, yet biologically meaningful, network context, particularly in highly interactive systems such as living organisms. As biological networks grow in complexity, the combinatorial space of possible multi-hop interactions expands rapidly, while human interpretive capacity remains fixed.

While large language models (LLMs) can synthesize large volumes of information, frontier inference cost scales with exploration length and reasoning depth, limiting agentic deployment for long-horizon network traversal. This mismatch creates a scaling limit in biological mechanistic interpretation: rich relational structures can be computed, but downstream synthesis remains constrained by human bandwidth and model cost. Consequently, the speed, breadth, and contextual depth of mechanistic discovery are limited, particularly for complex traits, polygenic diseases, and multi-layer regulatory systems.

4.1.3 AI-Guided Reasoning to Extend Mechanistic Interpretation Beyond Human Scale

Open-source LLMs present an opportunity to aid scientists in mechanistic discovery tasks without being cost prohibitive. Recent developments in agent-based systems enable models to invoke deterministic tools with structured interfaces, substantially

reducing hallucination and enabling provenance-preserving exploration. We propose MENTOR-RL, a reasoning agent trained on biological ground truth and verifiable interaction objectives, enabling scalable mechanistic exploration without reliance on proprietary models.

By training a transformer-based agent with reinforcement learning across tool-use tasks and contextual levels of biological network data, we aim to produce a model that can traverse complex networks, synthesize *multi-hop* mechanistic hypotheses, and integrate both local functional structure and global topological context. Leveraging open-source model offerings allows local deployment, with training costs amortized across many exploration sessions. Rather than replacing scientific judgment, MENTOR-RL extends the interpretive bandwidth of researchers, enabling broader, faster, and more context-aware mechanistic discovery.

We make five contributions:

1. We formalize mechanistic graph exploration as a sequential decision process with a single policy operating in two modes: an actor that proposes reasoning and tool calls, and a verifier that updates a structured hypothesis and emits a continuation decision.

2. We construct a dual-source module training corpus, comprising MENTOR-derived dendrogram modules and RWR++ elbow-filtered random-walk-with-restart lines-of-evidence (RWR-LOE) modules, supporting four task cases: partial group completion, mechanism explanation, noisy group refinement, and recognition of no shared mechanism.

3. We design a deterministic, schema-grounded reward system over structured state transitions, enabling reproducible scoring without proprietary judges at the step level during preference optimization and the trajectory level for policy gradient optimization.

4. We propose a staged training pipeline, including warm-start SFT, DPO on shared-prefix trajectory branches, and GRPO for long-horizon exploration. We pair this with a curriculum that scales task difficulty, noise, and trajectory length.

5. We will release the training environment, evaluation harness, and trained checkpoints, and we will benchmark against frontier and open models under both empirical and blind expert-preference evaluation.

4.2 Background and Related Work

4.2.1 Biological Networks and Mechanistic Ground Truth

Systems Biology and Mechanistic Discovery

Systems biology examines biological function by studying interactions between molecular entities such as genes, proteins, and metabolites, rather than analyzing their individual properties [54, 8]. A central goal is identifying *modules*, or groups of functionally related entities, and characterizing the mechanisms that underlie their behavior. Because network interactions can span multiple scales and regulatory layers, modern systems biology relies on multiplex and multilayer network representations to capture context-specific relationships [7, 70]. Module-level analysis has become a foundational tool for identifying disease mechanisms, prioritizing drug targets, and interpreting polygenic phenotypes [107], motivating the need for curated, mechanistic ground truth data against which computational methods can be developed and evaluated.

Curated Interaction Maps

Large-scale systems biology depends on curated molecular interaction databases that capture experimentally supported relationships at varying levels of resolution.

Creating high-coverage, expert-annotated networks is crucial for enabling large-scale systems biology studies and encouraging mechanistic discovery. BioGRID [123] catalogs physical and genetic interactions across humans and model organisms from both low- and high-throughput studies. Reactome [109, 46] provides manually curated, reaction-level pathway models for human biology. While both resources supply rich interaction evidence, neither organizes proteins into functional groups with verified co-membership. The CORUM database [48, 171] fills this gap by curating experimentally verified mammalian protein complexes exclusively from low-throughput experiments. By providing verified group membership, curated interaction maps serve as a high confidence reference for mechanistic relationships. These resources, however, cover only a small, well-studied fraction of the interactome and are costly to extend, motivating computationally derived module structures that scale across the entire genome.

Genome-Wide Module Discovery with MENTOR

A complementary approach to curated databases is deriving mechanistic ground truth directly from network structure. MENTOR (Multiplex Embedding of Networks for Team-Based Omics Research) [166] computes random-walk-restart (RWR) embeddings for every gene across the layers of a multiplex, converts the embeddings into pairwise distances, and applies hierarchical clustering to produce a genome-wide dendrogram in which leaves are genes and internal nodes represent successively coarser functional groups. Selecting subtrees from this dendrogram yields a module whose granularity is controlled by the merge height at which the cut is taken, producing a nested family of modules that span the entire gene universe of the underlying network. Unlike curated databases, MENTOR provides module structure at genome scale and for arbitrary multiplexes, including tissue- and context-specific networks, trading literature-based verification for broad coverage and mechanistic coherence.

Gene-Centric Module Discovery with RWR-LOE

While MENTOR produces modules through top-down hierarchical clustering of the full gene universe, a complementary bottom-up approach derives modules for each individual gene. Applying RWR-LOE [77] to a single seed gene g yields a ranked list of all other genes, ordered by propagation proximity to the seed. RWR-LOE walks the entire multiplex, treating each layer as an independent line of evidence, so that final rankings reflect the combined network structure rather than a single interaction type. Rather than imposing a fixed neighborhood size, RWR++ elbow filtering selects the cutoff at the bend in the score-versus-rank curve for each seed. Retaining the seed and all genes before this adaptive cutoff yields a gene-centric module C_g^{loe}, producing one module per gene in the network. RWR-LOE modules provide a complementary, bottom-up source of mechanistic ground truth compared to hierarchical MENTOR modules.

Network Construction and Exploration on Multiplexes

Because biological systems comprise many complementary interaction modalities, integrating them without losing information motivated the development of the multiplex network formalism [116, 31], which represents the same node set across multiple edge layers of differing types. Multiplex networks support propagation methods such as random walk with restart (RWR) [83, 27], community detection methods, and learned graph models for tasks such as node classification, link prediction, and disease gene prioritization. In this proposal, these methods primarily motivate the graph operators available to the agent, while MENTOR and RWR-LOE define the module targets used for training and evaluation.

4.2.2 Tool-Using Language Models for Scientific Reasoning

Transformer Reasoning with Tool Use

In addition to deep learning's growing utility for biological discovery, neural language models have shown promise for complex reasoning tasks, especially when grounded with tool use. [186] demonstrated that prompting a model to show its reasoning steps before answering questions led to improved multi-step task performance in a few-shot setting, and [84] extended this result to show its effectiveness in a zero-shot scenario. Furthermore, adding self-consistency mechanisms [182] and branched reasoning paths with lookahead and backtracking [195] have enhanced model reasoning capabilities.

Despite this progress in model reasoning, probabilistic decoding and training data cutoffs still leave language models prone to errors, including hallucination. In turn, equipping models with the ability to use external tools during training or inference provides a layer of deterministic verification and enables interaction with the outside world. Toolformer [147] showed that self-supervised fine-tuning of tool calls yielded better performance with lower hallucination across several benchmarks, and Gorilla [127] extended this paradigm to handle large API vocabularies with retrieval. ReAct [194] showed that combining reasoning techniques with tool-based actions improved complex reasoning performance, and PAL [43] converted reasoning into programs, showing improved performance on computational tasks. These works demonstrate the power of combined reasoning and tool use for complex, multi-step problem solving, but their applications to scientific reasoning remain underexplored.

Agentic Architectures with Verification

While reasoning and tool use techniques improve task solving capability, they lack an inherent verification mechanism, which is crucial for making factual scientific claims. Several works have explored self-verification through various methods. Self-Refine [104] and Reflexion [158] both define an iterative reflection loop without weight updates to improve answer quality. Self-Refine uses a single model to generate,

critique, and refine its own output, while Reflexion utilizes sequential refinement, storing reflections in a persistent verbal memory. Self-RAG [2] fine-tunes a language model to emit reflection tokens that govern when to retrieve passages and how to assess their relevance and support. Eubiota [102] extends the self-verification process to multi-agent systems, using a dedicated verifier with separate weights to check the groundedness and factuality of other agent outputs. These approaches establish self-verification as a core component of agentic systems, but none combine a shared-weight actor-verifier paradigm with reinforcement learning over deterministic, structured state rewards, which we develop in Section 4.3.

Agentic Gene Set Interpretation

LLM-based gene set interpretation has progressed from precomputed embeddings to fully agentic systems. GenePT [19] employs GPT-4 to create gene-level and cell-level embeddings from natural language for use in downstream interpretation tasks. GeneAgent [184] utilizes a multi-step GPT-4 pipeline with self-verification and retrieval from biological databases to provide a human-readable interpretation for experimentally derived gene lists. Mentor-IA [178] extends this paradigm by creating separate agents for mechanistic interpretation subtasks, such as literature search, enrichment analysis, network analysis, and verification, unifying these outputs into a single mechanistic summary. Eubiota [102] functions similarly to Mentor-IA, but with a focus on the human gut microbiome. The authors partially train a planning agent with GRPO while keeping proprietary model-powered subagents frozen. While these works share Mentor-RL's agentic framing for mechanistic interpretation, none implements a fully trainable, end-to-end policy over verifiable rewards, which we introduce in Section 4.3.

4.2.3 Reinforcement Learning for Long-Horizon Tool-Use

Preference Optimization

Christiano et al. introduced deep reinforcement learning from human feedback (RLHF) [21], replacing hand-specified reward functions with a value model trained on pairwise human preferences. This framework was extended by InstructGPT [124], which applied RLHF to align large language model outputs with human preferences. Direct preference optimization [132] built on this work by deriving a mathematically equivalent objective that operates directly on preference pairs, only using the policy and a reference while foregoing the reward model. Azar et al. unified RLHF and DPO under ΨPO, a general preference optimization framework, and introduced IPO to address DPO's tendency to overfit when observed preferences are deterministic or near-deterministic [4]. Hong et al. proposed ORPO, a single-stage alternative that merges preference alignment and SFT via an odds-ratio penalty, removing the need for a separate alignment stage [62]. While these methods carry varying tradeoffs, we elect to train MENTOR-RL with DPO because our deterministic runtime supports scalable preference pair generation and the resulting checkpoint serves as a preference-aligned reference policy in the GRPO stage.

Policy Gradient and Group-Relative Methods

While preference optimization aligns LLMs to pairwise judgments, policy gradient methods optimize directly against scalar rewards over rollouts, supporting online learning across a wide range of tasks. Proximal policy optimization (PPO) [150] was introduced as a simple, efficient, and scalable alternative to TRPO [151], using a clipped probability ratio term to form a pessimistic surrogate of performance, preventing excessively large policy updates. In RLHF [124], this is paired with a token-wise KL penalty against a reference policy to anchor the model to its supervised initialization. Group relative policy optimization (GRPO) [154] builds on PPO, computing the advantage term by measuring the deviation of outputs from a group

mean rather than using a trained value model. Removing the critic network roughly halves the trainable parameters relative to PPO, making large-scale training more feasible. DeepSeek-R1 [53] demonstrated that GRPO with verifiable rewards can elicit emergent long-horizon reasoning behaviors at frontier scale. Having no learned critic, group-normalized advantages, and compatibility with deterministic verifiable rewards makes GRPO conducive to MENTOR-RL, where the reward is a structured, schema-grounded signal over multi-hop biological graph exploration.

RL for Tool-Augmented Agents

In addition to its usefulness for general LLM training, RL and adjacent post-training methods have been extensively used to train LLM agents in tool-use scenarios. WebGPT [117] used a fine-tuning, PPO, and rejection sampling pipeline to train a language agent for browser use and citation-backed question answering. ReST^{EM} [163] frames self-training from synthetic data as expectation maximization, demonstrating that fine-tuning on model-generated, filtered data can outperform fine-tuning on human-labeled datasets in verifiable reward settings. Toolformer [147] demonstrated that LLMs can use external tools by sampling candidate API calls during pretraining and retaining those whose outputs reduce the loss on subsequent tokens. Building on this self-supervised paradigm, ToolLLM [130] trained a language model for multi-step, multi-tool question answering on over 16,000 APIs with imitation learning. Agent-Q [129] utilizes Monte Carlo tree search and process supervision to construct model trajectories, which are used to optimize an agent via DPO. These works provide methodological precedent for MENTOR-RL; however, we utilize on-policy trajectory generation and deterministic rewards grounded in network-derived targets as a source of truth.

4.3 Proposed Methodology

Our methodology has five components. Sections 4.3.1 and 4.3.2 formalize the iterative exploration loop and the mechanistic interpretation task. Sections 4.3.3 and 4.3.4 describe dataset construction and environment design. Sections 4.3.5, 4.3.6, and 4.3.7 define the training pipeline, reward functions, and curriculum. Finally, Section 4.3.8 describes evaluation and benchmarking.

4.3.1 Agent Loop Formulation

Loop Formalization

We model mechanistic graph exploration as a sequential decision process executed by a single policy model with an internal self-verification step. The loop begins when the user provides a query, q, and optional evidence e^{user}, which may include text context, seed genes, or a multiplex network. These inputs initialize the working interpretation I_0 and the structured state S_0.

For each decision point $t < T_{\max}$, the current decision context is

$$D_t = (q, e^{\text{user}}, I_t, S_t).$$

Given D_t, the agent emits an action consisting of a textual reasoning step r_t and an optional tool action a_t. The reasoning step specifies the current subgoal or interpretation update, while the tool action specifies a runtime-executable call. If a tool action is emitted, it is executed by a deterministic runtime ε; otherwise, the observation is empty.

After the action is produced and any deterministic tool calls are executed, the same policy enters a verification mode. Conditioned on the query, user-provided evidence, the current interpretation, the current structured state, and the current reasoning/action outputs, the verifier updates the interpretation and structured state,

yielding (I_{t+1}, S_{t+1}). It also emits a continuation decision,

$$z_t \in \{\texttt{continue}, \texttt{revise}, \texttt{stop}\},$$

which determines whether the current interpretation should be extended, revised, or terminated. The decision `revise` indicates that the next iteration proceeds by correcting the interpretation and structured state rather than simply extending them.

The process repeats until either $z_t = \texttt{stop}$ or the maximum decision budget $T_{\max}$ is reached. At termination, I_T and S_T are returned as outputs. I_T provides a human-readable answer to the query with respect to the accumulated evidence, while S_T stores the corresponding verifiable outputs for reward computation and provenance preservation.

We formalize one iteration of the agent loop as

$$r_t, a_t \sim \pi_\theta^{\text{act}}(\cdot \mid D_t), \qquad o_t = \varepsilon(a_t), \qquad I_{t+1}, S_{t+1}, z_t \sim \pi_\theta^{\text{verify}}(\cdot \mid r_t, a_t, o_t, D_t),$$

where r_t is the agent's textual reasoning or subgoal, a_t is the optional tool action, and o_t is the observation returned by the deterministic runtime ε, with $\varepsilon(\varnothing) = \varnothing$ by convention. The policies π_θ^{act} and $\pi_\theta^{\text{verify}}$ denote two prompting modes of the same underlying model, sharing weights so that biological vocabulary and tool-use competence learned in one mode transfers to the other. We treat the risk that the verifier collapses as a monitoring concern during training, discussed in Section 4.5.2. At each decision point, the pair (I_t, S_t) represents the current mechanistic hypothesis, with I_t serving as a human-readable view and S_t as a machine-readable view of the same underlying state. The full agent loop algorithm is detailed in Algorithm 2.

Working Interpretation

The working interpretation I_t is the human-readable form of the current mechanistic hypothesis and contains:

Algorithm 2 The MENTOR-RL agent loop

Require: query q, user evidence e^{user}, maximum decision budget $T_{\max}$, deterministic runtime ε

Initialize working interpretation I_0 and structured state S_0 from (q, e^{user})
$T \leftarrow 0$
for $t = 0, 1, \ldots, T_{\max} - 1$ **do**
    $D_t \leftarrow (q, e^{\text{user}}, I_t, S_t)$
    $(r_t, a_t) \sim \pi_\theta^{\text{act}}(\cdot \mid D_t)$
    if $a_t \neq \varnothing$ **then**
        $o_t \leftarrow \varepsilon(a_t)$
    else
        $o_t \leftarrow \varnothing$
    end if
    $(I_{t+1}, S_{t+1}, z_t) \sim \pi_\theta^{\text{verify}}(\cdot \mid r_t, a_t, o_t, D_t)$
    $T \leftarrow t + 1$
    if $z_t =$ `stop` **then**
        break
    end if
end for
return final interpretation I_T and structured state S_T

1. the current mechanistic claim,

2. the main evidence supporting that claim,

3. any uncertainty, conflict, or alternative explanation,

4. the next question or subgoal the model is trying to resolve.

Structured State

The structured state S_t stores the same hypothesis in a machine-readable form suitable for tool execution, reward computation, and provenance tracking. It contains:

1. the user-provided anchors (q, e^{user}),

2. the current relationship status (unknown, partially observed group, validated group, multiple groups, or insufficient support) (rel_t),

3. the current predicted group or groups (G_t),

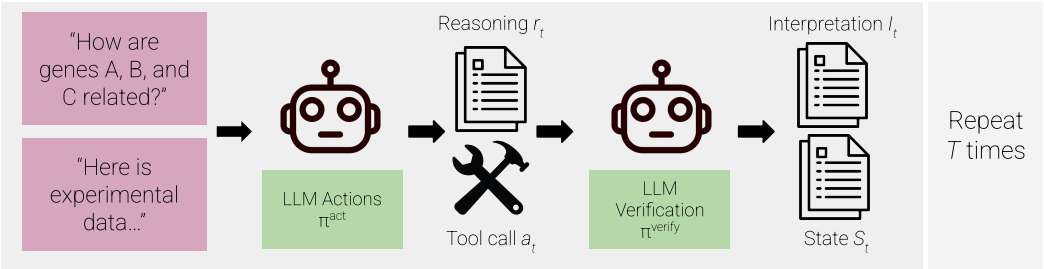

Figure 4.1: The MENTOR-RL agent loop.

4. the evidence and provenance collected so far (E_t),

5. the current predicted mechanistic labels, each tied to the evidence handles that support it,

6. the tool-use counters recording total and invalid tool calls (n_{total}, n_{inv}),

7. the remaining budget and continuation state, and, at termination, the recorded termination signal ($(T_{\max} - T), z_t$).

4.3.2 Task Outputs and Mechanistic Targets

Final Interpretation

The final outputs of the model are the terminal interpretation I_T and the terminal structured state S_T. Together, these outputs must support four cases: completion of a partially observed mechanistic group, explanation of an already coherent group, refinement of a noisy group, and recognition that no single shared mechanism is supported. The final interpretation states which case is supported by the evidence, summarizes the main evidence for that claim, and links the evidence to the claim through reasoning traces and tool calls. It also records uncertainty or abstention when evidence is incomplete. If the loop terminates with $z_t = \texttt{stop}$, the next subgoal field is left blank. If the loop terminates because the decision budget $T_{\max}$ is exhausted, the interpretation may include a recommended next subgoal to guide further exploration.

Final Structured State

The final structured state is a schema-enforced JSON object. It contains the same components defined in Section 4.3.1, but with finalized values for relationship status, predicted groups, collected evidence, and mechanistic labels. These values are used for reward evaluation and provenance tracking. The final structured state also records whether termination occurred because the model emitted $z_t = \texttt{stop}$ or because the budget $T_{\max}$ was exhausted. If the budget is exhausted, the remaining budget is 0; if the model stops early, the remaining budget is $T_{\max} - T$.

Supported Task Cases

We support four output cases during training to reflect the variability of experimental evidence that users may provide:

1. partial group completion,

2. mechanism explanation for an already related group,

3. group refinement from a noisy module,

4. no single shared mechanism.

Partial group completion occurs when the user provides a subset of related genes; the model expands this subset into a larger mechanistically related group and explains it. Mechanism explanation occurs when the user provides a complete module; the model explains the mechanism without unnecessary expansion. Group refinement occurs when the user provides a known module with several unrelated noise genes; the agent refines the group so that it contains only mechanistically related genes. Finally, in the no shared mechanism case, the user provides genes that do not support one coherent mechanism, and the model either splits them into multiple groups or returns insufficient support.

Targets for Training and Evaluation

We define two complementary target types for training and evaluation. **Module membership targets** are the hidden gene sets drawn from either MENTOR-derived dendrogram modules or elbow-filtered RWR-LOE modules (Sections 4.3.3 and 4.3.3). These targets are only used for scoring: the agent never observes hidden module members and must recover them through exploration. These modules supply the supervised signal for set-level recovery, refinement, and abstention.

Because dendrogram modules are derived computationally from multiplex structure, they offer genome-wide coverage, but often without per-entry experimental annotations. **Mechanistic evidence targets** therefore score the quality of a final explanation against the evidence retrieved during exploration, such as enrichment, annotation, and graph support. This allows interpretations to be rewarded for being grounded in evidence rather than simply matching a pre-defined annotation, which encourages the exploration of novel biology beyond existing databases.

4.3.3 Data, Resources, and Environment

Our dataset is constructed from a combination of existing internal tool infrastructure, open-source biological resources, and graph exploration environments.

Warm-Start SFT Dataset

We first construct a warm-start SFT subset from the MENTOR-IA tool stack [178], including gene metadata, gene ontology (GO) annotations, pathway membership, and literature evidence. These data are retrieved via MyGene.info [189] and Europe PMC [26]. This warm-start subset covers four task families:

1. entity normalization, such as resolving gene names and synonyms to canonical identifiers,

2. annotation retrieval, such as returning GO terms, localization, and pathway information for a gene or gene set,

3. literature-grounded extraction, such as identifying relevant genes, pathways, or mechanisms from retrieved abstracts, and

4. evidence-to-finding generation, which converts retrieved tool outputs into short, grounded mechanistic statements.

We include tool-enabled (open-book) and tool-disabled (closed-book) questions, with open-book examples emphasized to promote schema-valid tool use and evidence grounding, and closed-book examples used to strengthen biological vocabulary and lightweight recall. To make this stage reproducible, API responses and database versions will be cached during dataset construction.

To make the warm-start SFT stage concrete, we define a core tool vocabulary derived from the existing MENTOR-IA infrastructure [178] in Table 4.1. Early SFT emphasizes schema-valid biological annotation and enrichment calls, while graph-exploration and RWR++ operators are introduced during DPO and online RL.

Mentor-Derived Training Corpus

After constructing the warm-start SFT dataset, we create two complementary module corpora that together supply the hidden targets for trajectory generation: a MENTOR-derived corpus (this section) and an RWR-LOE corpus (Section 4.3.3). We first parse the genome-wide MENTOR dendrogram into modules, where each module is the gene set of a dendrogram subtree. We partition the modules into train, validation, and test splits to prevent leakage across stages.

For each module C_{module} of size n, we deterministically materialize up to four task instances, one per case in Section 4.3.2. This ensures that all four task types are represented for as many modules as possible rather than sampled independently. The `explanation` instance uses the full module as input, $C^{\text{in}} = C_{\text{module}}$. The `recovery` instance drops k_{drop} genes from the module, $C^{\text{in}} = C_{\text{module}} \setminus G_{\text{drop}}$, and the `refinement` instance adds k_{add} noise genes, $C^{\text{in}} = C_{\text{module}} \cup G_{\text{add}}$. In both the recovery and refinement cases, the hidden target remains $C = C_{\text{module}}$. The number of genes added or dropped scales with the task difficulty and module size, so that harder instances perturb a larger fraction of the module. The `none` instance constructs a pseudo-module of unrelated genes as input with hidden target $C = \varnothing$.

Noise and `none` genes are drawn from difficulty-modulated *dendrogram distance bands*, which are defined by walking up the module's ancestry in the tree, so that difficulty controls how topologically close the distractors are to the module. Additionally, `none` genes are constrained to be conflict-free (i.e. they are not related to each other), so that sampled genes do not form a coherent module themselves. All selection is deterministic given a fixed seed, making the corpus reproducible. The resulting MENTOR-based corpus provides supervision for schema-valid tool use and for the terminal outputs of Section 4.3.2.

Algorithm 3 Creation of the MENTOR-derived module training corpus

Require: dendrogram D, multiplex M, difficulty map δ

Parse the genome-wide MENTOR dendrogram D over multiplex M into modules $\{C_{\text{module}}\}$
Partition modules into train/validation/test splits at the module level
for each module C_{module} in split with difficulty map δ **do**
    $n \leftarrow |C_{\text{module}}|$
    // **explanation**
    emit ($C^{\text{in}} = C_{\text{module}},\ C = C_{\text{module}},$ `explanation`, $rel =$ `validated_group`)
    // **recovery**
    $k_{\text{drop}} \leftarrow \text{DROPCOUNT}(n, \delta_{\text{rec}})$; $G_{\text{drop}} \leftarrow \text{DETSELECT}(C_{\text{module}}, k_{\text{drop}})$
    emit ($C^{\text{in}} = C_{\text{module}} \setminus G_{\text{drop}},\ C = C_{\text{module}},$ `recovery`, $rel =$ `validated_group`)
    // **refinement**
    $k_{\text{add}} \leftarrow \text{NOISECOUNT}(n, \delta_{\text{ref}})$; $G_{\text{add}} \leftarrow \text{DENDRONEG}(C_{\text{module}}, k_{\text{add}}, \delta_{\text{ref}})$
    emit ($C^{\text{in}} = C_{\text{module}} \cup G_{\text{add}},\ C = C_{\text{module}},$ `refinement`, $rel =$ `validated_group`)
    // **none**
    $G_{\text{none}} \leftarrow \text{DENDRONEG}(C_{\text{module}}, n, \delta_{\text{none}}, \text{conflict\_free})$
    emit ($C^{\text{in}} = G_{\text{none}},\ C = \varnothing,$ `none`, $rel =$ `insufficient_support`)
end for
return all task instances

RWR-LOE Module Corpus

In addition to the top-down, hierarchical MENTOR-derived module corpus, we create a corpus of RWR-LOE exploration across all genes in the multiplex to provide a bottom-up complement. For each gene g in the same multiplex universe as the MENTOR dendrogram, we run `rwr_loe`(g) to obtain a propagation score and rank for every other gene in the network. We then apply the RWR++ elbow filter to the score-versus-rank curve for that seed, yielding an adaptive cutoff rank $r_g^\star$. The LOE module is defined as $C_g^{\text{loe}} = \{g\} \cup \{v : \text{rank}_g(v) \le r_g^\star\}$, producing a single seed-specific module per gene without imposing a fixed module size.

For each module, we materialize the same four task instances as the MENTOR-derived corpus. For difficulty control, we substitute RWR-LOE rank bands for dendrogram distance bands: noise and `none` genes are drawn from ranks just beyond, moderately beyond, or far beyond the seed's elbow cutoff. We partition RWR-LOE

Algorithm 4 Creation of the RWR-LOE module training corpus

Require: multiplex M, gene universe $\mathcal{G}$, elbow-filter parameters ϕ, difficulty map δ

Partition $\mathcal{G}$ into train/validation/test splits at the gene level
for each gene $g \in \mathcal{G}$ **do**
    Run `rwr_loe`(g, M) to obtain scores s_g and ranked list L_g
    $r_g^\star \leftarrow \textsc{ElbowCutoff}(L_g, s_g, \phi)$
    $E_g \leftarrow \{v \in L_g : \mathrm{rank}_g(v) \le r_g^\star\}$
    $C_g^{\mathrm{loe}} \leftarrow \{g\} \cup E_g, \quad n \leftarrow |C_g^{\mathrm{loe}}|$
    // **explanation**
    emit $(C^{\mathrm{in}} = C_g^{\mathrm{loe}},\ C = C_g^{\mathrm{loe}},\ \texttt{explanation},\ rel = \texttt{validated\_group})$
    // **recovery**
    $k_{\mathrm{drop}} \leftarrow \textsc{DropCount}(n, \delta_{\mathrm{rec}}); \quad G_{\mathrm{drop}} \leftarrow \textsc{DetSelect}(C_g^{\mathrm{loe}}, k_{\mathrm{drop}})$
    emit $(C^{\mathrm{in}} = C_g^{\mathrm{loe}} \setminus G_{\mathrm{drop}},\ C = C_g^{\mathrm{loe}},\ \texttt{recovery},\ rel = \texttt{validated\_group})$
    // **refinement**
    $k_{\mathrm{add}} \leftarrow \textsc{NoiseCount}(n, \delta_{\mathrm{ref}}); \quad G_{\mathrm{add}} \leftarrow \textsc{RankBandNeg}(L_g, r_g^\star, k_{\mathrm{add}}, \delta_{\mathrm{ref}})$
    emit $(C^{\mathrm{in}} = C_g^{\mathrm{loe}} \cup G_{\mathrm{add}},\ C = C_g^{\mathrm{loe}},\ \texttt{refinement},\ rel = \texttt{validated\_group})$
    // **none**
    $G_{\mathrm{none}} \leftarrow \textsc{RankBandNeg}(L_g, r_g^\star, n, \delta_{\mathrm{none}}, \mathrm{conflict\_free})$
    emit $(C^{\mathrm{in}} = G_{\mathrm{none}},\ C = \varnothing,\ \texttt{none},\ rel = \texttt{insufficient\_support})$
end for
return all task instances

modules into train, validation, and test splits independently of the MENTOR-derived modules, and module-size bins are assigned after construction from the realized values of $|C_g^{\mathrm{loe}}|$. The two corpora are complementary: MENTOR modules capture top-down, hierarchical functional organization, while RWR-LOE modules reflect bottom-up, seed-centered topology.

Graph Environment and Runtime

Each task instance is paired with a graph environment defined over the same canonical gene universe as the in-house brain multiplex, from which our module corpora are derived. The sampled input set C^{in} initializes the agent's graph anchors, while the hidden target C defines the supervisory signal.

To interface the graph environment with the training pipeline, we use a compiled native multiplex library exposed to Python through `ctypes` bindings [74]. Training,

rollout, and optimization code are implemented in Python [143], while graph storage and graph operations are handled by the native runtime for efficient execution on large multiplexes. Multiplex networks are persisted as raw, CSR-style binary stores described by JSON manifests; all graph input and output is binary, with the manifest carrying the layer names, node, and build metadata needed to load the graph.

The runtime uses a set of optimized multiplex functions developed as part of the MENTOR [166] and MENTOR-IA [178] projects known as RWR++. Core operators include neighborhood retrieval, shortest-path search, random walk with restart on both monoplex and multiplex views, and subgraph induction. These operators provide the primary graph-facing actions available to the agent during exploration.

All runtime operators return both graph objects and provenance metadata. In addition to nodes, edges, and scores, each returned object records the relevant layer identity, operator parameters, and source information needed to trace the final interpretation back to the graph evidence used to construct it. To ensure consistency across serialized multiplex files, C++ runtime objects, Python wrappers, and logged training artifacts, all nodes are represented in a single canonical identifier space and all edges are referenced using deterministic, layer-aware identifiers.

4.3.4 Trajectory Generation, Preference Mining, and Mechanistic Summary Construction

Overview

Using the module corpora and deterministic runtime defined in Sections 4.3.3, 4.3.3, and 4.3.3, we generate shared-prefix trajectories for SFT and DPO. For each sampled task, the warm-start policy proposes multiple candidate next steps, deterministic execution produces candidate observations and next states. We employ structured branch scoring functions to rank these candidates, and the resulting rankings are converted into shared-prefix preference pairs. One branch per step is retained as the continuation of the logged trajectory, from which we derive per-step findings

and a final mechanistic summary. The trajectory generation process is outlined in Algorithm 5.

Shared-prefix Trajectory Synthesis

Given a sampled task and initialized state, we sample multiple action candidates $(r_{t,i}, a_{t,i})$ given the current decision context D_t and execute valid tool calls in the deterministic graph runtime. To ensure that candidates with the same prefix explore genuinely distinct regions of the action space, we use verbalized sampling based on the task type. For example, recovery prompts bias candidates towards random-walk expansion, neighborhood expansion, subgraph validation, or a direct decision, while insufficient evidence (`none`) prompts direct the model towards disconfirming graph evidence. When a sampling round for recovery or refinement tasks fails to produce valid tool calls, we issue a tool-coverage retry that requires a valid graph operator whenever one can be formed, which prevents the branch pool from degenerating into tool-free continuations. Conditioned on each candidate action and observation, the same policy then samples multiple verification candidates, yielding a tree of candidate next states rooted at a shared prefix. The branched sampling scheme is visualized in Figure 4.2.

Initialization, Tool Validation, and Query Construction

We define $\text{InitState}(q, e^{\text{user}}, C^{\text{in}})$ as a deterministic function that constructs the initial interpretation I_0 and structured state S_0 from the query, user-provided evidence, and seed gene set. The working interpretation I_0 is populated from a fixed template: the mechanistic claim is empty, the evidence field records only the user-provided inputs, the uncertainty field is blank, and the next subgoal is set to the query q. The structured state S_0 is initialized as follows:

1. the user-provided anchors are set to (q, e^{user}),
2. the relationship status rel_0 is set to `unknown`,

Algorithm 5 Trajectory Generation for SFT and DPO

Require: $\pi_\theta^{act}, \pi_\theta^{verify}$, runtime ε, $T_{\max}, n_{act}, n_{ver}, n_{cov}$, directives $\{d_i\}$, pair margin τ, selection tolerance ϵ.

Sample module task $(C^{\text{in}}, C, C_{\text{type}})$ and evidence mode ρ.

$q, e^{\text{user}} \sim Q_{\text{templates}}(\cdot \mid C^{\text{in}}, C, C_{\text{type}}, \rho)$.

$(I_0, S_0) \leftarrow \text{InitState}(q, e^{\text{user}}, C^{\text{in}})$.

$\tau^\star \leftarrow [\,], \quad \mathcal{P} \leftarrow [\,], \quad \mathcal{F} \leftarrow [\,], \quad T \leftarrow 0$

$z_{-1} \leftarrow \texttt{continue}$

for $t = 0, 1, \ldots, T_{\max} - 1$ **do**

  if $z_{t-1} = \texttt{stop}$ **or** $\text{budget}(S_t) \le 0$ **then break**

  end if

  $D_t \leftarrow (q, e^{user}, I_t, S_t), \quad \mathcal{C}_t \leftarrow [\,], \quad \mathcal{A} \leftarrow [\,]$

  for $i = 0, \ldots, n_{act} - 1$ **do**

    $(r_{t,i}, a_{t,i}) \sim \pi_\theta^{act}(\cdot \mid D_t, d_i)$; append to $\mathcal{A}$

  end for

  if $C_{\text{type}} \in \{\texttt{recovery}, \texttt{refinement}\}$ **and** no $a \in \mathcal{A}$ is a tool call **then**

    for $i = 0, \ldots, n_{cov} - 1$ **do**

      $(r, a) \sim \pi_\theta^{act}(\cdot \mid D_t, d_i^{\text{cov}})$; append to $\mathcal{A}$

    end for

  end if

  for each $(r_{t,i}, a_{t,i}) \in \mathcal{A}$ **do**

    $o_{t,i} \leftarrow \varepsilon(a_{t,i})$ if $\text{ValidTool}(a_{t,i}, S_t)$ else $\varnothing$

    for $j = 0, \ldots, n_{ver} - 1$ **do**

      $(I'_{t,i,j}, S'_{t,i,j}, z'_{t,i,j}) \sim \pi_\theta^{verify}(\cdot \mid D_t, r_{t,i}, a_{t,i}, o_{t,i})$

      $s^{local}_{t,i,j} \leftarrow \text{LocalScore}(D_t, I'_{t,i,j}, S'_{t,i,j}, r_{t,i}, a_{t,i}, o_{t,i}, z'_{t,i,j}; C, C_{\text{type}})$

      Append $c_{t,i,j}$ to $\mathcal{C}_t$

    end for

  end for

  $\text{NormalizeScores}(\mathcal{C}_t)$

  $(i^\star, j^\star) \leftarrow \text{SelectBranch}(\mathcal{C}_t; \epsilon)$

  $\mathcal{P} \leftarrow \mathcal{P} \cup \text{MakePreferencePairs}(\mathcal{C}_t, c_{t,i^\star,j^\star}, \tau)$

  Append $(I_t, S_t, c_{t,i^\star,j^\star})$ to $\tau^\star$; $\quad f_t \leftarrow \text{RenderFinding}(\cdot)$; append f_t to $\mathcal{F}$

  $(I_{t+1}, S_{t+1}) \leftarrow (I'_{t,i^\star,j^\star}, S'_{t,i^\star,j^\star})$; $\quad T \leftarrow t + 1$

end for

$m \leftarrow \text{RenderSummary}(S_T, \mathcal{F})$

return $\tau^\star, \mathcal{F}, m, \mathcal{P}$

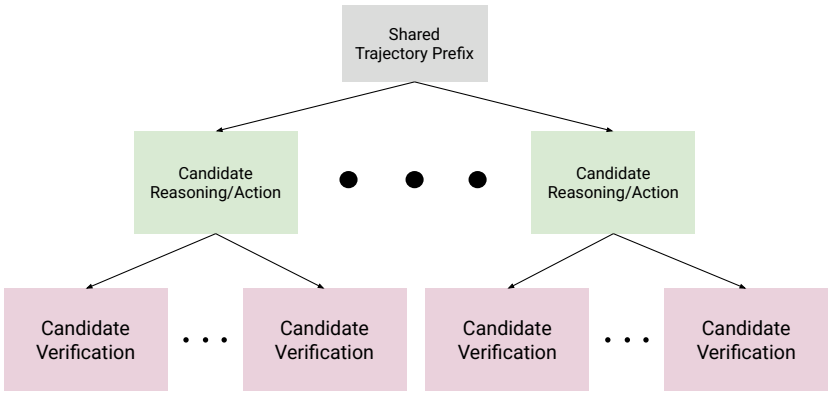

Figure 4.2: The shared prefix generation tree.

3. the predicted group is initialized to $G_0 = C^{\text{in}}$,

4. the evidence log E_0 is empty,

5. the predicted mechanistic labels are empty,

6. the remaining budget is $T_{\max}$ and the continuation state is $z_0 = \texttt{continue}$.

The agent does not observe the task type C_{type}, the true relationship status, or the hidden target C; it must infer the appropriate task case through exploration.

We define ValidTool$(a_{t,i}, S_t)$ as a deterministic function that checks whether a proposed tool action $a_{t,i}$ is executable given the current structured state S_t. Validation proceeds in two stages. First, the syntax check verifies that the tool name referenced in $a_{t,i}$ exists in the current tool vocabulary and that all arguments conform to the

tool's input schema. Second, the semantic check verifies that basic conditions are satisfied with respect to the graph environment. For example, the function ensures that queried genes exist in the multiplex, requested layers are valid, and required fields in S_t are populated. If either check fails, ValidTool returns false and the observation $o_{t,i}$ is set to $\varnothing$. Invalid tool calls are counted toward n_{inv} in the efficiency penalty (Equation 4.5).

At inference time, users provide a natural language query q alongside optional evidence e^{user}. Queries are unconstrained and may range from broad exploratory requests to specific hypothesis-driven questions. Evidence may include any combination of:

1. a seed gene list,
2. a multiplex graph or subgraph,
3. free-text context such as experimental notes or prior findings, and
4. structured annotations such as GO terms, pathway labels, or expression data from prior analysis.

During training, we simulate this variability by generating diverse queries and evidence configurations from module metadata. For each sampled task $(C^{\text{in}}, C, C_{\text{type}})$, we sample a query from task-type-specific deterministic templates and an evidence mode $\rho \in \{\texttt{minimal}, \texttt{graph}\}$ that controls which evidence types are provided to the agent. Table 4.2 defines representative query templates and compatible evidence modes for each task type. To promote phrasing diversity, we use an LLM to paraphrase each template query into natural language variants conditioned on the gene symbols of the sampled module. All generated queries and evidence configurations are cached during dataset construction for reproducibility.

Branch Ranking, Preference Mining, and Selection

Each candidate branch is assigned a deterministic local score computed from the structured state, runtime outputs, and hidden module targets. These local scores are used to rank same-prefix candidates and mine DPO preference pairs against that selection. Because several branches frequently tie or fall within a small margin of each other, continuation selection does not reduce to a strict arg max, but instead, within a tolerance margin ϵ of the best normalized score, ties are broken with task-specific heuristics such as prompted seed gene retention and tool-supported evidence, pushing the logged trajectory to favor well-grounded candidates over those that score highly by chance. We defer the formal score definitions to Section 4.3.5.

4.3.5 Preference Scoring and Reward Design

Design Principles and Scoreable Objects

We define all training-time scores over structured state transitions, deterministic runtime metadata, and hidden module-derived targets rather than textual reasoning. This design choice keeps scoring reproducible, machine-parseable, and scalable, while avoiding dependence on human judges or semantic evaluation of natural-language trajectories. We decompose all scores into four components: schema compliance, group membership, mechanistic quality, and efficiency.

Deterministic Local Branch Scoring for DPO

We define local branch scoring for DPO as

$$s_{t,i,j}^{local} = w_s\, s^{schema} + w_c\, \Delta s^{module} + w_m\, \Delta s^{mech} - w_e\, p^{eff} - p^{mismatch} \tag{4.1}$$

where Δs^{mech} is the evidence-grounded mechanistic quality delta. Additionally, mechanistic quality is clamped to be non-positive when the candidate degrades the

predicted group relative to the task type (e.g. by adding noise genes during recovery or removing true members during refinement), and $p^{mismatch}$ penalizes an `explanation` candidate that reduces recovery of an already-complete module.

We define $s_{t,i,j}^{schema}$ as a binary indicator that equals 1 if the candidate's structured state and interpretation updates are schema valid, and 0 otherwise. Specifically, schema compliance is defined by a properly-formed JSON with all required fields present and tool calls referencing valid tool names and arguments. We formalize this term as

$$s_{t,i,j}^{schema} = \mathbf{1}(\text{SchemaValid}(c_{t,i,j})). \tag{4.2}$$

We use Δs_{module} to measure the change in module membership accuracy from step t to $t+1$. Because the runtime state may contain multiple predicted groups, we first score a state by the best-matched predicted group:

$$s^{module}(z; C, C_{\text{type}}) = \max_{G \in \mathcal{G}(z)} \left[\alpha J(G, C) + \beta P(G, C) + \gamma R(G, C)\right], \tag{4.3}$$

where $\mathcal{G}(z)$ denotes the set of predicted groups in state z, with an empty group used when no prediction is present. We then define the local module improvement as

$$\Delta s_{t,i,j}^{module} = s^{module}(z_{t+1,i,j}; C, C_{\text{type}}) - s^{module}(z_t; C, C_{\text{type}}). \tag{4.4}$$

where J, P, and R denote Jaccard, precision, and recall respectively, the weights α, β, γ are set per task type. Recovery upweights recall ($\alpha, \beta, \gamma = 0.2, 0.2, 0.6$) to reward found members, refinement upweights precision $(0.2, 0.6, 0.2)$ to penalize retained noise, and explanation weights them equally ($\frac{1}{3}$ each). This best-group formulation supports the multiple-groups state representation while keeping the hidden target single-module for scoring.

We define the mechanism quality of a state as an evidence-grounded score $m_t \in [0, 1]$. From the evidence log, we extract three support signals: enrichment support e_t, annotation support g_t (from `query_mygene`), and network support u_t,

each $\in [0, 1]$. We define a consensus term $\kappa_t = \max(e_t, g_t)$, a cross-tool agreement term $\chi_t = \min(e_t, g_t)$, an evidence strength term $\sigma_t = \max(e_t,\ 0.75\, g_t,\ 0.5\, u_t)$, and an annotation-only strength term $\sigma_t^{\text{ann}} = \max(e_t,\ 0.75\, g_t)$. We additionally compute a grounding term γ_t (claim entities are supported by observed evidence), a specificity term ψ_t (vague or generic claims are penalized), a stability term b_t (enrichment consistency across query subsets), and a discounted network-agreement term $\hat{u}_t = u_t$ if $\kappa_t > 0$ and $0.35\, u_t$ otherwise. An abstention term α_t rewards withholding a claim under weak support: $\alpha_t = 1 - \sigma_t$ when the state abstains (relationship status is `insufficient_support` or an abstaining claim), 0 when an unsupported claim is asserted ($\sigma_t \leq 0.05$), and 0.5 otherwise.

The score takes one of two branches depending on whether the state abstains:

$$m_t = \begin{cases} 0.60\, \alpha_t + 0.20\, \hat{u}_t + 0.20\, \gamma_t & \text{(abstaining)} \\ 0.25\, \gamma_t + 0.25\, \kappa_t + 0.18\, \psi_t + 0.12\, \hat{u}_t + 0.10\, b_t + 0.10\, \chi_t & \text{(asserting)} \end{cases}$$

In the asserting branch, the score is multiplied by 0.35 when a claim or labels are present but unsupported ($\sigma_t \leq 0.05$). In the abstaining branch m_t is forced to 0 for any non-`none` task whose annotation evidence is absent ($\sigma_t^{\text{ann}} \leq 0.05$), preventing weak-evidence abstention from being rewarded on tasks that should produce a grounded group. The per-step mechanism reward is the change $\Delta s_{t,i,j}^{mech} = m_{t+1,i,j} - m_{t,i,j}$, set to 0 if no mechanistic claim has been made.

p_{eff} is a small penalty designed to discourage two behaviors: excessively long trajectories and invalid tool calling. Long trajectories are penalized via the term $t/T_{\max}$, while we count invalid tool calls (those that are schema-invalid, exact duplicates, or return empty observations) with n_{inv}, penalizing them proportionally to the total number of tool calls: (n_{inv}/n_{total}). We formalize this scoring term as

$$p_{t,i,j}^{eff} = \lambda_1(t/T_{\max}) + \lambda_2(n_{inv}/n_{total}). \tag{4.5}$$

For the `none` task type, where the hidden target is empty, the membership component is replaced by an abstention score

$$s^{none} = w_{rel}\,\mathbf{1}[rel = \texttt{insufficient\_support}] + w_{abs}\,\mathbf{1}[G = \varnothing],$$

with $w_{rel} = 0.6$ and $w_{abs} = 0.4$. This rewards the agent both for declaring no shared mechanism and for refusing to emit a spurious group.

Terminal Reward Decomposition for GRPO

Unlike DPO, which compares local same-prefix branches, policy gradient methods require a scalar reward for a completed rollout. We therefore define a terminal reward over the final structured state and trajectory log:

$$R_T = w_s^T s_{schema}^T + w_{abs}^T s_{module}^T + w_c^T \Delta s_{module}^T + w_{mabs}^T s_{mech}^T + w_m^T \Delta s_{mech}^T - w_e^T p_{eff}^T. \quad (4.6)$$

While all terms included in local scoring appear in the terminal reward, they are modified to capture global behavior. Additionally, two new rewards are included to capture absolute recovery. Absolute terms appear in the terminal score but not the local score. Because local scoring is evaluated on examples that share the same starting state S_t, absolute levels cancel, and only relative metrics matter. Conversely, the terminal reward is evaluating complete trajectories, which may vary in starting difficulty and length, so absolute metrics capture global problem-solving ability.

We define the absolute module membership score as the composite Jaccard, precision, and recall evaluated on the final predicted group G_T against the hidden target C:

$$s_{module}^T = \alpha J(G_T, C) + \beta P(G_T, C) + \gamma R(G_T, C), \quad (4.7)$$

using the same weighting coefficients α, β, γ defined in Equation 4.4. The cumulative module delta is the difference between the final and initial composite scores:

$$\Delta s^T_{module} = s^T_{module} - s^0_{module}, \tag{4.8}$$

where $s^0_{module} = \alpha J(G_0, C) + \beta P(G_0, C) + \gamma R(G_0, C)$ is the composite score at initialization.

The absolute mechanism score is the evidence-grounded mechanism quality m_T of the final state:

$$s^T_{mech} = m_T. \tag{4.9}$$

The cumulative mechanistic delta is the corresponding difference:

$$\Delta s^T_{mech} = m_T - m_0. \tag{4.10}$$

If no mechanistic claim has been made at termination, we set $s^T_{mech} = 0$ and $\Delta s^T_{mech} = 0$.

The terminal schema score checks that the final structured state is schema-valid and that the termination mode is properly recorded:

$$s^T_{schema} = \mathbf{1}(\text{SchemaValid}(S_T) \wedge \text{TerminationRecorded}(S_T)). \tag{4.11}$$

Finally, the terminal efficiency penalty aggregates trajectory length and invalid tool use across all steps:

$$p^T_{eff} = \lambda_1 \left(\frac{T}{T_{\max}} \right) + \lambda_2 \left(\frac{\sum_{t=0}^{T-1} n^t_{inv}}{\sum_{t=0}^{T-1} n^t_{total}} \right). \tag{4.12}$$

Preference Pair Construction, Score Normalization, and Filtering

At each decision point t, we create DPO preference tuples of the form (x_t, c_t^+, c_t^-) from our candidate set $\mathcal{C}_t = \{c_{t,i,j}\}$. We define $x_t = (q, e^{\text{user}}, I_t, S_t)$ as the shared-prefix context and $c_{t,i,j}$ as a sample interpretation continuation from Algorithm 5. Each continuation contains the model-generated reasoning step, tool action, verifier update, and continuation decision.

Before constructing pairs, we normalize scores to ensure that the selection margin has a consistent meaning across task types and trajectory lengths. We apply min-max normalization within each candidate pool $\mathcal{C}_t$, rescaling the composite local scores to $[0, 1]$. We denote the normalized score of candidate $c_{t,i,j}$ as $\bar{s}^{\text{local}}_{t,i,j}$.

To create paired examples, we set the preferred candidate c_t^+ to the branch selected as the trajectory continuation, $c_{t,i^\star,j^\star}$ (Section 4.3.4), rather than to a strict arg max of the normalized score. Because selection resolves near-ties with task-specific heuristics within a tolerance ϵ, the preferred branch is the one judged best for the task, not merely the highest-scoring candidate. We then define a selection margin τ and drop all candidates for which

$$\bar{s}^{\text{local}}_{t,i^\star,j^\star} - \bar{s}^{\text{local}}_{t,i,j} < \tau,$$

where $(i^\star, j^\star)$ indexes the selected candidate. This prevents the construction of pairs that are near ties. The remaining candidates form the eligible dispreferred pool $\mathcal{C}_t^-$.

From $\mathcal{C}_t^-$, we draw dispreferred examples across three difficulty bins, $d \in \{\texttt{easy}, \texttt{medium}, \texttt{hard}\}$. We construct `easy` pairs by selecting the lowest-scoring candidate in $\mathcal{C}_t^-$, `medium` pairs by selecting the candidate with the median score, and `hard` pairs by selecting the highest-scoring candidate in $\mathcal{C}_t^-$.

We store each preference pair (x_t, c_t^+, c_t^-) along with its difficulty bin d, task type C_{type}, decision step t, raw and normalized scores, and the score margin in the preference corpus $\mathcal{P}$. After trajectory generation is complete, we rebalance $\mathcal{P}$

across task types and difficulty bins by downsampling overrepresented categories to ensure that the training distribution is not dominated by task types that produce disproportionately many informative pairs. The curriculum strategy described in Section 4.3.7 controls which difficulty bins are included at each training stage.

Mechanistic Rendering and Training Outputs

Each selected trajectory is logged as a sequence of turn records, from which we render short per-step findings and a final mechanistic summary. These artifacts are then converted into SFT trajectory examples, evidence-to-finding records, and DPO preference pairs for the downstream training pipeline. Additionally, scored trajectories can be utilized for policy gradient optimization, such as PPO or GRPO.

4.3.6 Training Pipeline

We propose a training pipeline that closely aligns with standard model post-training pipelines. First, we utilize supervised fine-tuning for general biological knowledge and tool calling. We continue training with DPO on stepwise interpretation tree comparisons. Finally, we promote novel exploration over long-horizon tasks with policy gradient optimization.

Warm-Start SFT

We first train our policy model with warm-start SFT using the closed-book dataset outlined in Section 4.3.3. We will train our policy model on OLCF's Frontier Supercomputer [3], utilizing slurm [196] for job scheduling, huggingface and TRL [188] for weight updating, DeepSpeed ZeRO-3 for model parallelism [134], Low-Rank Adaptation for parameter-efficient fine-tuning [64], Weights and Biases [12] for tracking convergence, and gpt-oss-120b [122] as our base policy. We will utilize early stopping checkpointing to prevent model overtraining.

Tool-Call and Evidence-to-Finding SFT

We continue the supervised fine-tuning stage by adding open-book tool calling examples and evidence-to-finding examples to our data mixture. We initialize our policy from the warm-start SFT checkpoint with the lowest loss, and we utilize the same pipeline for model training.

DPO Training

After training our model via SFT, we will utilize its learned domain knowledge and tool calling capabilities to perform DPO training. We will utilize examples generated with the trajectory construction methods specified in 4.3.4 to create paired preference data. Branch continuations will be scored using the functions defined in Section 4.3.5. At each decision point, the branch selected as the trajectory continuation serves as the preferred candidate c_t^+, while dispreferred candidates c_t^- are drawn from the eligible dispreferred pool across the `easy`, `medium`, and `hard` difficulty bins. Model weights will be optimized using the DPO objective function [132]:

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right].$$

Additionally, during training, we will generate trajectories with new, trained model checkpoints to improve our model's generated preference data. This will allow us to continue making model improvements even after scoring is optimized on our initial dataset.

Online RL for Long-Horizon Exploration

After training with DPO, we will implement a final GRPO stage for long-horizon exploration. Unlike DPO, GRPO requires no annotated trajectories; rather,

optimization is done by calculating the advantage of an output relative to the rollout group $\{o_1, o_2, \ldots, o_G\}$ using the function

$$\mathcal{J}_{GRPO} = \mathbb{E}[q \sim P(Q), \{y_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(Y \mid q)]$$
$$\frac{1}{G}\sum_{i=1}^{G}\frac{1}{\mid y_i \mid}\sum_{t=1}^{|y_i|}\left\{\min\Big[\frac{\pi_\theta(y_{i,t} \mid q, y_{i,<t})}{\pi_{\theta_{old}}(y_{i,t} \mid q, y_{i,<t})}\hat{A}_{i,t}, \text{clip}\Big(\frac{\pi_\theta(y_{i,t} \mid q, y_{i,<t})}{\pi_{\theta_{old}}(y_{i,t} \mid q, y_{i,<t})}, 1-\varepsilon, 1+\varepsilon\Big)\hat{A}_{i,t}\Big] - \beta\mathbb{D}_{KL}[\pi_\theta \mid\mid \pi_{ref}]\right\},$$

where

$$\hat{A}_{i,t} = \tilde{r}_i = \frac{R_i^T - \text{mean}(R^T)}{\text{std}(R^T)}$$

By foregoing structured, annotated preference pairs, GRPO promotes novel exploration for problem solving. We aim to utilize this policy gradient method to prevent mode collapse in exploration trajectories and reinforce full coverage of the exploration space. The reference policy π_{ref} is fixed at the DPO checkpoint that initializes GRPO and is not updated during online rollouts, so the KL term anchors exploration to a stable reference rather than drifting with the trained policy.

4.3.7 Curriculum and Optimization Strategy

We will employ a training curriculum throughout the optimization process, scaling problem difficulty with model performance.

During the SFT stage, we scale difficulty by moving from closed-book biological questions to open-book biological questions that require tool use. This promotes learning biological understanding first, and then moving to structured, schema-respecting tool calling tasks second.

After training a joint closed- and open-book SFT checkpoint, we scale difficulty for DPO pairs by training on increasingly difficult examples, based on difficulty bins

defined in Section 4.3.5. We begin DPO training by limiting preference pairs to $d =$ `easy` examples, then adding `medium` and `hard` examples as training saturates for each difficulty. After DPO training converges on `hard` preference pairs, we transition to online RL to promote exploration beyond the behaviors captured in logged trajectory outputs.

We scale long-horizon reinforcement learning difficulty with several strategies. First, we modulate the trajectory length $T_{\max}$, starting with shorter exploration budgets and gradually increasing exploration length for problems that require multi-hop exploration. Additionally, we modulate realized module size, beginning training on small modules and shifting to larger ones as rewards saturate. Finally, we scale the per-task difficulty level, which jointly controls the number of dropped and added genes and the source-specific distractor band from which noise and `none` genes are drawn: dendrogram-distance bands for MENTOR-derived modules (Section 4.3.3) and post-elbow rank bands for RWR-LOE modules (Section 4.3.3). Higher difficulties remove or add more genes and draw distractors from nearer, more easily confused regions of the corresponding module source, making recovery and refinement harder. At each stage, training batches are sampled from both the MENTOR-derived and RWR-LOE module pools and are rebalanced by task type and difficulty bin to prevent either corpus from dominating training.

4.3.8 Evaluation and Benchmarks

Module Holdout Evaluation

We will use several metrics to evaluate the effectiveness of our model. Each metric will be calculated by parsing the model's terminal structured state S_T, allowing for scalable evaluation of our checkpoints. All evaluations will be conducted on a held out set of MENTOR- and RWR-LOE-derived modules from the brain multiplex.

We will report four metrics for evaluation: Jaccard index J, precision P, recall R, and F_1 score on the predicted module C^{pred} vs. the hidden module target C. Jaccard

index is defined as

$$J(C^{pred}, C) = \frac{\mid C^{pred} \cap C \mid}{\mid C^{pred} \cup C \mid},$$

penalizing both missing and extra entities. Precision is defined as

$$P = \frac{TP}{TP + FP},$$

measuring the proportion of true positive entities compared to all predicted positives. Recall is defined as

$$R = \frac{TP}{TP + FN},$$

measuring the proportion of true positive predictions to the sum of true positives and false negatives. The F1 score is defined as the harmonic mean of precision and recall,

$$F1 = \frac{2}{\frac{1}{P} + \frac{1}{R}},$$

providing a balanced view of precision and recall to measure general performance. In addition to these module recovery metrics, we will measure mechanistic grounding quality, defined in Equation 4.9, to assess whether mechanistic claims are supported by retrieved evidence.

We will report the weighted composite reward R_T, defined in Section 4.3.5, to directly compare training reward magnitude on validation and holdout splits. If the task type is `none`, meaning $C = \varnothing$, we will parse the S_T for the predicted relationship rel_T. Along with computing global accuracy, we will distinguish between both error directions, analyzing when the model claims no mechanism exists for truly related gene sets and when the model fabricates a mechanism for truly unrelated entities.

Furthermore, we will report efficiency metrics, such as mean trajectory length, budget utilization, and invalid tool call rate. This allows for comparison between holdout efficiency and validation efficiency metrics computed in Equation 4.12.

In addition to general evaluation, we will stratify our evaluation across several axes: module size, noise, task type, and module source. We will divide module size and noise into three discrete bins each for comparative evaluation. Reporting stratified metrics allows us to analyze how MENTOR-RL performs as task difficulty increases and task type changes, and comparing metrics by module source exposes whether the model generalizes across module construction methods or exhibits biases toward one of the two module types.

Novel-Domain Evaluation

Because the holdout split above is drawn from the same brain multiplex used for training, it measures in-distribution generalization. To assess out-of-distribution performance, we additionally evaluate on MENTOR-derived modules from independently published multiplex studies the model was not trained on, such as networks built for opioid use disorder and cardiac studies. These modules are produced by the same MENTOR dendrogram procedure (Section 4.3.3) over different tissue- and disease-specific multiplexes, so they share the module-recovery scoring methodology while presenting genuinely novel network context. We report the same metrics defined in Section 4.3.8 and expect a performance gap relative to the brain-multiplex holdout, reflecting distribution shift. Because these modules derive from real experimental networks with expert- and algorithm-verified structure, they let us characterize the strengths and limitations of MENTOR-RL on real-world, out-of-distribution data.

Blind Comparative Evaluation

In conjunction with empirical validation, which measures model accuracy, we will conduct blind comparative evaluations to measure expert domain scientists' model preferences. First, we will perform an inter-model preference test, supplying interpretation outputs from MENTOR-RL, GPT-5 [162], gpt-oss-120b [122], LLaMA 4 [1], Kimi K2 [169], and Nemotron 3 Super [120]. We will provide between 5 and 15

domain experts with paired samples containing MENTOR-RL outputs and outputs from another randomly selected model. We will then ask scientists to indicate their preferred interpretation, and with this information, we will calculate the proportion of MENTOR-RL outputs that are preferred compared to other models. To support interpretation of these proportions, we will overlap a subset of evaluation pairs across raters and report inter-expert rating reliability.

After conducting inter-model preference testing, we will perform intra-model preference testing to compare preferences for different model checkpoints. Using the same protocol described for inter-model comparative evaluation, we will compare the base, SFT, DPO, and GRPO-trained MENTOR-RL checkpoints. Blind comparative testing will allow us to evaluate model characteristics such as style, tone, and concision that are not easily evaluable with empirical metrics.

Efficiency and Ablations

To ensure that each component of our methodology contributes to an improvement in mechanistic interpretation, we will ablate both training stages and design elements. All ablations will be evaluated using the holdout metrics defined in Section 4.3.8.

We will ablate phases of our training pipeline to confirm that each stage improves over the previous checkpoints. First, we will test our SFT-only checkpoint compared to the SFT- and DPO-trained checkpoint to ensure that preference optimization yields interpretation gains. Next, we will compare the DPO checkpoint to the full training pipeline to determine whether long-horizon RL training leads to improvement over local preference training. Finally, we will examine the effects of training with only closed-book SFT data vs. a mixed closed- and open-book training set to evaluate whether tool-calling examples in SFT lead to better DPO optimization.

Along with testing various training stages, we will ablate architectural and reward decisions. First, we will ablate the policy's verifier mode, removing it to test whether self-verification is useful for producing better results. Additionally, we will remove the efficiency penalty p^{eff} from the reward to test whether MENTOR-RL can learn

reasonable trajectory lengths without regularization. Finally, we will ablate difficulty bins in our training curriculum, training on all difficulty bins simultaneously rather than staging them progressively. This ablation will allow us to examine whether the model can learn from hard examples without curriculum staging.

4.4 Expected Results and Deliverables

4.4.1 Expected Empirical Results

We expect MENTOR-RL to recover held-out MENTOR-derived and RWR-LOE modules with a composite F_1 competitive with frontier models at substantially lower per-query inference cost while improving monotonically across the training pipeline. Each of the SFT, DPO, and GRPO checkpoints should outperform the preceding one on module recovery and mechanistic grounding. Under the stratified evaluation defined in Section 4.3.8, we expect the largest gains on `recovery` and `refinement` tasks, where deterministic graph operators provide dense supervisory signals, and smaller but nontrivial gains on `explanation` and `none` tasks, where reward is driven by mechanistic grounding and abstention rather than set-level recovery. On the novel-domain evaluation against experimentally-derived MENTOR modules, we expect a performance gap relative to brain multiplex holdout, reflecting distributional shift, but with the gap narrowing as curriculum difficulty and noise levels increase during training. We expect comparable F_1 on held-out RWR-LOE modules relative to MENTOR-derived modules given the shared task structure, with any gap reflecting differences in module coherence between the two construction methods.

4.4.2 Expected Behavioral Results

Beyond empirical accuracy, we expect three behavioral outcomes. First, the act-verify loop should reduce schema-invalid tool calls and unsupported mechanistic claims relative to a verifier-ablated baseline, measured by invalid tool call rate

and ungrounded-claim rate on the holdout split. Second, the efficiency penalty should produce shorter average trajectories without loss of recovery accuracy relative to a penalty-ablated baseline. Third, blind comparative evaluation should show that expert scientists prefer MENTOR-RL outputs to base-policy outputs at a significantly higher rate, and preferences relative to frontier models should improve across successive training stages.

4.4.3 Deliverables

We will release, under licenses compatible with the underlying data sources, the following artifacts:

1. the module-grounded training environment, including the `ctypes` multiplex runtime, tool vocabulary, and deterministic scoring functions,
2. train, validation, and test split indices at the module level, together with the code required to reconstruct the raw training corpus and cached API responses,
3. trained MENTOR-RL checkpoints for each stage of the pipeline (warm-start SFT, joint closed- and open-book SFT, DPO, and GRPO),
4. the evaluation harness, including MENTOR module holdout benchmarks, out-of-distribution published multiplex benchmarks, and blind comparative evaluation protocols, and
5. training and generation code, curriculum configurations, and logged trajectories to support reproducible re-training and ablation.

4.4.4 Broader Impact

By releasing a deterministically-scored, open-source mechanistic exploration agent together with its training environment, we aim to lower the cost of biological interpretation research and establish a reproducible benchmark that does not require

proprietary model access. The module-grounded scoring framework is domain-specific, but the underlying pattern, deterministic structured-state rewards over a tool-using policy, is transferable to other scientific settings with curated ground truth.

4.5 Discussion and Conclusion

4.5.1 Reproducibility Without Proprietary Judges

The training pipeline for MENTOR-RL foregoes the use of proprietary judge models to ensure reproducibility and compliance with terms of use. Rewards are computed deterministically over structured state transitions and module-derived ground truth (MENTOR-derived and RWR-LOE) rather than by semantic evaluation from a frontier model. Trajectories are generated either by sampling the policy under training or by deterministic templating, not by proxy labeling through a closed API. One exception is query paraphrasing during dataset construction, for which we will use an open-source model with a pinned version so that the paraphrasing step can be reproduced without API access. Despite avoiding proprietary models during training, we will still compare MENTOR-RL to them through blind comparative evaluation, detailed in Section 4.3.8.

4.5.2 Limitations

Several limitations constrain the scope of the claims we can make with this work.

Reward-hacking risk. The composite module-membership score rewards moves toward the hidden target C. Since C is never observed directly, the agent cannot short-circuit it, but shortcut policies are still possible. For example, a policy that always returns a single-pass random-walk-with-restart expansion from the seed set will recover a substantial fraction of MENTOR modules without genuine multi-step reasoning. The efficiency penalty does not counteract this behavior. We will monitor tool-use entropy and trajectory diversity during training and, if shortcut

behavior emerges, introduce adversarially constructed modules or a diversity bonus as mitigations.

Shared-weight verifier. The actor and verifier are two prompting modes of the same policy, so DPO and RL updates on one mode also update the other. This introduces a risk of verifier collapse, in which the verifier learns to endorse the actor's proposals regardless of quality. We will monitor the verifier disagreement rate and, if collapse is observed, consider separate LoRA adapters per mode or a consistency regularizer between modes.

Task difficulty imbalance. The four task types are constructed uniformly during corpus construction, but they are not of equal difficulty; `explanation` tasks require no graph modification, while `refinement` tasks require the verifier to remove members without a tool that directly marks membership. We treat the per-task sampling weights as a hyperparameter and expect to tune them during pilot training.

Split-level leakage. Dendrogram modules share genes across different cuts of the tree; therefore, a module-level split does not by itself prevent leakage through genes shared across overlapping modules. We will report overlap statistics between train and held-out splits and, if overlap is material, adopt a gene-overlap-aware split strategy.

Expert evaluation scope. The blind comparative evaluation relies on 5 to 15 domain experts making pairwise preference judgments. This sample size supports descriptive comparison but not tight confidence intervals on preference rates, and expert rating reliability will be reported alongside preference proportions.

4.5.3 Conclusion

We have presented MENTOR-RL, a proposed open-source reasoning agent for biological mechanistic interpretation, trained through a staged pipeline of supervised fine-tuning, direct preference optimization over shared-prefix trajectory branches, and policy gradient optimization for long-horizon exploration. The training environment grounds rewards in deterministic graph operators and module targets drawn from

the MENTOR-derived and RWR-LOE corpora, removing proprietary judges from the loop and exposing every optimization signal as a reproducible, structured quantity. We view MENTOR-RL as one instance of a broader methodological pattern: domain-specialized scientific agents whose behavior is shaped by deterministic structured-state rewards over curated ground truth, rather than by semantic evaluation from frontier models. If successful, this pattern offers a route toward scientific AI systems that are cheap to deploy, auditable in their reasoning, and reproducible across research groups without centralized model access.

Table 4.1: Tool vocabulary for the MENTOR-RL agent. Related RWR++ calls are grouped by function while preserving the model-facing tool names.

Tool(s)	Purpose	Primary inputs	Stage
`query_mygene`	Gene metadata, GO terms, pathways, and summaries	`query`; requested `fields`	SFT
`enrich_gene_set`	Functional enrichment over a candidate gene set	`genes`; `background`; `top_k`; threshold	SFT, DPO, RL
`get_neighbors`; `induce_subgraph`	Local native graph and induced-subgraph evidence	gene or gene-set identifiers; layers	DPO, RL
`rwr`; `rwr_loe`; `shortest_paths`	RWR++ expansion, leave-one-out relatedness, and path evidence	seed, query, source, or target genes; layers; `top_k`	DPO, RL
`get_rank`; `get_distance`; `get_spearman`; `get_pearson`; `get_dot_similarity`	Rank, distance, and vector-similarity comparisons	paired genes; metric; layers	DPO, RL
`get_rank_vector_summary`; `get_encoding_summary`	Compact rank-vector and encoding summaries	seed or query genes; layers; `top_k`	DPO, RL
`get_gene_layers`; `get_nodes_by_layer`; `get_layer_stats`; `get_path_layer_counts`; `get_component_summary`	Layer membership, layer statistics, path support, and components	genes or layers, depending on query	DPO, RL
`get_seed_essentiality`; `get_layer_ablation`; `get_node_perturbation`	Stability, ablation, and perturbation analyses	seed or perturbation genes; layers; `top_k`	DPO, RL

Table 4.2: Representative query templates and compatible evidence modes by task type. Evidence modes: `minimal` (seed gene list) or `graph` (seed gene list + multiplex subgraph).

Task type	Example query	Evidence modes
`recovery`	"What other genes are functionally related to {seed genes}?"	minimal, graph
`refinement`	"Which of these genes do not belong together?"	minimal, graph
`explanation`	"What mechanism connects {seed genes}?"	minimal, graph
`none`	"Are these genes mechanistically related?"	minimal, graph

Bibliography

[1] Redacted by arXiv. *The Llama 4 Herd: Architecture, Training, Evaluation, and Deployment Notes*. 2026. arXiv: 2601.11659 [cs.SE]. URL: https://arxiv.org/abs/2601.11659.

[2] Akari Asai et al. "Self-rag: Learning to retrieve, generate, and critique through self-reflection". In: *The Twelfth International Conference on Learning Representations*. 2023.

[3] Scott Atchley et al. "Frontier: Exploring Exascale". In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. SC '23. Denver, CO, USA: Association for Computing Machinery, 2023. ISBN: 9798400701092. DOI: 10.1145/3581784.3607089. URL: https://doi.org/10.1145/3581784.3607089.

[4] Mohammad Gheshlaghi Azar et al. "A general theoretical paradigm to understand learning from human preferences". In: *International Conference on Artificial Intelligence and Statistics*. PMLR. 2024, pp. 4447–4455.

[5] Christina B. Azodi et al. "Benchmarking Parametric and Machine Learning Models for Genomic Prediction of Complex Traits". In: *G3: Genes, Genomes, Genetics* 9.11 (2019), pp. 3691–3702. DOI: 10.1534/g3.119.400498.

[6] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. "Layer normalization". In: *arXiv preprint arXiv:1607.06450* (2016).

[7] Albert-László Barabási, Natali Gulbahce, and Joseph Loscalzo. "Network medicine: a network-based approach to human disease". In: *Nature reviews genetics* 12.1 (2011), pp. 56–68.

[8] Albert-Laszlo Barabasi and Zoltan N Oltvai. "Network biology: understanding the cell's functional organization". In: *Nature reviews genetics* 5.2 (2004), pp. 101–113.

[9] Sumanta Basu et al. "Iterative random forests to discover predictive and stable high-order interactions". In: *Proceedings of the National Academy of Sciences* 115.8 (2018), pp. 1943–1948. DOI: 10.1073/pnas.1711236115.

[10] Sumanta Basu et al. "Iterative random forests to discover predictive and stable high-order interactions". In: *Proceedings of the National Academy of Sciences* 115.8 (2018), pp. 1943–1948.

[11] Emma J. Bennett, Jeremy A. Roberts, and Carol Wagstaff. "The role of the pod in seed development: strategies for manipulating yield". In: *New Phytologist* 190.4 (), pp. 838–853. DOI: https://doi.org/10.1111/j.1469-8137.2011.03714.x. eprint: https://nph.onlinelibrary.wiley.com/doi/pdf/10.1111/j.1469-8137.2011.03714.x. URL: https://nph.onlinelibrary.wiley.com/doi/abs/10.1111/j.1469-8137.2011.03714.x.

[12] Lukas Biewald. *Experiment Tracking with Weights and Biases*. Software available from wandb.com. 2020. URL: https://www.wandb.com/.

[13] Kevin A. Bird et al. "Structure and sequence evolution in the pennycress (*Thlaspi arvense*) pangenome". In: *New Phytologist* 250.5 (2026), pp. 2723–2741. DOI: 10.1111/nph.71111.

[14] James Bradbury et al. *JAX: composable transformations of Python+NumPy programs*. Version 0.3.13. 2018. URL: http://github.com/jax-ml/jax.

[15] G. Bradski. "The OpenCV Library". In: *Dr. Dobb's Journal of Software Tools* (2000).

[16] Nicolas Carion et al. "Sam 3: Segment anything with concepts". In: *arXiv preprint arXiv:2511.16719* (2025).

[17] Liang-Chieh Chen et al. "Encoder-decoder with atrous separable convolution for semantic image segmentation". In: *Proceedings of the European conference on computer vision (ECCV)*. 2018, pp. 801–818.

[18] Qiuyue Chen, Jacob D. Washburn, Dayane Cristina Lima, et al. "Genomes to fields 2024 maize genotype by environment prediction competition". In: *BMC Research Notes* 19.1 (2026), p. 113. DOI: 10.1186/s13104-026-07629-5.

[19] Yiqun Chen and James Zou. "Simple and effective embedding model for single-cell biology built from ChatGPT". In: *Nature biomedical engineering* 9.4 (2025), pp. 483–493.

[20] Bowen Cheng et al. "Mask2former for video instance segmentation". In: *arXiv preprint arXiv:2112.10764* (2021).

[21] Paul F Christiano et al. "Deep reinforcement learning from human preferences". In: *Advances in neural information processing systems* 30 (2017).

[22] Ashley Cliff et al. "A high-performance computing implementation of iterative random forest for the creation of predictive expression networks". In: *Genes* 10.12 (2019), p. 996.

[23] James W. Clohessy et al. "A Low-Cost Automated System for High-Throughput Phenotyping of Single Oat Seeds". In: *The Plant Phenome Journal* 1 (2018), pp. 1–13. DOI: 10.2135/tppj2018.07.0005.

[24] Rachel Combs-Giroir et al. "Morpho-physiological and transcriptomic responses of field pennycress to waterlogging". In: *Frontiers in Plant Science* 15 (2024), p. 1478507. DOI: 10.3389/fpls.2024.1478507.

[25] Rachel Combs-Giroir et al. “Morpho-physiological and transcriptomic responses of field pennycress to waterlogging”. In: *Frontiers in Plant Science* 15 (2024), p. 1478507.

[26] Europe PMC Consortium. “Europe PMC: a full-text literature database for the life sciences and platform for innovation”. In: *Nucleic acids research* 43.D1 (2015), pp. D1042–D1048.

[27] Lenore Cowen et al. “Network propagation: a universal amplifier of genetic associations”. In: *Nature Reviews Genetics* 18.9 (2017), pp. 551–562.

[28] José Crossa et al. “Genomic selection in plant breeding: methods, models, and perspectives”. In: *Trends in plant science* 22.11 (2017), pp. 961–975.

[29] Gabriela Csurka, Diane Larlus, and Florent Perronnin. “What is a good evaluation measure for semantic segmentation?” In: *Proceedings of the British Machine Vision Conference (BMVC)*. 2013. DOI: 10.5244/C.27.32.

[30] George Cybenko. “Approximation by superpositions of a sigmoidal function”. In: *Mathematics of control, signals and systems* 2.4 (1989), pp. 303–314.

[31] Manlio De Domenico et al. “Mathematical formulation of multilayer networks”. In: *Physical Review X* 3.4 (2013), p. 041022.

[32] Jia Deng et al. “Imagenet: A large-scale hierarchical image database”. In: *2009 IEEE conference on computer vision and pattern recognition*. Ieee. 2009, pp. 248–255.

[33] Jacob Devlin et al. “Bert: Pre-training of deep bidirectional transformers for language understanding”. In: *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 2019, pp. 4171–4186.

[34] Julien Di Berardino et al. “Autophagy controls resource allocation and protein storage accumulation in Arabidopsis seeds”. In: *Journal of Experimental Botany* 69.6 (2018), pp. 1403–1414. DOI: 10.1093/jxb/ery012.

[35] Christine M. Ellis et al. "AUXIN RESPONSE FACTOR1 and AUXIN RESPONSE FACTOR2 regulate senescence and floral organ abscission in Arabidopsis thaliana". In: *Development* 132.20 (2005), pp. 4563–4574. DOI: 10.1242/dev.02012.

[36] Kieran Elmes et al. "SNVformer: An Attention-based Deep Neural Network for GWAS Data". In: *bioRxiv* (2022), pp. 2022–07.

[37] EvalAI. *2024 Genomes to Fields (G2F) Genotype by Environment Prediction Competition: Leaderboard*. 2025. URL: https://eval.ai/web/challenges/challenge-page/2376/leaderboard (visited on 07/27/2026).

[38] Mark Everingham et al. "The pascal visual object classes (voc) challenge". In: *International journal of computer vision* 88 (2010), pp. 303–338.

[39] Li Fei-Fei, Rob Fergus, and Pietro Perona. "Learning generative visual models from few training examples: An incremental bayesian approach tested on 101 object categories". In: *2004 conference on computer vision and pattern recognition workshop*. IEEE. 2004, pp. 178–178.

[40] Christiane Fellbaum. *WordNet: An electronic lexical database*. MIT press, 1998.

[41] Kunihiko Fukushima. "Cognitron: A self-organizing multilayered neural network". In: *Biological cybernetics* 20.3 (1975), pp. 121–136.

[42] Kunihiko Fukushima. "Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position". In: *Biological cybernetics* 36.4 (1980), pp. 193–202.

[43] Luyu Gao et al. "Pal: Program-aided language models". In: *International conference on machine learning*. PMLR. 2023, pp. 10764–10799.

[44] Margarita Garcia-Hernandez et al. "TAIR: a resource for integrated Arabidopsis data". In: *Functional & integrative genomics* 2.6 (2002), pp. 239–253.

[45] Lei Ge et al. “Arabidopsis ROOT UVB SENSITIVE2/WEAK AUXIN RESPONSE1 is required for polar auxin transport”. In: *The Plant Cell* 22.6 (2010), pp. 1749–1761. DOI: 10.1105/tpc.110.074195.

[46] Marc Gillespie et al. “The reactome pathway knowledgebase 2022”. In: *Nucleic acids research* 50.D1 (2022), pp. D687–D692.

[47] Ross Girshick. “Fast r-cnn”. In: *Proceedings of the IEEE international conference on computer vision*. 2015, pp. 1440–1448.

[48] Madalina Giurgiu et al. “CORUM: the comprehensive resource of mammalian protein complexes—2019”. In: *Nucleic acids research* 47.D1 (2019), pp. D559–D563.

[49] Xavier Glorot and Yoshua Bengio. “Understanding the difficulty of training deep feedforward neural networks”. In: *Proceedings of the thirteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings. 2010, pp. 249–256.

[50] Xavier Glorot, Antoine Bordes, and Yoshua Bengio. “Deep sparse rectifier neural networks”. In: *Proceedings of the fourteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings. 2011, pp. 315–323.

[51] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. http://www.deeplearningbook.org. MIT Press, 2016.

[52] Anne Guiboileau et al. “Autophagy machinery controls nitrogen remobilization at the whole-plant level under both limiting and ample nitrate conditions in Arabidopsis”. In: *New Phytologist* 194.3 (2012), pp. 732–740. DOI: 10.1111/j.1469-8137.2012.04084.x.

[53] Daya Guo et al. “Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning”. In: *arXiv preprint arXiv:2501.12948* (2025).

[54] Leland H Hartwell et al. "From molecular to modular cell biology". In: *Nature* 402.Suppl 6761 (1999), pp. C47–C52.

[55] Kaiming He et al. "Deep residual learning for image recognition". In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 770–778.

[56] Kaiming He et al. "Deep residual learning for image recognition". In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 770–778.

[57] Kaiming He et al. "Delving deep into rectifiers: Surpassing human-level performance on imagenet classification". In: *Proceedings of the IEEE international conference on computer vision*. 2015, pp. 1026–1034.

[58] Kaiming He et al. "Mask r-cnn". In: *Proceedings of the IEEE international conference on computer vision*. 2017, pp. 2961–2969.

[59] Donald O Hebb. *The Organization of Behavior: A Neuropsychological Theory*. New York: John Wiley & Sons, 1949.

[60] Nicolas Heslot et al. "Integrating environmental covariates and crop modeling into the genomic selection framework to predict genotype by environment interactions". In: *Theoretical and Applied Genetics* 127.2 (2014), pp. 463–480. DOI: 10.1007/s00122-013-2231-5.

[61] Geoffrey E Hinton, Simon Osindero, and Yee-Whye Teh. "A fast learning algorithm for deep belief nets". In: *Neural computation* 18.7 (2006), pp. 1527–1554.

[62] Jiwoo Hong, Noah Lee, and James Thorne. "Orpo: Monolithic preference optimization without reference model". In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 2024, pp. 11170–11189.

[63] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. "Multilayer feedforward networks are universal approximators". In: *Neural networks* 2.5 (1989), pp. 359–366.

[64] Edward J Hu et al. “Lora: Low-rank adaptation of large language models.” In: *Iclr* 1.2 (2022), p. 3.

[65] Wei Hua et al. “Maternal control of seed oil content in *Brassica napus*: the role of silique wall photosynthesis”. In: *The Plant Journal* 69.3 (2012), pp. 432–444. DOI: 10.1111/j.1365-313X.2011.04802.x.

[66] Meng Huang et al. “BLINK: a package for the next level of genome-wide association studies with both individuals and markers in the millions”. In: *GigaScience* 8.2 (2019), giy154. DOI: 10.1093/gigascience/giy154.

[67] David H Hubel and Torsten N Wiesel. “Receptive fields of single neurones in the cat’s striate cortex”. In: *The Journal of Physiology* 148.3 (1959), pp. 574–591.

[68] David H Hubel and Torsten N Wiesel. “Receptive fields, binocular interaction and functional architecture in the cat’s visual cortex”. In: *The Journal of physiology* 160.1 (1962), pp. 106–154.

[69] Peter J. Huber. “Robust Estimation of a Location Parameter”. In: *The Annals of Mathematical Statistics* 35.1 (1964), pp. 73–101. DOI: 10.1214/aoms/1177703732.

[70] Trey Ideker and Nevan J Krogan. “Differential network biology”. In: *Molecular systems biology* 8 (2012), p. 565.

[71] Yuko Inoue et al. “AtATG genes, homologs of yeast autophagy genes, are involved in constitutive autophagy in Arabidopsis root tip cells”. In: *Plant and Cell Physiology* 47.12 (2006), pp. 1641–1652. DOI: 10.1093/pcp/pcl031.

[72] Sergey Ioffe and Christian Szegedy. “Batch normalization: Accelerating deep network training by reducing internal covariate shift”. In: *International conference on machine learning*. pmlr. 2015, pp. 448–456.

[73] Fabian Isensee et al. "nnU-Net: a self-configuring method for deep learning-based biomedical image segmentation". In: *Nature methods* 18.2 (2021), pp. 203–211.

[74] ISO. *ISO/IEC 14882:2011 Information technology — Programming languages — C++*. Geneva, Switzerland: International Organization for Standardization, Feb. 2012. URL: http://www.iso.org.

[75] Diego Jarquín et al. "A reaction norm model for genomic selection using high-dimensional genomic and environmental data". In: *Theoretical and Applied Genetics* 127.3 (2014), pp. 595–607. DOI: 10.1007/s00122-013-2243-1.

[76] Yangqing Jia et al. "Caffe: Convolutional Architecture for Fast Feature Embedding". In: *arXiv preprint arXiv:1408.5093* (2014).

[77] David Kainer et al. "RWRtoolkit: multi-omic network analysis using random walks on multiplex networks in any species". In: *GigaScience* 14 (2025), giaf028.

[78] Sangwoo Kang et al. "Autophagy-related (ATG) 11, ATG9 and the phosphatidylinositol 3-kinase control ATG2-mediated formation of autophagosomes in Arabidopsis". In: *Plant Cell Reports* 37.4 (2018), pp. 653–664. DOI: 10.1007/s00299-018-2258-9.

[79] Jens Kattge et al. "TRY–a global database of plant traits". In: *Global change biology* 17.9 (2011), pp. 2905–2935.

[80] Mahdi Khozaei et al. "Overexpression of plastid transketolase in tobacco results in a thiamine auxotrophic phenotype". In: *The Plant Cell* 27.2 (2015), pp. 432–447. DOI: 10.1105/tpc.114.131011.

[81] Diederik P Kingma. "Adam: A method for stochastic optimization". In: *arXiv preprint arXiv:1412.6980* (2014).

[82] Alexander Kirillov et al. "Segment anything". In: *Proceedings of the IEEE/CVF international conference on computer vision*. 2023, pp. 4015–4026.

[83] Sebastian Köhler et al. "Walking the interactome for prioritization of candidate disease genes". In: *The American Journal of Human Genetics* 82.4 (2008), pp. 949–958.

[84] Takeshi Kojima et al. "Large language models are zero-shot reasoners". In: *Advances in neural information processing systems* 35 (2022), pp. 22199–22213.

[85] Alex Krizhevsky. "Learning Multiple Layers of Features from Tiny Images". In: (2009), pp. 32–33. URL: https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf.

[86] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. "Imagenet classification with deep convolutional neural networks". In: *Advances in neural information processing systems* 25 (2012).

[87] John Lagergren et al. *Few-Shot Learning Enables Population-Scale Analysis of Leaf Traits in Populus trichocarpa*. 2023. arXiv: 2301.10351 [cs.CV].

[88] Louis Tung Faat Lai et al. "Subnanometer resolution cryo-EM structure of Arabidopsis thaliana ATG9". In: *Autophagy* 16.3 (2020), pp. 575–583. DOI: 10.1080/15548627.2019.1639300.

[89] Colin D. Leasure et al. "ROOT UV-B SENSITIVE2 acts with ROOT UV-B SENSITIVE1 in a root ultraviolet B-sensing pathway". In: *Plant Physiology* 150.4 (2009), pp. 1902–1915. DOI: 10.1104/pp.109.139253.

[90] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. "Deep learning". In: *nature* 521.7553 (2015), pp. 436–444.

[91] Yann LeCun, Ido Kanter, and Solla A Solla. "Efficient BackProp". In: *Neural Networks: Tricks of the Trade*. Originally published in 1991 as 'Eigenvalues of covariance matrices: Application to neural-network learning'. Springer, 2012, pp. 9–50.

[92] Yann LeCun et al. "Gradient-based learning applied to document recognition". In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.

[93] Yann LeCun et al. “Handwritten digit recognition with a back-propagation network”. In: *Advances in neural information processing systems* 2 (1989).

[94] Lei Li, Qin Zhang, and Danfeng Huang. “A review of imaging techniques for plant phenotyping”. In: *Sensors* 14.11 (2014), pp. 20078–20111.

[95] Lawrence I-Kuei Lin. “A Concordance Correlation Coefficient to Evaluate Reproducibility”. In: *Biometrics* 45.1 (1989), pp. 255–268. DOI: 10.2307/2532051.

[96] Tsung-Yi Lin et al. “Microsoft coco: Common objects in context”. In: *European conference on computer vision*. Springer. 2014, pp. 740–755.

[97] Seppo Linnainmaa. “The representation of the cumulative rounding error of an algorithm as a Taylor expansion of the local rounding errors”. PhD thesis. Master’s Thesis (in Finnish), Univ. Helsinki, 1970.

[98] Xiaolei Liu et al. “Iterative usage of fixed and random effect models for powerful and efficient genome-wide association studies”. In: *PLoS Genetics* 12.2 (2016), e1005767. DOI: 10.1371/journal.pgen.1005767.

[99] Zhenning Liu et al. “ARF2–ARF4 and ARF5 are essential for female and male gametophyte development in Arabidopsis”. In: *Plant and Cell Physiology* 59.1 (2018), pp. 179–189. DOI: 10.1093/pcp/pcx174.

[100] Marco Lopez-Cruz et al. “Leveraging data from the Genomes-to-Fields Initiative to investigate genotype-by-environment interactions in maize in North America”. In: *Nature communications* 14.1 (2023), p. 6904.

[101] Ilya Loshchilov and Frank Hutter. “Decoupled weight decay regularization”. In: *arXiv preprint arXiv:1711.05101* (2017).

[102] Pan Lu et al. “Eubiota: Modular Agentic AI for Autonomous Discovery in the Gut Microbiome”. In: *bioRxiv* (2026). DOI: 10.64898/2026.02.27.708412. eprint: https://www.biorxiv.org/content/early/2026/02/28/2026.02.

27.708412.full.pdf. URL: https://www.biorxiv.org/content/early/2026/02/28/2026.02.27.708412.

[103] Mengqian Luo et al. "Arabidopsis AUTOPHAGY-RELATED2 is essential for ATG18a and ATG9 trafficking during autophagosome closure". In: *Plant Physiology* 193.1 (2023), pp. 304–321. DOI: 10.1093/plphys/kiad287.

[104] Aman Madaan et al. "Self-refine: Iterative refinement with self-feedback". In: *Advances in neural information processing systems* 36 (2023), pp. 46534–46594.

[105] Martín Abadi et al. *TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems*. Software available from tensorflow.org. 2015. URL: https://www.tensorflow.org/.

[106] Warren S McCulloch and Walter Pitts. "A logical calculus of the ideas immanent in nervous activity". In: *The bulletin of mathematical biophysics* 5.4 (1943), pp. 115–133.

[107] Jörg Menche et al. "Uncovering disease-disease relationships through the incomplete interactome". In: *Science* 347.6224 (2015), p. 1257601.

[108] Theo HE Meuwissen, Ben J Hayes, and ME1461589 Goddard. "Prediction of total genetic value using genome-wide dense marker maps". In: *genetics* 157.4 (2001), pp. 1819–1829.

[109] Marija Milacic et al. "The reactome pathway knowledgebase 2024". en. In: *Nucleic Acids Res.* 52.D1 (Jan. 2024), pp. D672–D678.

[110] Marvin Minsky and Seymour Papert. *Perceptrons: An Introduction to Computational Geometry*. Cambridge, MA, USA: MIT Press, 1969.

[111] Sharada P. Mohanty, David P. Hughes, and Marcel Salathé. "Using Deep Learning for Image-Based Plant Disease Detection". In: *Frontiers in Plant Science* 7 (2016), p. 1419. DOI: 10.3389/fpls.2016.01419.

[112] Abelardo Montesinos-López et al. “Multi-environment Genomic Prediction of Plant Traits Using Deep Learners With Dense Architecture”. In: *G3: Genes, Genomes, Genetics* 8.12 (2018), pp. 3813–3828. DOI: 10.1534/g3.118.200740.

[113] Osval A. Montesinos-López et al. “A review of multimodal deep learning methods for genomic-enabled prediction in plant breeding”. In: *Genetics* 228.4 (2024), iyae161. DOI: 10.1093/genetics/iyae161.

[114] Osval Antonio Montesinos-López et al. “A review of deep learning applications for genomic selection”. In: *BMC genomics* 22.1 (2021), p. 19. DOI: 10.1186/s12864-020-07319-x. URL: https://doi.org/10.1186/s12864-020-07319-x.

[115] Bryan R Moser et al. “Production and evaluation of biodiesel from field pennycress (Thlaspi arvense L.) oil”. In: *Energy & Fuels* 23.8 (2009), pp. 4149–4155.

[116] Peter J Mucha et al. “Community structure in time-dependent, multiscale, and multiplex networks”. In: *science* 328.5980 (2010), pp. 876–878.

[117] Reiichiro Nakano et al. “Webgpt: Browser-assisted question-answering with human feedback”. In: *arXiv preprint arXiv:2112.09332* (2021).

[118] Justin B. Nichol, Shakshi A. Dutt, and Marcus A. Samuel. “The Shock of Shatter: Understanding Silique and Silicle Dehiscence for Improving Oilseed Crops in Brassicaceae”. In: *Plant Direct* 9.4 (2025), e70058. DOI: 10.1002/pld3.70058.

[119] Adam Nunn et al. “Chromosome-level *Thlaspi arvense* genome provides new tools for translational research and for a newly domesticated cash cover crop of the cooler climates”. In: *Plant Biotechnology Journal* 20.5 (2022), pp. 944–963. DOI: 10.1111/pbi.13775.

[120] NVIDIA. *NVIDIA Nemotron 3: Efficient and Open Intelligence*. White Paper. 2025. URL: https://arxiv.org/abs/2512.20856.

[121] Bruno A Olshausen and David J Field. "Emergence of simple-cell receptive field properties by learning a sparse code for natural images". In: *Nature* 381.6583 (1996), pp. 607–609.

[122] OpenAI. *gpt-oss-120b & gpt-oss-20b Model Card*. 2025. arXiv: 2508.10925 [cs.CL]. URL: https://arxiv.org/abs/2508.10925.

[123] Rose Oughtred et al. "The BioGRID database: A comprehensive biomedical resource of curated protein, genetic, and chemical interactions". In: *Protein Science* 30.1 (2021), pp. 187–200.

[124] Long Ouyang et al. "Training language models to follow instructions with human feedback". In: *Advances in neural information processing systems* 35 (2022), pp. 27730–27744.

[125] Dario Paolo et al. "Genetic interaction of SEEDSTICK, GORDITA and AUXIN RESPONSE FACTOR 2 during seed development". In: *Genes* 12.8 (2021), p. 1189. DOI: 10.3390/genes12081189.

[126] Adam Paszke et al. "Pytorch: An imperative style, high-performance deep learning library". In: *Advances in neural information processing systems* 32 (2019).

[127] Shishir G Patil et al. "Gorilla: Large language model connected with massive apis". In: *Advances in Neural Information Processing Systems* 37 (2024), pp. 126544–126565.

[128] Xinyang Pu et al. "ClassWise-SAM-adapter: Parameter-efficient fine-tuning adapts segment anything to SAR domain for semantic segmentation". In: *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing* 18 (2025), pp. 4791–4804.

[129] Pranav Putta et al. "Agent q: Advanced reasoning and learning for autonomous ai agents". In: *arXiv preprint arXiv:2408.07199* (2024).

[130] Yujia Qin et al. "Toolllm: Facilitating large language models to master 16000+ real-world apis". In: *arXiv preprint arXiv:2307.16789* (2023).

[131] Alec Radford et al. "Language models are unsupervised multitask learners". In: *OpenAI blog* 1.8 (2019), p. 9.

[132] Rafael Rafailov et al. "Direct preference optimization: Your language model is secretly a reward model". In: *Advances in neural information processing systems* 36 (2023), pp. 53728–53741.

[133] Rajat Raina, Anand Madhavan, and Andrew Y Ng. "Large-scale deep unsupervised learning using graphics processors". In: *Proceedings of the 26th annual international conference on machine learning*. 2009, pp. 873–880.

[134] Jeff Rasley et al. "Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters". In: *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 2020, pp. 3505–3506.

[135] Nikhila Ravi et al. "Sam 2: Segment anything in images and videos". In: *arXiv preprint arXiv:2408.00714* (2024).

[136] Deepak K. Ray et al. "Yield Trends Are Insufficient to Double Global Crop Production by 2050". In: *PLoS ONE* 8.6 (2013), e66428. DOI: 10.1371/journal.pone.0066428.

[137] Shaoqing Ren et al. "Faster R-CNN: Towards real-time object detection with region proposal networks". In: *IEEE transactions on pattern analysis and machine intelligence* 39.6 (2016), pp. 1137–1149.

[138] Agostinho G. Rocha et al. "Phosphorylation of Arabidopsis transketolase at Ser428 provides a potential paradigm for the metabolic control of chloroplast carbon metabolism". In: *Biochemical Journal* 458.2 (2014), pp. 313–322. DOI: 10.1042/BJ20130631.

[139] Anna R Rogers et al. “The importance of dominance and genotype-by-environment interactions on grain yield variation in a large-scale public cooperative maize experiment”. In: *G3* 11.2 (2021), jkaa050.

[140] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. “U-Net: Convolutional Networks for Biomedical Image Segmentation”. In: *CoRR* abs/1505.04597 (2015). arXiv: 1505.04597. URL: http://arxiv.org/abs/1505.04597.

[141] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. “U-net: Convolutional networks for biomedical image segmentation”. In: *International Conference on Medical image computing and computer-assisted intervention*. Springer. 2015, pp. 234–241.

[142] Frank Rosenblatt. “The perceptron: a probabilistic model for information storage and organization in the brain.” In: *Psychological review* 65.6 (1958), p. 386.

[143] G. van Rossum. *Python tutorial*. Tech. rep. CS-R9526. Amsterdam: Centrum voor Wiskunde en Informatica (CWI), May 1995.

[144] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. “Learning representations by back-propagating errors”. In: *Nature* 323.6088 (1986), pp. 533–536. DOI: 10.1038/323533a0. URL: https://www.nature.com/articles/323533a0.

[145] Olga Russakovsky et al. “ImageNet Large Scale Visual Recognition Challenge”. In: *International Journal of Computer Vision (IJCV)* 115.3 (2015), pp. 211–252. DOI: 10.1007/s11263-015-0816-y.

[146] Shibani Santurkar et al. “How Does Batch Normalization Help Optimization?” In: *Advances in Neural Information Processing Systems*. Vol. 31. 2018.

[147] Timo Schick et al. “Toolformer: Language models can teach themselves to use tools”. In: *Advances in neural information processing systems* 36 (2023), pp. 68539–68551.

[148] Yair Schiff et al. “Caduceus: Bi-directional equivariant long-range dna sequence modeling”. In: *Proceedings of machine learning research* 235 (2024), p. 43632.

[149] Marie C. Schruff et al. “The AUXIN RESPONSE FACTOR 2 gene of Arabidopsis links auxin signalling, cell division, and the size of seeds and other organs”. In: *Development* 133.2 (2006), pp. 251–261. DOI: 10.1242/dev.02194.

[150] John Schulman et al. “Proximal policy optimization algorithms”. In: *arXiv preprint arXiv:1707.06347* (2017).

[151] John Schulman et al. “Trust region policy optimization”. In: *International conference on machine learning*. PMLR. 2015, pp. 1889–1897.

[152] John C Sedbrook, Winthrop B Phippen, and M David Marks. “New approaches to facilitate rapid domestication of a wild plant to an oilseed crop: example pennycress (Thlaspi arvense L.)” In: *Plant Science* 227 (2014), pp. 122–132.

[153] Vincent Segura et al. “An efficient multi-locus mixed-model approach for genome-wide association studies in structured populations”. In: *Nature Genetics* 44.7 (2012), pp. 825–830. DOI: 10.1038/ng.2314.

[154] Zhihong Shao et al. “Deepseekmath: Pushing the limits of mathematical reasoning in open language models”. In: *arXiv preprint arXiv:2402.03300* (2024).

[155] Noam Shazeer et al. “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer”. In: *International Conference on Learning Representations*. 2017. URL: https://arxiv.org/abs/1701.06538.

[156] Wenyun Shen et al. “Involvement of a glycerol-3-phosphate dehydrogenase in modulating the NADH/NAD+ ratio provides evidence of a mitochondrial glycerol-3-phosphate shuttle in Arabidopsis”. In: *The Plant Cell* 18.2 (2006), pp. 422–441. DOI: 10.1105/tpc.105.039750.

[157] Kwang Deok Shin, Han Nim Lee, and Taijoon Chung. "A revised assay for monitoring autophagic flux in Arabidopsis thaliana reveals involvement of AUTOPHAGY-RELATED9 in autophagy". In: *Molecules and Cells* 37.5 (2014), pp. 399–405. DOI: 10.14348/molcells.2014.0042.

[158] Noah Shinn et al. "Reflexion: Language agents with verbal reinforcement learning". In: *Advances in neural information processing systems* 36 (2023), pp. 8634–8652.

[159] Johnathon Shook et al. "Crop yield prediction integrating genotype and weather variables using deep learning". In: *Plos one* 16.6 (2021), e0252402.

[160] Jamie Shotton et al. "Textonboost: Joint appearance, shape and context modeling for multi-class object recognition and segmentation". In: *Computer Vision–ECCV 2006: 9th European Conference on Computer Vision, Graz, Austria, May 7-13, 2006. Proceedings, Part I 9*. Springer. 2006, pp. 1–15.

[161] Karen Simonyan and Andrew Zisserman. "Very deep convolutional networks for large-scale image recognition". In: *arXiv preprint arXiv:1409.1556* (2014).

[162] Aaditya Singh et al. *OpenAI GPT-5 System Card*. 2025. arXiv: 2601.03267 [cs.CL]. URL: https://arxiv.org/abs/2601.03267.

[163] Avi Singh et al. "Beyond human data: Scaling self-training for problem-solving with language models". In: *arXiv preprint arXiv:2312.06585* (2023).

[164] Nitish Srivastava et al. "Dropout: a simple way to prevent neural networks from overfitting". In: *The journal of machine learning research* 15.1 (2014), pp. 1929–1958.

[165] Kyle A Sullivan et al. "Analyses of GWAS signal using GRIN identify additional genes contributing to suicidal behavior". In: *Communications Biology* 7.1 (2024), p. 1360.

[166] Kyle A. Sullivan et al. "MENTOR: Multiplex Embedding of Networks for Team-Based Omics Research". In: *bioarXiv* (2024). DOI: 10.1101/2024.07.17.603821.

[167] Christian Szegedy et al. "Going deeper with convolutions". In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2015, pp. 1–9.

[168] Takanari Tanabata et al. "SmartGrain: High-Throughput Phenotyping Software for Measuring Seed Shape through Image Analysis". In: *Plant Physiology* 160.4 (2012), pp. 1871–1880. DOI: 10.1104/pp.112.205120.

[169] Kimi Team et al. *Kimi K2: Open Agentic Intelligence*. 2026. arXiv: 2507.20534 [cs.LG]. URL: https://arxiv.org/abs/2507.20534.

[170] Texas Tech Davis College of Agricultural Sciences & Natural Resources. *Research Farm at New Deal*. 2025. URL: https://www.depts.ttu.edu/agriculturalsciences/About/facilities/maps/resFarm.php.

[171] George Tsitsiridis et al. "CORUM: the comprehensive resource of mammalian protein complexes–2022". In: *Nucleic acids research* 51.D1 (2023), pp. D539–D545.

[172] Mason Tuite et al. "Rapid and non-destructive screening of seed components in domesticated pennycress using near-infrared spectroscopy". In: *Industrial Crops and Products* 235 (2025), p. 121752. DOI: 10.1016/j.indcrop.2025.121752.

[173] Jordan Ubbens et al. "The use of plant models in deep learning: an application to leaf counting in rosette plants". In: *Plant Methods* 14 (2018), p. 6. DOI: 10.1186/s13007-018-0273-z.

[174] Emil Uffelmann et al. "Genome-wide association studies". In: *Nature Reviews Methods Primers* 1.1 (2021), p. 59.

[175] L.C. Uzal et al. "Seed-per-pod estimation for plant breeding using deep learning". In: *Computers and Electronics in Agriculture* 150 (2018), pp. 196–204. ISSN: 0168-1699. DOI: https://doi.org/10.1016/j.compag.2018.04.024. URL: https://www.sciencedirect.com/science/article/pii/S016816991731582X.

[176] P.M. VanRaden. "Efficient Methods to Compute Genomic Predictions". In: *Journal of Dairy Science* 91.11 (2008), pp. 4414–4423. ISSN: 0022-0302. DOI: https://doi.org/10.3168/jds.2007-0980. URL: https://www.sciencedirect.com/science/article/pii/S0022030208709901.

[177] Ashish Vaswani et al. "Attention is all you need". In: *Advances in neural information processing systems* 30 (2017).

[178] A. H. Vlot et al. "The MENTOR Interpretation Agent: From Network Embeddings to Mechanistic Narratives via Retrieval-Augmented LLMs". In: *PASC 2025*. 2025.

[179] Stéfan van der Walt et al. "scikit-image: image processing in Python". In: *PeerJ* 2 (June 2014), e453. ISSN: 2167-8359. DOI: 10.7717/peerj.453. URL: https://doi.org/10.7717/peerj.453.

[180] Jiabo Wang and Zhiwu Zhang. "GAPIT version 3: boosting power and accuracy for genomic association and prediction". In: *Genomics, proteomics & bioinformatics* 19.4 (2021), pp. 629–640.

[181] Xu Wang et al. "High-throughput phenotyping with deep learning gives insight into the genetic architecture of flowering time in wheat". In: *GigaScience* 8.11 (2019), giz120.

[182] Xuezhi Wang et al. "Self-consistency improves chain of thought reasoning in language models". In: *arXiv preprint arXiv:2203.11171* (2022).

[183] Yanbing Wang et al. “The Arabidopsis APC4 subunit of the anaphase-promoting complex/cyclosome (APC/C) is critical for both female gametogenesis and embryogenesis”. In: *The Plant Journal* 69.2 (2012), pp. 227–240. DOI: 10.1111/j.1365-313X.2011.04785.x.

[184] Zhizheng Wang et al. “GeneAgent: self-verification language agent for gene-set analysis using domain databases”. In: *Nature Methods* 22.8 (2025), pp. 1677–1685.

[185] Jacob D Washburn et al. “Global genotype by environment prediction competition reveals that diverse modeling strategies can deliver satisfactory maize yield estimates”. In: *Genetics* 229.2 (2025), iyae195.

[186] Jason Wei et al. “Chain-of-thought prompting elicits reasoning in large language models”. In: *Advances in neural information processing systems* 35 (2022), pp. 24824–24837.

[187] Paul J Werbos. “Applications of advances in nonlinear sensitivity analysis”. In: *System Modeling and Optimization: Proceedings of the 10th IFIP Conference New York City, USA, August 31–September 4, 1981*. Springer. 2005, pp. 762–770.

[188] Leandro von Werra et al. *TRL: Transformers Reinforcement Learning*. https://github.com/huggingface/trl. 2020.

[189] Chunlei Wu, Ian Macleod, and Andrew I Su. “BioGPS and MyGene.info: organizing online, gene-centric information”. en. In: *Nucleic Acids Res.* 41.Database issue (Jan. 2013), pp. D561–5.

[190] Cuiling Wu et al. “A transformer-based genomic prediction method fused with knowledge-guided module”. In: *Briefings in Bioinformatics* 25.1 (2023).

[191] Xing Wu et al. “Population and adaptation history of 739 *Thlaspi arvense* natural accessions”. In: *bioRxiv* (2025). preprint. DOI: 10.1101/2025.03.21.644658.

[192] Xide Xia and Brian Kulis. "W-net: A deep model for fully unsupervised image segmentation". In: *arXiv preprint arXiv:1711.08506* (2017).

[193] Enze Xie et al. "SegFormer: Simple and efficient design for semantic segmentation with transformers". In: *Advances in neural information processing systems* 34 (2021), pp. 12077–12090.

[194] Shunyu Yao et al. "React: Synergizing reasoning and acting in language models". In: *The eleventh international conference on learning representations*. 2022.

[195] Shunyu Yao et al. "Tree of thoughts: Deliberate problem solving with large language models". In: *Advances in neural information processing systems* 36 (2023), pp. 11809–11822.

[196] Andy B. Yoo, Morris A. Jette, and Mark Grondona. "SLURM: Simple Linux Utility for Resource Management". In: *Job Scheduling Strategies for Parallel Processing*. Springer Berlin Heidelberg, 2003, pp. 44–60. DOI: 10.1007/10968987_3.

[197] Jianming Yu et al. "A unified mixed-model method for association mapping that accounts for multiple levels of relatedness". In: *Nature Genetics* 38.2 (2006), pp. 203–208. DOI: 10.1038/ng1702.

[198] Xiaoyi Zhu et al. "Important photosynthetic contribution of silique wall to seed yield-related traits in *Arabidopsis thaliana*". In: *Photosynthesis Research* 137.3 (2018), pp. 493–501. DOI: 10.1007/s11120-018-0532-x.

[199] Yuanyuan Zhu et al. "Validation and characterization of a seed number per silique quantitative trait locus qSN.A7 in rapeseed (*Brassica napus* L.)" In: *Frontiers in Plant Science* 11 (2020), p. 68. DOI: 10.3389/fpls.2020.00068.

Appendix A

Experimental Results

A.1 Experiment 1

Results from Experiment 1 go here.

A.2 Experiment 2

Results from Experiment 2 go here.

Appendix B

Supplementary Tables for Chapter 2

Table B.1: Broad-sense heritability (H^2) of all 52 U-Net-derived pod traits in the dryland environment, ordered by H^2. Estimates are from a linear mixed model with genotype as a random effect, fit on 25 replicated genotypes, and significance p is from a restricted likelihood-ratio test on the genotype variance component ($H_0 : \sigma^2_G = 0$). Traits marked * fell below the $H^2 \geq 0.05$ screen and were excluded from association testing; the remaining 34 were carried into GWAS and are reported with their GWAS status in Table 2.1.

Trait	H^2	Significance (p)
envelope-to-wing area ratio	0.461	< 0.0001
envelope-to-total area ratio	0.423	0.0002
wing-to-envelope area ratio	0.409	0.0003
envelope perimeter	0.390	0.0016
envelope area	0.389	0.0017
wing-to-total area ratio	0.387	0.0003
wing area	0.366	0.0010
wing HSV saturation	0.321	0.0022
wing perimeter	0.302	0.0060
seed area	0.297	0.0214
envelope-to-wing perimeter ratio	0.292	0.0006
wing-to-envelope perimeter ratio	0.287	0.0009
wing-to-seed area ratio	0.285	0.0077
seed perimeter	0.263	0.0334
seed-to-wing area ratio	0.247	0.0117
envelope-to-total perimeter ratio	0.230	0.0051
wing CIELAB b^*	0.227	0.0525
envelope HSV saturation	0.220	0.0418
seed count	0.218	0.0319
envelope CIELAB b^*	0.217	0.0315
seed RGB red	0.204	0.0881
envelope-to-seed area ratio	0.200	0.0501
seed CIELAB b^*	0.190	0.0627
seed-to-envelope area ratio	0.188	0.0681
seed-to-total area ratio	0.188	0.0453
seed HSV hue	0.185	0.0796
seed HSV saturation	0.155	0.1048
wing RGB blue	0.131	0.1770
seed HSV brightness	0.120	0.1518
seed CIELAB lightness	0.097	0.1672
envelope RGB green	0.089	0.2169
envelope CIELAB lightness	0.086	0.2213
seed RGB blue	0.083	0.1234
envelope HSV brightness	0.068	0.2530
seed RGB green*	0.050	0.2874
envelope HSV hue*	0.028	0.3328
wing-to-total perimeter ratio*	0.026	0.3427
envelope RGB red*	0.006	0.4167
envelope RGB blue*	0.000	0.4405

Table B.2: Composition of the nine-layer *Arabidopsis thaliana* multiplex network used for GRIN filtering and MENTOR module derivation. Each layer encodes a distinct line of biological evidence over a shared pool of 26,605 genes; "Genes" is the number of genes with at least one edge in that layer, and layers are ordered by edge count. All layers are undirected, unweighted, and contribute equally to network propagation, giving 918,640 edges in total. Network identifiers follow the exascale-generated *A. thaliana* network collection.

Layer	Source network	Network ID	Genes	Edges
PP	PPI-6merged	`AT-UU-PP-00-AA-01`	19,191	317,787
RP	Regulation-Plantregmap	`AT-UU-RP-03-AA-01`	16,014	167,851
PX	PEN-Diversity	`AT-UU-PX-01-AA-01`	19,975	145,407
KS	Knockout Similarity	`AT-UU-KS-00-AA-01`	1,841	94,952
GA	Coex Gene-Atlas	`AT-UU-GA-01-AA-01`	7,683	84,959
PY	iRF-LOOP Methylation	`AT-UU-PY-01-LF-01`	13,314	71,287
RX	Metabolic-AraCyc	`AT-UU-RX-00-AA-01`	2,857	21,524
DU	CoEvolution-DUO 0.67 trans	`AT-UU-DU-67-AA-01`	2,283	13,514
RE	Regulation-ATRM	`AT-UU-RE-00-AA-01`	789	1,359

Vita

Vita goes here...